

# Mobile & CPS — Modulo Paganelli (Reti Mobili)

Fabio Piscitelli

2026

# Indice

|          |                                                                             |           |
|----------|-----------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Reti Wireless e Standard IEEE 802.11</b>                                 | <b>1</b>  |
| 1.1      | Fondamenti delle Reti Wireless . . . . .                                    | 1         |
| 1.2      | Fisica dei Segnali Radio . . . . .                                          | 2         |
| 1.3      | Le Sfide della Propagazione . . . . .                                       | 4         |
| 1.4      | Protocolli per Reti Wireless . . . . .                                      | 5         |
| <b>2</b> | <b>Protocolli MAC e Reti Wireless: Da CSMA/CD allo Standard IEEE 802.11</b> | <b>6</b>  |
| 2.1      | I Protocolli MAC per le Reti Cablate: Un Riepilogo . . . . .                | 6         |
| 2.2      | Le Reti Wireless: Sfide e Problemi Strutturali . . . . .                    | 7         |
| 2.3      | I Protocolli MACA e MACAW: Evitare le Collisioni . . . . .                  | 9         |
| 2.4      | Lo Standard IEEE 802.11 (Wi-Fi) e la sua Architettura . . . . .             | 10        |
| 2.5      | Il Sottolivello MAC 802.11: DCF, PCF e Dinamiche CSMA/CA . . . . .          | 12        |
| <b>3</b> | <b>Mobile Networks: dalle reti cellulari al 4G/5G</b>                       | <b>15</b> |
| 3.1      | L'evoluzione delle reti cellulari: dal 1G al 6G . . . . .                   | 15        |
| 3.2      | Diffusione e standardizzazione delle reti 4G/5G . . . . .                   | 16        |
| 3.3      | Architettura di alto livello: RAN, Backhaul, Mobile Core . . . . .          | 17        |
| 3.4      | Gli elementi dell'architettura 4G LTE . . . . .                             | 17        |
| 3.5      | Separazione tra Control Plane e Data Plane . . . . .                        | 19        |
| 3.6      | Il tunneling GTP: come la rete supporta la mobilità . . . . .               | 19        |
| 3.7      | Lo stack del Data Plane LTE . . . . .                                       | 20        |
| 3.8      | Sleep modes e procedura di associazione . . . . .                           | 21        |
| 3.9      | La rivoluzione del 5G: prestazioni e use case . . . . .                     | 21        |
| 3.10     | La 5G Core e la Service Based Architecture . . . . .                        | 22        |
| 3.11     | Opzioni di deployment: Standalone vs Non-Standalone . . . . .               | 23        |
| 3.12     | La rete cellulare come "rete di reti IP" . . . . .                          | 24        |
| 3.13     | Sicurezza e autenticazione in 4G LTE: il protocollo AKA . . . . .           | 24        |
| 3.14     | Sicurezza in 5G: cosa cambia rispetto al 4G . . . . .                       | 27        |
| <b>4</b> | <b>Mobile Networks — Mobilità nelle Reti Cellulari</b>                      | <b>28</b> |
| 4.1      | Lo Spettro della Mobilità e la Home Network . . . . .                       | 28        |
| 4.2      | Approcci alla Gestione della Mobilità . . . . .                             | 30        |
| 4.3      | La Fase di Registrazione . . . . .                                          | 30        |
| 4.4      | Routing Indiretto e Routing Diretto . . . . .                               | 30        |
| 4.5      | L'Architettura Pratica: Mobilità nelle Reti 4G . . . . .                    | 31        |
| 4.6      | Mobile IP e Impatto sui Protocolli di Trasporto . . . . .                   | 32        |
| <b>5</b> | <b>Software Defined Networking (SDN)</b>                                    | <b>33</b> |
| 5.1      | L'Evoluzione dei Requisiti e la Complessità del Traffico . . . . .          | 33        |
| 5.2      | Limiti Strutturali delle Architetture Tradizionali . . . . .                | 34        |
| 5.3      | La Separazione tra Data Plane e Control Plane . . . . .                     | 34        |
| 5.4      | Il Paradigma SDN: Centralizzazione Logica e Programmabilità . . . . .       | 35        |

|           |                                                            |           |
|-----------|------------------------------------------------------------|-----------|
| 5.5       | Architettura SDN: I Tre Componenti . . . . .               | 35        |
| 5.6       | Cronologia e Sviluppi Storici . . . . .                    | 36        |
| <b>6</b>  | <b>Software Defined Networking (SDN) — Architettura</b>    | <b>38</b> |
| 6.1       | Motivazioni: Perché SDN? . . . . .                         | 38        |
| 6.2       | Il Paradigma SDN . . . . .                                 | 39        |
| 6.3       | SDN Data Plane . . . . .                                   | 42        |
| 6.4       | SDN Control Plane . . . . .                                | 45        |
| 6.5       | Forwarding e Topology Discovery . . . . .                  | 48        |
| <b>7</b>  | <b>SDN Applications e P4 Language</b>                      | <b>51</b> |
| 7.1       | B4 — La WAN Software-Defined di Google . . . . .           | 51        |
| 7.2       | Verso un Data Plane Programmabile . . . . .                | 57        |
| 7.3       | P4 Language . . . . .                                      | 57        |
| 7.4       | Service Function Chaining . . . . .                        | 60        |
| 7.5       | Sintesi . . . . .                                          | 61        |
| <b>8</b>  | <b>Network Function Virtualization</b>                     | <b>62</b> |
| 8.1       | Il problema: middlebox e rigidità hardware . . . . .       | 62        |
| 8.2       | NFV: l'idea fondamentale . . . . .                         | 63        |
| 8.3       | Vantaggi di NFV . . . . .                                  | 64        |
| 8.4       | Da funzioni di rete a servizi di rete . . . . .            | 64        |
| 8.5       | Il problema del VNF Placement . . . . .                    | 66        |
| 8.6       | L'architettura NFV . . . . .                               | 67        |
| 8.7       | Network Service Descriptor: un esempio . . . . .           | 69        |
| 8.8       | Casi d'uso . . . . .                                       | 70        |
| 8.9       | Sintesi . . . . .                                          | 70        |
| <b>9</b>  | <b>Lezione 21 — (Lab) Network Emulator con ComNetsEmu</b>  | <b>72</b> |
| 9.1       | Laboratorio: Network Emulator con ComNetsEmu . . . . .     | 72        |
| <b>10</b> | <b>Multi-Access Edge Computing (MEC)</b>                   | <b>82</b> |
| 10.1      | Dal Cloud all'Edge . . . . .                               | 82        |
| 10.2      | MEC: definizione e standard . . . . .                      | 83        |
| 10.3      | Il concetto di “edge”: dove si trova? . . . . .            | 84        |
| 10.4      | Benefici di MEC . . . . .                                  | 84        |
| 10.5      | Scenari di servizio MEC . . . . .                          | 84        |
| 10.6      | Architettura MEC . . . . .                                 | 85        |
| 10.7      | Categorie di casi d'uso . . . . .                          | 86        |
| 10.8      | MEC Service APIs . . . . .                                 | 87        |
| 10.9      | MultiAccess Traffic Steering (MTS) API . . . . .           | 87        |
| <b>11</b> | <b>Teoria dei Segnali: Introduzione e Serie di Fourier</b> | <b>89</b> |
| 11.1      | Perché la Teoria dei Segnali? . . . . .                    | 89        |
| 11.2      | Classificazione dei Segnali . . . . .                      | 90        |
| 11.3      | Trasmissione e Canale . . . . .                            | 91        |
| 11.4      | Serie di Fourier: dal Tempo alla Frequenza . . . . .       | 92        |
| 11.5      | Serie di Fourier con Periodo Arbitrario . . . . .          | 93        |
| 11.6      | Numeri Complessi e Formula di Eulero . . . . .             | 94        |
| 11.7      | Serie di Fourier in Forma Esponenziale . . . . .           | 94        |
| 11.8      | Lo Spettro di un Segnale . . . . .                         | 96        |
| 11.9      | Segnali Aperiodici e Supporto Limitato . . . . .           | 96        |
| <b>12</b> | <b>Lezione 26 — (Lab) Network Slicing</b>                  | <b>98</b> |
| 12.1      | (Lab) Network Slicing con SDN e ComNetsEmu . . . . .       | 98        |

---

|                                                                                |            |
|--------------------------------------------------------------------------------|------------|
| <b>13 Teoria dei Segnali: Trasformata di Fourier, Campionamento e DFT</b>      | <b>105</b> |
| 13.1 Trasformata di Fourier . . . . .                                          | 105        |
| 13.2 Da Segnale Analogico a Digitale: Campionamento e Quantizzazione . . . . . | 110        |
| 13.3 Il Teorema di Campionamento di Nyquist-Shannon . . . . .                  | 111        |
| 13.4 Aliasing: Il Problema del Sottocampionamento . . . . .                    | 111        |
| 13.5 Trasformata Discreta di Fourier (DFT) . . . . .                           | 114        |



# Capitolo 1

## Reti Wireless e Standard IEEE 802.11

L'introduzione alle reti senza fili segna un passaggio fondamentale nello studio delle telecomunicazioni moderne, delineando il confine tra il mondo statico dei cavi e l'ubiquità della connettività mobile. In questo capitolo esploreremo le basi fisiche e architetturali delle reti wireless, le sfide intrinseche alla propagazione dei segnali radio e le prime nozioni sui protocolli necessari per gestire l'accesso a un mezzo condiviso e invisibile. Il testo segue un approccio *top-down*, in linea con la filosofia didattica di Kurose e Ross.

---

### 1.1 Fondamenti delle Reti Wireless

#### 1.1.1 Perché le Reti Wireless

Tradizionalmente, i cavi nei computer sono stati impiegati per due scopi primari: comunicare e fornire energia. Tuttavia, l'evoluzione tecnologica ha portato alla nascita dei **Sistemi Cyber-Fisici** (*Cyber-Physical Systems*), i quali integrano capacità di calcolo, sensori, meccanismi di controllo e funzionalità di rete direttamente all'interno di oggetti fisici e infrastrutture, connettendoli a Internet e tra di loro. Poiché è materialmente impossibile cablare ogni singolo oggetto fisico, la tecnologia si è adattata di conseguenza: le reti wireless sostituiscono i cavi per le telecomunicazioni, mentre le batterie rimpiazzano i cavi per l'alimentazione elettrica.

#### 1.1.2 Elementi di una Rete Wireless

Una rete wireless è costituita da host connessi tramite collegamenti senza fili. Gli **host wireless** sono i dispositivi finali (*end-system*) che eseguono le applicazioni: smartphone, tablet, laptop, sensori, elettrodomestici smart e veicoli. Essi sono solitamente alimentati a batteria e possono essere stazionari oppure mobili.

##### Attenzione – Wireless Mobile

È fondamentale sfatare un equivoco comune: **wireless non implica necessariamente mobilità** (*wireless  $\neq$  mobile*). Un dispositivo può essere connesso senza fili pur rimanendo fisso in un punto. Mobilità e assenza di cavi sono dimensioni ortogonali.

Un ruolo cardine è giocato dalla **Stazione Base** (*Base Station*) o **Access Point**: essa è responsabile dell'invio e della ricezione dei dati da e verso gli host wireless ad essa associati. Fungerne da relè è il suo compito principale, essendo tipicamente connessa a una rete cablata più ampia (si pensi alle torri cellulari o agli Access Point IEEE 802.11). Un host wireless si definisce “associato” a una stazione base quando si trova entro la distanza utile di comunicazione e la utilizza per instradare i dati verso la rete più ampia.

I **collegamenti wireless** (*Wireless Links*) connettono i dispositivi mobili alla stazione base e sono utilizzati anche come collegamenti di backhaul. Poiché il canale è condiviso, un protocollo di accesso multiplo coordina

l'accesso al collegamento tramite tecniche come l'accesso casuale, FDMA, TDMA, CDMA o Polling. Un aspetto significativo della complessità moderna è che un singolo dispositivo wireless è dotato di diverse interfacce radio per connettersi a reti differenti: un iPhone 16, ad esempio, è equipaggiato con circa 11 radio differenti e numerose antenne, tra cui 5 diverse radio cellulari, WiFi, Bluetooth, UWB, radio satellitare, NFC e GPS.

### 1.1.3 Tassonomia e Prestazioni

Le reti wireless possono essere classificate incrociando due dimensioni: la presenza o assenza di un'infrastruttura centralizzata e il numero di salti (*hop*) necessari per la comunicazione. Le quattro categorie risultanti sono le seguenti.

Nella configurazione **single-hop con infrastruttura** l'host si connette direttamente a una stazione base (WiFi, WiMAX, cellulare 3G/4G/5G), la quale fornisce l'accesso a Internet. Tutte le operazioni di rete tradizionali — autenticazione, fatturazione, assegnazione degli indirizzi IP e routing — sono gestite dall'infrastruttura cablata. È supportato l'**Handoff**, ovvero il processo tramite cui un dispositivo mobile cambia la stazione base di riferimento senza perdere la connessione.

Nella configurazione **multi-hop con infrastruttura** l'host deve passare attraverso diversi nodi wireless prima di raggiungere il nodo connesso a Internet. Questo è il caso delle reti *Mesh* (WiFi Mesh o tecnologie radio per la sicurezza pubblica come TETRA).

Nella configurazione **single-hop senza infrastruttura** non c'è stazione base e non è necessariamente garantita una connessione a Internet. Un esempio classico è il Bluetooth.

Infine, nelle reti **multi-hop senza infrastruttura** i nodi devono fare affidamento sugli altri partecipanti per l'inoltro dei messaggi. Questo è il modello tipico di ZigBee, delle **MANET** (*Mobile Ad-hoc Networks*) e delle **VANET** (*Vehicular Ad-hoc Networks*).

| Modalità                        | Infrastruttura | Hop | Esempi               |
|---------------------------------|----------------|-----|----------------------|
| Single-hop con infrastruttura   | Sì             | 1   | WiFi, 4G/5G          |
| Multi-hop con infrastruttura    | Sì             | >1  | WiFi Mesh, TETRA     |
| Single-hop senza infrastruttura | No             | 1   | Bluetooth            |
| Multi-hop senza infrastruttura  | No             | >1  | ZigBee, MANET, VANET |

Dal punto di vista delle prestazioni esiste un chiaro compromesso tra il tasso di trasferimento dati e l'intervallo di copertura. Per ambienti indoor (10–30 m), lo standard 802.11ax può raggiungere i 14 Gbps, mentre il Bluetooth si attesta sui 2 Mbps. Spostandosi su scenari outdoor a lungo raggio (4–15 km), il 5G può fornire fino a 10 Gbps mentre il 4G LTE garantisce un'ampia copertura.

## 1.2 Fisica dei Segnali Radio

### 1.2.1 Onde Elettromagnetiche e Modulazione

La comunicazione wireless è resa possibile dall'elettromagnetismo: una variazione di corrente in un punto (trasmettitore) genera un campo elettromagnetico nello spazio che induce una corrente in una posizione remota (ricevitore).

Le **onde elettromagnetiche** uniscono il campo elettrico e quello magnetico e si propagano alla velocità della luce. Sono caratterizzate dalla **lunghezza d'onda** ( $\lambda$ ), ovvero la distanza spaziale tra due picchi d'onda successivi; dal **tempo di ciclo**, il tempo intercorso tra due picchi successivi; e dalla **frequenza**, l'inverso del tempo di ciclo (misurata in Hz), calcolabile come  $c/\lambda$ . Si aggiungono la direzionalità di propagazione e l'ampiezza variabile nel tempo.

Un concetto fondamentale per la trasmissione dei bit è la **fase** (*Phase*), che indica la posizione di un segnale periodico all'interno del suo ciclo a un dato istante  $t$  (spesso misurata in gradi, da  $0^\circ$  a  $360^\circ$ ). È possibile codificare informazioni binarie nel segnale alterando la sua fase pur mantenendo la stessa frequenza: questa tecnica è alla base della **modulazione di fase** (*Phase Shift Keying*).

### 1.2.2 Potenza, Banda e Capacità di Shannon

La forza di un segnale radio è definita dalla sua **Potenza**, ovvero l'energia trasmessa o ricevuta dall'antenna per unità di tempo. Può essere misurata in scala lineare usando i Watt (o milliwatt) o in scala logaritmica mediante i decibel (dBm), con la seguente formula di conversione:

$$P_{dBm} = 10 \cdot \log_{10} \left( \frac{P_{mW}}{1 \text{ mW}} \right)$$

Ad esempio, la potenza trasmissiva massima tipica di un dispositivo mobile è di 250 mW, che equivale a 24 dBm.

L'occupazione dello spettro è descritta dalla **Densità Spettrale di Potenza** (*Power Spectral Density* — PSD), che illustra come la potenza del segnale è distribuita in funzione della frequenza, di solito centrata attorno a una **frequenza portante** (*Carrier Frequency*). La **Larghezza di Banda** (*Bandwidth*) — da non confondersi con la “banda” intesa come capacità massima di trasmissione dati — indica l'estensione in Hz del range di frequenze utilizzato dal segnale. Ad esempio, il canale 6 di una rete WiFi utilizza 22 MHz di larghezza di banda (distribuiti tra 2.427 e 2.449 GHz, centrati a 2.438 GHz).

Nella realtà il segnale deve scontrarsi con il **Rumore** (*Noise*) e l'**Interferenza**, causata da altri trasmettitori che operano nella stessa banda. Il parametro che descrive la qualità del canale è il **Rapporto Segnale-Rumore** (*SNR* — *Signal-to-Noise Ratio*), spesso misurato in dB:

$$SNR_{dB} = 10 \cdot \log_{10} \left( \frac{\text{received signal power}}{\text{noise power}} \right)$$

Un SNR pari a 0 dB implica che segnale e rumore hanno uguale potenza. A SNR elevati è facile estrarre il segnale; i limiti operativi inferiori sono circa  $-10/-6$  dB per i cellulari e 20 dB per il WiFi.

#### Nota – La banda ISM 2.4 GHz e il problema dell'affollamento

La banda a 2.4 GHz, nota come **banda ISM** (*Industrial, Scientific and Medical*), è non licenziata ed estremamente affollata: ospita contemporaneamente dispositivi WiFi, Bluetooth, ZigBee, forni a microonde, telecomandi per garage, baby monitor e droni. Questo rende il canale particolarmente soggetto a interferenze, e spiega in parte la migrazione verso la banda a 5 GHz introdotta dagli standard più recenti.

#### Definizione – Shannon Capacity

Il lavoro di Claude Shannon del 1948 definisce il limite massimo teorico della velocità con cui i dati possono essere trasmessi su un canale in funzione della larghezza di banda e dell'SNR:

$$C = B \cdot \log_2 \left( 1 + \frac{\text{received signal power}}{\text{noise power}} \right)$$

Dove  $C$  è la capacità in bit/s e  $B$  è la larghezza di banda in Hz. Nessun sistema reale può superare questo limite, indipendentemente dalla tecnica di codifica o modulazione adottata.

#### Tip – Il trade-off tra SNR e modulazione

La capacità di Shannon scala **linearmente** con la banda, ma solo **logaritmicamente** con l'SNR. Superato un certo limite, incrementare a dismisura la potenza del segnale — e quindi l'SNR — offre vantaggi

sempre minori in termini di bitrate. Di conseguenza, la scelta pratica non è “più potenza sempre”, ma adattare lo schema di modulazione all’SNR disponibile: con SNR basso si usano modulazioni robuste come BPSK (1 Mbps), mentre con SNR elevato si può passare a modulazioni ad alto rateo come QAM-256 (8 Mbps).

## 1.3 Le Sfide della Propagazione

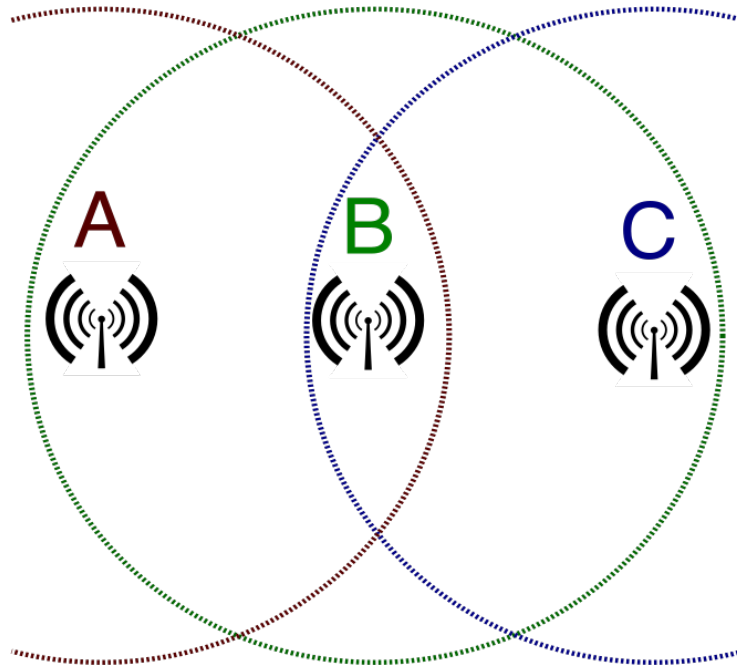
### 1.3.1 Path Loss, Terminali Nascosti e Multipath

La natura stessa delle onde radio introduce severe sfide per le architetture di rete, che non hanno equivalenti nel mondo cablato.

La prima sfida è il **Path Loss** (o *fading*): il segnale radio si attenua perdendo potenza mentre si propaga. L’attenuazione segue una proporzione generale  $1/(f \cdot d)^n$ , dove  $f$  è la frequenza,  $d$  la distanza e l’esponente  $n$  dipende dall’ambiente — vale 2 per lo spazio libero, tra 2.7 e 3.5 in aree urbane, e tra 3 e 6 all’interno degli edifici. All’aumentare della distanza, quindi, la ricezione diventa progressivamente più difficile, e frequenze più alte si attenuano più rapidamente.

#### Attenzione – Hidden Terminal Problem

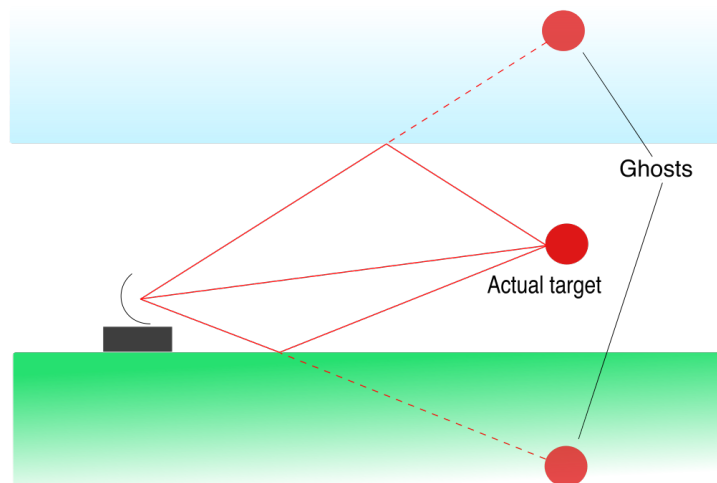
A causa dell’attenuazione con la distanza (o per la presenza di ostacoli fisici come montagne e palazzi), due nodi potrebbero non essere in grado di percepirsi a vicenda, pur essendo entrambi nel raggio di un terzo nodo intermedio. Se A e C vogliono comunicare con B, ma A e C sono fuori portata l’uno per l’altro, entrambi potrebbero iniziare a trasmettere contemporaneamente — inconsapevoli della reciproca interferenza — causando una collisione disastrosa al ricevitore B. Questo è il **Problema del Terminale Nascosto** (*Hidden Terminal Problem*), e non può essere rilevato dal classico meccanismo CSMA/CD.



**Figura 1.1.** Fonte: Wikimedia Commons — I nodi A e C non si trovano nel reciproco raggio di copertura (cerchi tratteggiati non sovrapposti), ma entrambi raggiungono B. Le trasmissioni simultanee causano una collisione invisibile ai due mittenti.

La terza sfida è il **Multipath**: il segnale trasmesso non viaggia solo in linea d’aria, ma si riflette, viene diffratto e disperso (*scattering*) dal suolo e dagli edifici circostanti. Al ricevitore giungono quindi diverse copie

del segnale originale, sfalsate temporalmente. Per evitare che questi echi interferiscano con la trasmissione successiva, i pacchetti di impulsi devono essere distanziati nel tempo. L'intervallo minimo richiesto prima di poter inviare un nuovo segnale senza sovrapposizioni si chiama **Tempo di Coerenza** (*Coherence Time* —  $T_c$ ). Il  $T_c$  è inversamente proporzionale alla frequenza utilizzata e alla velocità del ricevitore in caso di mobilità: sistemi ad alta frequenza e ricevitori in movimento richiedono guardie temporali più brevi ma più frequenti.



**Figura 1.2.** Fonte: Wikimedia Commons — Il trasmettitore (sinistra) invia il segnale verso il ricevitore (destra) attraverso un percorso diretto e più percorsi riflessi dagli edifici circostanti. Al ricevitore giungono “fantasmi” del segnale sfalsati nel tempo, che possono interferire con la trasmissione successiva.

Questi tre fenomeni influenzano direttamente il **Bit Error Rate (BER)**, ovvero la probabilità che un bit trasmesso arrivi errato. Per un dato livello fisico, aumentare la potenza riduce il BER incrementando l'SNR. L'adattamento dinamico dello schema di modulazione in base all'SNR rilevato — tecnica nota come *Adaptive Modulation and Coding* — è uno dei meccanismi fondamentali dei sistemi wireless moderni.

## 1.4 Protocolli per Reti Wireless

Le criticità fisiche descritte sopra creano sfide sistemiche che si ripercuotono su tutti i livelli dello stack protocollare: visibilità limitata della rete da parte del singolo nodo (terminali nascosti e terminali esposti), fallimenti e mobilità dei terminali, limiti intrinseci di batteria e memoria, e il problema della privacy causato dall'intercettazione dei segnali (*eavesdropping*).

Per mitigare questi limiti è necessario adottare meccanismi appositi. Il **CSMA/CD**, protocollo standard per reti cablate Ethernet, non può essere utilizzato: un nodo radio non è in grado di trasmettere e ricevere contemporaneamente per “ascoltare” una collisione nel proprio segnale. È quindi necessario un protocollo MAC alternativo. Serve inoltre la gestione dell'**Hand-off** per la transizione tra Access Point, e complessi protocolli di **Routing multi-hop** (per reti ad hoc) capaci di reagire ai cambiamenti arbitrari della topologia di vicinato (*neighborhood*).

Nello stack dei protocolli per reti wireless, mentre i livelli Applicazione e Trasporto (TCP/UDP) restano identici al mondo cablato, le fondamenta si differenziano profondamente. Il livello di Rete per ecosistemi senza infrastruttura si affida a protocolli speciali come AODV, DSR o DYMO. Il livello Datalink implementa l'accesso al mezzo tramite il protocollo **CSMA/CA** (*Carrier Sense Multiple Access with Collision Avoidance*), appositamente studiato per aggirare le trappole dei collegamenti radio.

## Capitolo 2

# Protocolli MAC e Reti Wireless: Da CSMA/CD allo Standard IEEE 802.11

Questo capitolo esplora in modo approfondito l'evoluzione dei protocolli di accesso al mezzo (MAC), partendo dalle fondamenta delle reti cablate fino ad arrivare alle complesse dinamiche delle reti wireless. Analizzeremo le sfide fisiche della trasmissione via radio, introducendo i meccanismi architetturali e i protocolli, come il CSMA/CA e la famiglia IEEE 802.11, che rendono possibile la connettività Wi-Fi moderna.

---

## 2.1 I Protocolli MAC per le Reti Cablate: Un Riepilogo

Per comprendere le reti wireless, è fondamentale fare un passo indietro e analizzare le assunzioni di base che governano i protocolli MAC (Medium Access Control) nelle tradizionali reti cablate. In questi scenari, si assume generalmente che sia disponibile un **singolo canale condiviso** per tutte le comunicazioni e che tutte le stazioni collegate possano trasmettere e ricevere su di esso. Un principio cardine è che se due o più frame vengono inviati simultaneamente, il segnale risultante sul canale risulterà incomprensibile, generando quella che viene definita una **collisione**. Nelle reti cablate, tutte le stazioni sono fisicamente in grado di rilevare queste collisioni. Nel tempo, sono stati sviluppati diversi protocolli per gestire questo accesso, partendo da ALOHA e slotted ALOHA, fino ad arrivare a CSMA e CSMA/CD.

### 2.1.1 Il Protocollo CSMA/CD

Il protocollo **CSMA/CD** (Carrier Sense Multiple Access with Collision Detection) si basa su un'idea fondamentale: “ascoltare prima di parlare”. Quando una stazione ha un frame pronto per l'invio, si mette in ascolto sul canale per verificare se qualcun altro sta già trasmettendo. Se il canale risulta occupato, la stazione attende pazientemente che diventi libero (idle). Non appena il mezzo è libero, la stazione inizia a trasmettere il suo frame. Se durante questa fase si verifica una collisione, la stazione interrompe (aborts) immediatamente la trasmissione, attende una quantità di tempo casuale e ripete l'intera procedura.

#### **Definizione – CSMA/CD — Carrier Sense Multiple Access with Collision Detection**

Protocollo di accesso al mezzo che combina l'ascolto preventivo del canale (Carrier Sense) con il rilevamento attivo delle collisioni durante la trasmissione (Collision Detection). Quando viene rilevata una collisione, la trasmissione viene interrotta immediatamente, riducendo lo spreco di banda. È il cuore dello standard IEEE 802.3 Ethernet.

Questo meccanismo di rilevamento precoce fa sì che, se due stazioni percepiscono simultaneamente il canale libero e iniziano a trasmettere, entrambe interrompano l'invio non appena rilevano la sovrapposizione dei

segnali, risparmiando così tempo prezioso e larghezza di banda. Per la sua efficienza, il CSMA/CD è stato ampiamente adottato nei sottolivelli MAC delle LAN, diventando il cuore dello standard **IEEE 802.3 Ethernet**.

### 2.1.2 Limiti Temporal e Dimensione Minima del Frame

Il comportamento del CSMA/CD impone vincoli fisici ben precisi. Definendo con  $T$  il tempo di propagazione necessario per raggiungere la stazione più lontana della rete, nel caso peggiore occorrerà un tempo pari al Round Trip Time ( $RTT$ ), ovvero  $2T$ , affinché la stazione trasmittente possa rilevare con certezza una collisione.

- Ad esempio, se la stazione A inizia a trasmettere al tempo  $t = 0$ , e la stazione B (la più remota) inizia a trasmettere poco prima che il segnale di A la raggiunga (al tempo  $t = T - \delta$ ), B rileverà la collisione quasi subito, al tempo  $t = T$ .
- Tuttavia, il fronte d'onda della collisione impiegherà un altro tempo  $T$  per viaggiare a ritroso verso A, che si accorgerà del problema solo al tempo  $t = 2T - \delta$ .

Di conseguenza, la stazione A deve obbligatoriamente monitorare il canale per l'intero intervallo di tempo  $RTT$  con gli "occhi aperti" per intercettare eventuali interferenze. Una volta che l'intero frame è stato inviato, infatti, la stazione mittente non ne conserva una copia nel buffer MAC e non controlla più il mezzo trasmissivo per eventuali collisioni. Per garantire che la trasmissione duri almeno quanto il tempo  $RTT$ , è necessario imporre un vincolo sulla **dimensione minima del frame**.

Facciamo un esempio pratico: consideriamo una rete CSMA/CD con una larghezza di banda di 10 Mbps e un tempo massimo di propagazione (inclusi i ritardi hardware) di  $25.6 \mu\text{sec}$ . Nel caso peggiore, la stazione deve trasmettere per un tempo di trasmissione minimo del frame  $T_{fr} = 2 \times T_p = 51.2 \mu\text{sec}$  per poter intercettare con successo una collisione. Traducendo questo tempo in dati, la dimensione minima del frame risulta essere:  $10 \text{ Mbps} \times 51.2 \mu\text{sec} = \mathbf{512 \text{ bit}}$ , ovvero **64 byte** (la dimensione minima effettiva adottata nell'Ethernet standard).

### 2.1.3 Il Binary Exponential Backoff

Quando avviene una collisione, come si stabilisce il tempo di attesa casuale? Il CSMA/CD utilizza l'algoritmo di **Binary Exponential Backoff**. Il tempo successivo a una collisione viene suddiviso in *contention slots* (slot di contesa), la cui lunghezza è esattamente pari al tempo di propagazione andata e ritorno nel caso peggiore ( $2T$ ).

- Dopo la prima collisione, ogni stazione coinvolta attende 0 oppure 1 slot in modo casuale prima di riprovare.
- Dopo la collisione  $i$ -esima ( $i$ ), la stazione sceglie un numero casuale  $x$  nell'intervallo  $[0, 2^i - 1]$  e salta esattamente  $x$  slot prima di un nuovo tentativo.
- Per evitare attese infinite in reti congestionate, dopo 10 collisioni consecutive l'intervallo di randomizzazione viene "congelato" a un massimo di 0..1023.
- Se si raggiungono le 16 collisioni, il livello MAC si arrende e riporta il fallimento della trasmissione ai livelli superiori.

---

## 2.2 Le Reti Wireless: Sfide e Problemi Strutturali

Nelle reti cablate il rilevamento delle collisioni in fase di trasmissione funziona perfettamente, ma nelle reti wireless questo principio fallisce a causa della natura stessa del mezzo radio. Nelle trasmissioni senza fili, la potenza del segnale decade rapidamente, diminuendo in proporzione al quadrato della distanza.

A causa di questa attenuazione, il segnale emesso dall'antenna del trasmettitore (il *self-signal*) risulta infinitamente più forte rispetto a qualsiasi altro segnale debole in arrivo da una stazione distante. Questo "acceca" il trasmettitore, rendendogli impossibile rilevare una collisione mentre sta inviando dati. Inoltre, le condizioni del canale wireless sono spazialmente diverse tra chi trasmette e chi riceve. Una collisione rilevata dal trasmettitore potrebbe non essere una collisione al ricevitore, e viceversa. Nelle reti wireless, l'unica cosa che conta veramente è l'**interferenza al ricevitore**, non al mittente. Questa limitazione genera due problemi classici della comunicazione radio: il *Terminale Nascosto* e il *Terminale Esposto*.

### 2.2.1 Il Problema del Terminale Nascosto (Hidden Terminal)

#### Definizione – Terminale Nascosto (Hidden Terminal)

Si verifica quando due o più stazioni, reciprocamente fuori dal raggio radio l'una dell'altra, trasmettono simultaneamente verso un destinatario comune. Poiché nessuna delle due stazioni riesce a rilevare la trasmissione dell'altra, entrambe credono il canale libero e avviano l'invio, causando una collisione al ricevitore che nessuna delle sorgenti è in grado di percepire.

Immaginiamo quattro stazioni allineate: A, B, C e D. A è in raggio radio con B; B è nel raggio di A e C; C è nel raggio di B e D. Supponiamo che A stia attualmente trasmettendo dei dati a B. C, desiderando trasmettere, si mette in ascolto sul mezzo. Poiché C si trova fisicamente al di fuori del raggio di copertura radio di A, non percepirà alcuna trasmissione in corso. Credendo che il canale sia libero, C avvia una trasmissione (diretta a B o a D). Il risultato è disastroso: i segnali di A e di C si sovrapporranno fisicamente in corrispondenza dell'antenna di B, causando una collisione e distruggendo i dati. In questo scenario, C è incapace di rilevare il potenziale competitore (A) ed è quindi definito come **nascosto** (hidden) rispetto alla comunicazione da A verso B. Il problema del terminale nascosto si verifica quando due o più stazioni, reciprocamente fuori raggio, trasmettono simultaneamente a un destinatario comune. Anche A, per lo stesso motivo legato alla distanza, non si accorgerà della collisione provocata da C.

### 2.2.2 Il Problema del Terminale Esposto (Exposed Terminal)

#### Definizione – Terminale Esposto (Exposed Terminal)

Si verifica quando una stazione rinuncia inutilmente a trasmettere perché rileva sul canale il segnale di un'altra stazione, pur non essendoci alcun rischio di collisione al ricevitore di destinazione. La stazione "esposta" è in grado di ascoltare una trasmissione vicina ma irrilevante, e ne viene erroneamente bloccata dall'invio di frame del tutto legittimi verso un destinatario distante.

Consideriamo la stessa topologia (A, B, C, D). Questa volta, B sta trasmettendo dati verso A, e parallelamente C desidera inviare un messaggio a D. C, mettendosi in ascolto, rileva forte e chiaro il segnale di B. Applicando ciecamente le regole del CSMA, C conclude erroneamente di non poter trasmettere verso D, per paura di creare una collisione. In realtà, se C iniziasse a trasmettere, il suo segnale raggiungerebbe D senza problemi, e le due trasmissioni (da B verso A, e da C verso D) potrebbero avvenire perfettamente in parallelo senza disturbarsi a vicenda (le loro "zone di interferenza" ai ricevitori non si sovrappongono in modo distruttivo). In questo caso, C è un terminale **esposto** alla comunicazione tra B e A. Il problema del terminale esposto impedisce a una stazione trasmittente di inviare frame del tutto legittimi a causa dell'ascolto di un'interferenza locale generata da un'altra stazione.

Il fatto che non si possa verificare lo stato del canale al ricevitore semplicemente mettendosi in ascolto dal trasmettitore rende palese la necessità di progettare protocolli MAC sensibilmente diversi da quelli delle classiche reti LAN cablate.



## 2.3 I Protocolli MACA e MACAW: Evitare le Collisioni

Per mitigare le problematiche descritte, nel 1990 Phil Karn presentò un protocollo rivoluzionario chiamato **MACA** (Multiple Access with Collision Avoidance), originariamente concepito per il “packet radio”. L’idea fondante del MACA non è quella di rilevare le collisioni, ma di prevenirle stimolando il ricevitore a inviare un breve frame di controllo prima che inizi la trasmissione dei dati veri e propri (molto più lunghi). Le stazioni vicine, sentendo questo frame di controllo, si asterranno dal trasmettere durante il successivo invio dei dati.

### 2.3.1 Il Meccanismo RTS / CTS

Il funzionamento si basa su uno scambio di messaggi denominato RTS/CTS:

1. **RTS (Request To Send):** Quando la stazione A desidera inviare dati a B, invia prima un breve pacchetto RTS indirizzato a B. Questo pacchetto non è generico, ma contiene l’ID della sorgente, l’ID della destinazione e, fattore cruciale, la lunghezza (durata) del frame dati che seguirà. Tutte le stazioni nel raggio di A (ad esempio C ed E) riceveranno questo RTS.
2. **CTS (Clear To Send):** Se B riceve l’RTS ed è pronto e desideroso di ricevere il messaggio, risponde trasmettendo un pacchetto CTS. Anche il CTS è un frame molto breve che copia e diffonde l’informazione sulla lunghezza dei dati ricevuta nell’RTS. Questo CTS verrà ascoltato da A (che ottiene così il permesso di procedere), ma anche da tutte le stazioni nel raggio di B (ad esempio D ed E).
3. Alla ricezione del frame CTS, la stazione A inizia a trasmettere il frame dati vero e proprio in totale sicurezza.

#### Tip – Insight chiave del meccanismo RTS/CTS

Il meccanismo RTS/CTS risolve entrambi i problemi topologici in modo elegante: il terminale nascosto viene messo a tacere dal CTS del ricevitore (che esso sente, anche se non ha sentito l’RTS del mittente), mentre il terminale esposto viene liberato dall’obbligo di silenzio perché sente l’RTS ma non il CTS (e quindi deduce che la ricezione avviene fuori dalla sua area di influenza). Il canale viene così “prenotato” in modo distribuito, senza coordinamento centralizzato.

Vediamo come il MACA risolve i problemi topologici precedenti:

- **Gestione dell’Esposto (C):** La stazione C ascolta l’RTS inviato da A, ma, essendo troppo lontana da B, non sentirà mai il relativo CTS. Deduce quindi di essere un terminale esposto, comprende che la ricezione avverrà lontano dalla sua area di influenza ed è quindi del tutto libera di iniziare una propria trasmissione.
- **Gestione del Nascosto (D):** La stazione D non ascolta l’RTS di A, ma riceve forte e chiaro il CTS di risposta inviato da B. D capisce immediatamente che B è in procinto di ricevere dati e che una sua trasmissione disturberebbe B. Pertanto, D si silenzia disciplinatamente fino al completamento della trasmissione del frame dati (calcolando il tempo di attesa in base alla durata dichiarata nel CTS).
- Se una stazione centrale come E ascolta sia l’RTS che il CTS, sa ovviamente di dover rimanere in silenzio per non disturbare la delicata operazione in corso.

Nel MACA, le collisioni possono ancora avvenire? Assolutamente sì, ma **solo tra pacchetti RTS** (ad esempio se C e B inviano un RTS simultaneamente verso A). Quando due RTS collidono, il destinatario non genererà alcun CTS. Il grande vantaggio è che in queste collisioni non viene perso alcun dato applicativo (NO DATA information is lost) e, dato che gli RTS sono brevissimi (circa 20 byte), il canale viene tenuto occupato e sprecato solo per una frazione di tempo minuscola. In caso di collisione di RTS, i mittenti rinviando i messaggi utilizzando anch’essi la tecnica del **Binary Exponential Backoff** mutuata dall’Ethernet.

### 2.3.2 L’Evoluzione: MACAW e l’Integrazione nel CSMA/CA

Il MACA originario è stato successivamente affinato dando vita al protocollo **MACAW** (MACA for Wireless LANs), documentato da Bharghavan et al. nel 1994. Il MACAW ha introdotto miglioramenti fondamentali:

- L'introduzione di un frame **ACK (Acknowledgement)** inviato dal ricevitore per confermare esplicitamente la ricezione corretta del frame dati.
- Meccanismi di condivisione delle informazioni (come i contatori di backoff) tra le stazioni per garantire un'allocazione equa (fair) del throughput.

La combinazione dei concetti del MACAW con l'aggiunta di una fase preventiva di "Carrier Sensing" (per evitare di inviare pacchetti RTS inutili in un canale già visibilmente saturo) ha dato vita al protocollo **CSMA/CA** (Carrier Sense Multiple Access with Collision Avoidance), che è il cuore pulsante dello standard IEEE 802.11 moderno.

### 2.3.3 Esercizio Pratico: Topologia di Rete

Per testare la comprensione del meccanismo RTS/CTS, si consideri la seguente topologia di rete a grafo: il nodo A è connesso a B e C; il nodo D forma uno snodo centrale connesso a B, C, E, F; E è connesso a D e F; F è connesso a D e E. Applicando le regole appena spiegate:

1. In una trasmissione da E verso D, le stazioni B, C e F riceverebbero il CTS da D ma non l'RTS da E (agendo come terminali nascosti da proteggere).
2. Se D ascolta un RTS da E ma non il relativo CTS (ad esempio se E sta contattando F), D deduce di essere un terminale esposto rispetto a E.
3. Similmente, se B ascolta un CTS da D senza aver sentito alcun RTS, deve silenziarsi temporaneamente. Per esplorare appieno questi casi, gli studenti possono avvalersi delle simulazioni interattive per reti 802.11 (con o senza terminali nascosti) fornite durante il corso.

## 2.4 Lo Standard IEEE 802.11 (Wi-Fi) e la sua Architettura

L'**IEEE** (Institute of Electrical and Electronics Engineers) è l'ente principale a livello mondiale per la standardizzazione delle reti locali (LAN). All'interno di questo ecosistema, il progetto IEEE 802 è gestito dall'IEEE 802 Working Group. La filosofia architetturale prevede che il sottolivello LLC (Logical Link Control) e i livelli superiori (es. Rete/IP, Trasporto/TCP o UDP, Applicazione) rimangano comuni tra le diverse tecnologie, mentre i livelli inferiori (il sottolivello MAC e il livello Fisico) vengano specializzati in base al mezzo trasmissivo adottato. Così come lo standard IEEE 802.3 definisce l'Ethernet CSMA/CD su cavo (UTP, STP, fibra), la famiglia **IEEE 802.11** definisce lo standard per le reti Wireless LAN (universalmente note con il marchio commerciale **Wi-Fi**) che utilizzano la trasmissione radio. Altri standard noti includono l'802.15 (Wireless PAN, come il Bluetooth) e l'802.16 (Broadband wireless, WiMAX).

### 2.4.1 L'Evoluzione della Famiglia 802.11

Tutte le reti 802.11 operano sfruttando canali radio collocati nelle bande **ISM** (Industrial, Scientific, and Medical) o **U-NII** (Unlicensed National Information Infrastructure), comunemente nelle frequenze dei 900 MHz, 2.4 GHz e 5 GHz, garantendo progressivamente decine fino a migliaia di Mbit/s di capacità. Ripercorriamo l'evoluzione storica del livello fisico (PHY):

- **IEEE 802.11 (Legacy mode):** Rilasciato originariamente nel 1997 e chiarito nel 1999. Operava nella banda dei 2.4 GHz offrendo data rate molto bassi (1-2 Mbps) implementati tramite raggi infrarossi (IR) o frequenze radio. Avendo troppi gradi di libertà implementativi, poneva severi problemi di interoperabilità ed è oggi considerato obsoleto.
- **IEEE 802.11a:** Rilasciato nel 1999, è stato pioniere dell'uso della banda a 5 GHz (U-NII), offrendo un throughput tipico di 23 Mbps e un tetto massimo di 54 Mbps.

- **IEEE 802.11b:** Contemporaneo alla versione ‘a’ (1999), divenne lo standard di massa iniziale. Operando a 2.4 GHz (massimo 11 Mbps), implementava schemi di modulazione specifici per limitare le forti interferenze in quella banda affollata (telefoni cordless, microonde, ecc.).
- **IEEE 802.11g:** Introdotto nel 2003, mantenne l’uso della banda a 2.4 GHz ma aumentò la velocità fino a 54 Mbps.
- **IEEE 802.11n (WiFi 4):** Rilasciato nel 2009, ha introdotto il rivoluzionario supporto alla tecnologia **MIMO** (multiple-input multiple-output). Usando più antenne in trasmissione e ricezione contemporaneamente sia a 2.4 GHz che a 5 GHz, ha spinto il data rate massimo fino a 248 Mbps (e teoricamente 600 Mbps) estendendo la portata.
- **IEEE 802.11ac (WiFi 5):** Standardizzato nel 2013, si concentra esclusivamente sui 5 GHz, superando la barriera del gigabit (fino a 1.3 Gbps per flussi base, fino a 3.47 Gbps massimi).
- **IEEE 802.11ax (WiFi 6):** Rilasciato intorno al 2019/2020, supporta 2.4 GHz, 5 GHz e introduce i 6 GHz. Progettato per efficienza estrema in ambienti densi, raggiunge fino a 10-14 Gbps, con sensibili miglioramenti nel consumo energetico dei dispositivi e nei protocolli di sicurezza.
- (*Menzioni speciali*): Versioni come 802.11af (2014, che sfrutta le bande TV inutilizzate) e 802.11ah (2017, per applicazioni a 900 MHz a lunghissimo raggio fino a 1 Km) si rivolgono ad ambiti specialistici o all’IoT.

| Standard                 | Anno | Max Data Rate | Copertura (Range) | Frequenza             |
|--------------------------|------|---------------|-------------------|-----------------------|
| <b>802.11b</b>           | 1999 | 11 Mbps       | 30 m              | 2.4 GHz               |
| <b>802.11g</b>           | 2003 | 54 Mbps       | 30 m              | 2.4 GHz               |
| <b>802.11n (WiFi 4)</b>  | 2009 | 600 Mbps      | 70 m              | 2.4, 5 GHz            |
| <b>802.11ac (WiFi 5)</b> | 2013 | 3.47 Gbps     | 70 m              | 5 GHz                 |
| <b>802.11ax (WiFi 6)</b> | 2020 | 14 Gbps       | 70 m              | 2.4, 5 GHz            |
| <b>802.11af</b>          | 2014 | 35-560 Mbps   | 1 Km              | Bande TV (54-790 MHz) |
| <b>802.11ah</b>          | 2017 | 347 Mbps      | 1 Km              | 900 MHz               |

Tabella riassuntiva delle specifiche IEEE 802.11. Tutti utilizzano CSMA/CA e supportano configurazioni base-station o reti ad-hoc.

### 2.4.2 Elementi Architettureali: BSS ed ESS

Dal punto di vista della struttura della rete, il blocco logico fondamentale dell’802.11 è il **BSS (Basic Service Set)**. Un BSS è semplicemente un gruppo di stazioni wireless in grado di comunicare tra loro sotto il controllo di una singola funzione di coordinamento. Esistono due modalità operative principali per un BSS:

1. **Rete Ad-Hoc:** L’infrastruttura di controllo è assente. Le stazioni comunicano direttamente e paritetivamente (peer-to-peer) l’una con l’altra.

2. **Rete Infrastrutturata:** È presente una stazione base specializzata chiamata **AP (Access Point)**. Tutte le comunicazioni, anche tra due host della stessa cella, vengono obbligatoriamente incanalate e gestite attraverso l'AP.

Per formare reti più vaste, più BSS vengono interconnessi tra loro creando un **ESS (Extended Service Set)**. In questo scenario, gli Access Point fungono da ponti e forniscono connettività inter-cella instradando il traffico attraverso un'infrastruttura fissa (solitamente cablata) denominata **DS (Distribution System)**.

### 2.4.3 Selezione del Canale e Meccanismi di Associazione

Lo spettro radio assegnato al Wi-Fi è logicamente suddiviso in vari canali operanti a frequenze leggermente diverse. L'amministratore di rete configura manualmente o automaticamente la frequenza in cui opera un determinato AP. Un problema tipico di progettazione si verifica quando AP limitrofi (vicini fisicamente) vengono configurati sullo stesso canale, generando interferenze distruttive tra le celle.

Quando un dispositivo wireless (es. smartphone o laptop) arriva in un'area coperta, deve associarsi a un AP prima di poter navigare. Questo avviene tramite una fase di scansione dei canali (scansione alla ricerca della rete e del suo **SSID** - Service Set Identifier, oltre all'indirizzo MAC dell'AP noto come BSSID). Esistono due modalità di scansione:

1. **Scansione Passiva (Passive Scanning):** L'host si mette in ascolto silente dei *beacon frames* periodicamente e automaticamente trasmessi dagli AP circostanti. Una volta ricevuti i beacon, l'host sceglie tipicamente l'AP il cui segnale arriva con la massima potenza (highest signal strength). A questo punto, l'host invia all'AP selezionato un frame di *Association Request*, e attende in risposta un frame di *Association Response*.
2. **Scansione Attiva (Active Scanning):** L'host assume l'iniziativa e invia attivamente in broadcast un *Probe Request frame* su vari canali. Gli AP nel raggio d'azione che ricevono la richiesta rispondono con *Probe Response frames*. L'host valuta le risposte, sceglie l'AP e avvia il classico handshake inviando l' *Association Request* e ricevendo la *Association Response*.

Completata l'associazione, la stazione segue tipicamente le normali procedure di autenticazione di sicurezza e avvia un client DHCP per ottenere un indirizzo IP valido nella sottorete gestita dall'AP.

---

## 2.5 Il Sottolivello MAC 802.11: DCF, PCF e Dinamiche CSMA/-CA

Il sottolivello MAC dello standard IEEE 802.11 è diviso in due funzioni architetturali ben distinte che determinano le modalità di accesso al mezzo: la DCF e la PCF.

- **PCF (Point Coordination Function):** È un modulo opzionale che si posiziona concettualmente "sopra" il livello DCF e gestisce servizi *contention-free* (ovvero privi di collisioni) in modo centralizzato, controllando rigorosamente i turni di trasmissione degli host.
- **DCF (Distributed Coordination Function):** È il fondamento universale del MAC 802.11. È sempre disponibile in ogni rete (ad-hoc o infrastrutturata) e gestisce l'accesso *contention-based* tramite il protocollo **CSMA/CA**. Essendo un sistema decentralizzato e distribuito, le collisioni sono ovviamente una possibilità reale.

### 2.5.1 Il Carrier Sensing: Fisico e Virtuale (NAV)

Nel CSMA/CA, prima di trasmettere, una stazione deve necessariamente accertarsi che il canale sia libero eseguendo il **Carrier Sensing**. Questo avviene su due livelli interconnessi:

1. **Sensore Fisico (Physical CS):** L'hardware analizza la frequenza per rilevare energia o intercettare l'arrivo di un segnale elettromagnetico, stabilendo se c'è attività nel canale dovuta ad altre fonti.

2. **Sensore Virtuale (Virtual CS):** È il meccanismo logico software. Il protocollo prevede l'inserimento di precise informazioni sulla durata temporale della comunicazione nell'header dei primissimi byte di frame come RTS, CTS e pacchetti Dati generici. Ascoltando queste durate, ogni stazione aggiorna una variabile locale chiamata **NAV** (Network Allocation Vector). Il NAV funge da timer: indica al sistema per quanto tempo il canale sarà "virtualmente occupato" e quanto bisogna aspettare prima di poter persino pensare di campionare nuovamente lo stato fisico del mezzo.

Basta che uno solo dei due sensori (Fisico o Virtuale/NAV) dia esito "occupato", affinché la stazione consideri l'intero canale indisponibile (busy).

### 2.5.2 Gli Interframe Spaces (IFS) e le Priorità

Nel mondo 802.11, il concetto di "canale libero" è strettamente legato ai tempi. L'accesso prioritario al mezzo è regolamentato imponendo l'uso di periodi obbligatori di attesa e inattività chiamati **IFS** (Interframe Space). Lo standard definisce durate precise:

- **DIFS (Distributed IFS):** È un intervallo temporale "lungo". È il tempo minimo di default che una stazione normale deve attendere (misurando canale ininterrottamente libero) prima di poter avviare una trasmissione regolare.
- **SIFS (Short IFS):** È un intervallo temporale "brevissimo". Viene utilizzato esclusivamente come tempo di attesa prima di inviare pacchetti critici e ad altissima priorità, come i CTS o i messaggi di conferma ACK. Poiché rigorosamente **SIFS < DIFS**, la stazione che sta completando un handshake (es. sta per inviare un ACK) riuscirà sempre a prendere possesso del canale prima che le altre stazioni in attesa finiscano di contare il proprio lungo periodo DIFS, garantendo l'assenza di interruzioni durante le transazioni.

### 2.5.3 L'Algoritmo CSMA/CA in Azione

Sintetizziamo i passaggi della DCF per trasmettitore e ricevitore:

1. **Attesa Iniziale:** Se la stazione mittente percepisce il canale costantemente libero per un periodo pari almeno al DIFS, trasmette l'intero frame dati (ricordiamo, senza collision detection).
2. **Backoff Randomico:** Se invece il canale risulta occupato (fisicamente o tramite NAV), la stazione fa partire un tempo di backoff casuale. Questo timer scorre alla rovescia (counts down) solamente mentre il canale risulta libero; si congela istantaneamente se un'altra stazione inizia a trasmettere. Quando il timer raggiunge lo zero (ed è trascorso anche il DIFS), la stazione trasmette.
3. **Il ruolo dell'ACK:** Ricevuto correttamente un frame, il destinatario attende il breve tempo SIFS e risponde immediatamente con un pacchetto ACK per confermare l'avvenuto recapito (l'ACK è vitale in wireless a causa delle perdite naturali e dei terminali nascosti).
4. **Gestione delle Collisioni:** Cosa succede se l'ACK non arriva? Il mittente assume che si sia verificata una collisione (o una perdita per attenuazione). Si innesca la penalità: il mittente incrementa l'intervallo casuale di backoff (raddoppiandolo ad ogni tentativo fallito secondo la logica Binary Exponential Backoff), aspetta il DIFS e ritenta la procedura.

*Nota sull'inefficienza delle collisioni semplici:* Se avviene una collisione e l'RTS/CTS non viene impiegato, poiché nessuna stazione è in grado di interrompere la trasmissione a metà, l'intero frame dati verrà completamente sprecato, "sprecando" una massiccia porzione di larghezza di banda, specialmente se il frame era molto grande.

### 2.5.4 RTS e CTS nello Standard IEEE 802.11

Per evitare l'enorme spreco di risorse causato dalle collisioni sui frame dati primari, l'implementazione 802.11 permette di "prenotare" (reserve) il mezzo trasmissivo utilizzando i mini-pacchetti RTS e CTS (descritti nel MACA). L'host invia prima l'RTS in modalità CSMA; l'AP di destinazione diffonde in broadcast un CTS per notificare l'autorizzazione e zittire tutte le altre stazioni (che aggiornano il loro NAV), permettendo poi

la trasmissione incontrastata dei dati. Anche qui, se due RTS collidono per il possesso (reservation collision), solo i pacchettini corti vanno persi.

Gli amministratori di rete possono configurare la modalità DCF per gestire l'RTS/CTS in tre modi differenti (RTS Threshold):

1. **Mai (Never):** Non si usano RTS/CTS. Questa impostazione è eccellente per reti con carico di traffico estremamente leggero, in cui il ritardo aggiunto dallo scambio RTS/CTS sarebbe inutile e inefficiente.
2. **Soglia di dimensione:** RTS/CTS viene abilitato unicamente per messaggi “lunghi”, ovvero quando la dimensione del pacchetto supera un certo limite (RTS Threshold) calcolato dall'amministratore.
3. **Sempre (Always):** L'uso dell'RTS/CTS è imposto per ogni trasmissione. È l'impostazione raccomandata in condizioni di altissimo carico e densità sul mezzo wireless per minimizzare le disastrose probabilità di sovrapposizione e la gestione dei nodi nascosti.

---

#### Nota – Riepilogo dei Concetti Chiave

Nelle LAN cablate (802.3) il rilevamento delle collisioni avviene direttamente durante la trasmissione (CSMA/CD), bloccando tempestivamente l'invio. Nelle WLAN (802.11) questo è fisicamente precluso a causa dell'attenuazione del segnale radio, per cui si adottano tecniche di “Avoidance” preventivo basate su ACK obbligatori (CSMA/CA). I problemi strutturali del mezzo radio — terminale nascosto ed esposto — derivano dall'impossibilità di verificare lo stato del canale al ricevitore ascoltando dal trasmettitore; entrambi vengono gestiti tramite il meccanismo di prenotazione del canale MACA/MACAW integrato nello standard 802.11 attraverso lo scambio RTS/CTS. Il Virtual Carrier Sensing e il NAV garantiscono il blocco totale delle trasmissioni indesiderate attraverso timer logici interni che propagano in modo distribuito la durata delle comunicazioni in corso. L'architettura unificata dello standard IEEE 802.11 — basata su BSS e ESS, tempi di attesa differenziati (SIFS vs DIFS) e Backoff Esponenziale — opera sulle bande ISM/U-NII libere da licenze, consentendo la connettività Wi-Fi moderna in una vasta gamma di scenari, dalle reti ad-hoc alle infrastrutture enterprise.

## Capitolo 3

# Mobile Networks: dalle reti cellulari al 4G/5G

Questa lezione introduce i fondamenti dell'architettura delle reti cellulari moderne, ripercorrendo l'evoluzione storica dal 1G fino al 5G e proiettandosi verso il 6G. Il filo conduttore è capire come ogni generazione abbia trasformato non solo la velocità di trasmissione, ma soprattutto il modello architetturale: dalla commutazione a circuito puramente analogica del 1G, alla **all-IP core** del 4G LTE, fino alla **Service Based Architecture** cloud-native del 5G. Lungo questo percorso emergono i meccanismi che rendono le reti cellulari peculiari rispetto a Internet cablata: la mobilità come servizio nativo di prima classe, il tunneling per preservare gli indirizzi IP durante gli spostamenti, l'autenticazione mutua basata sulla SIM e la separazione netta tra piano di controllo e piano dati.

### Nota – Riferimenti bibliografici

Le slide sono un libero adattamento del libro “**Computer Networking: A Top-Down Approach**” (8<sup>a</sup> edizione, 2020) di **James F. Kurose** e **Keith W. Ross**. Per l'architettura 5G il riferimento è “**5G Mobile Networks: A Systems Approach**” di **Larry Peterson** e **Oguz Sunay** (CC BY-NC-ND 4.0). Il quadro evolutivo verso il 6G riprende: Zakria Qadir et al., “*Towards 6G Internet of Things: Recent advances, use cases, and open challenges*”, ICT Express, Elsevier, Giugno 2023.

### 3.1 L'evoluzione delle reti cellulari: dal 1G al 6G

Le reti cellulari hanno attraversato in quattro decenni una trasformazione che le ha portate da semplici sistemi di telefonia analogica a infrastrutture distribuite per l'Internet of Things globale. Ogni “generazione” non rappresenta solo un salto di velocità ma un ripensamento dell'architettura.

Il **1G** degli anni '80 (standard AMPS, TACS) trasportava solo voce in formato analogico, multiplexato in frequenza (FDMA), con copertura e sicurezza molto limitate. Negli anni '90 il **2G** introduce la voce digitale (GSM, CDMA) con multiplexing combinato TDM+FDMA e gli SMS; con l'estensione **2.5G/GPRS** (General Packet Radio Service) si affaccia per la prima volta il concetto di **commutazione a pacchetto per i dati**, accanto alla commutazione a circuito che continua a gestire la voce. Il **3G** degli anni 2000 (UMTS, CDMA2000, HSPDA) porta l'accesso Web in mobilità: la rete voce resta a circuito ma in parallelo opera una rete dati a pacchetto.

La svolta architetturale arriva con il **4G** (LTE Advanced, anni 2010): tutto il traffico — voce inclusa — viaggia su un **All-IP core**, tunnelizzato come pacchetti IP dalla Base Station al gateway. Cade la separazione fra rete voce e rete dati e si introduce la separazione fra piano di controllo e piano dati. Il **5G** degli anni 2020 (5G NR) ridisegna il core in chiave cloud-native (**Service Based Architecture**), introduce il **network slicing**

per partizionare logicamente la rete, e sfrutta sia frequenze sub-6 GHz sia onde millimetriche (mmWave). Il **6G**, atteso intorno al 2030, è già oggetto di ricerca e punta a un ecosistema “fully-connected” potenziato da AI con architetture *cell-less*.

| Gen         | Anni          | Standard              | Velocità        | Accesso        | Voice switch | Data switch | Caratteristiche                                  |
|-------------|---------------|-----------------------|-----------------|----------------|--------------|-------------|--------------------------------------------------|
| <b>1G</b>   | 1980s         | AMPS, TACS            | 2.4 kbps        | FDMA           | Circuit      | Circuit     | Voce analogica, copertura limitata               |
| <b>2G</b>   | 1990s         | GSM, CDMA, EDGE       | 64 kbps         | TDM+FDMA       | Circuit      | Circuit     | Voce digitale + SMS                              |
| <b>2.5G</b> | fine '90s     | GPRS                  | —               | —              | Circuit      | Packet      | Prima commutazione a pacchetto sui dati          |
| <b>3G</b>   | 2000s         | UMTS, CDMA2000, HSPDA | fino a 2 Mbps   | CDMA           | Circuit      | Packet      | Web in mobilità                                  |
| <b>4G</b>   | 2010s         | LTE Advanced          | 100–1000 Mbps   | OFDMA          | Packet       | Packet      | All-IP core, tunneling, separazione control/data |
| <b>5G</b>   | 2020s         | 5G NR                 | fino a ~20 Gbps | OFDMA + mmWave | Packet       | Packet      | SBA, network slicing, sub-6 GHz + mmWave         |
| <b>6G</b>   | 2030s (prev.) | —                     | ~1 Tbps         | —              | —            | —           | Cell-less, AI-native, fully-connected            |

#### Tip – La traiettoria di fondo

A ogni generazione la rete diventa più simile a Internet: prima si introduce la commutazione a pacchetto sui dati (2.5G), poi anche sulla voce (4G), poi si modularizza il core in microservizi (5G). La direzione è quella di una rete cellulare **indistinguibile da un cloud distribuito**, dove la mobilità è una feature applicativa e non un'eccezione.

## 3.2 Diffusione e standardizzazione delle reti 4G/5G

Le reti 4G/5G sono oggi la soluzione standard per l'Internet mobile su area geografica ampia. Già nel 2019 i dispositivi connessi via banda larga mobile superavano quelli a banda larga fissa con un rapporto di **5 a 1**. La disponibilità del 4G ha raggiunto il **97% del tempo in Corea** e il **90% negli Stati Uniti**, con velocità tipiche di **20 Mbps e oltre**. La standardizzazione tecnica è curata dal consorzio **3GPP** (3rd Generation

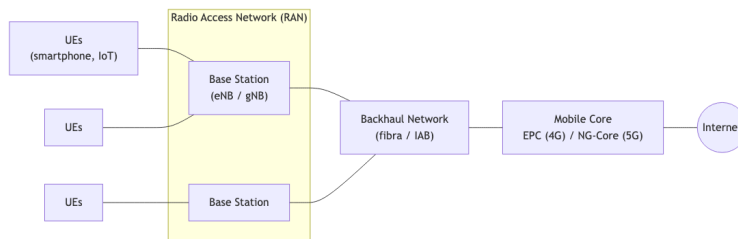


Partnership Project), che definisce in particolare lo standard **LTE (Long-Term Evolution)** per il 4G e il **5G NR (New Radio)** per il 5G.

### 3.3 Architettura di alto livello: RAN, Backhaul, Mobile Core

L'architettura di una rete cellulare 4G/5G si scompone in tre blocchi funzionali che collaborano per portare un pacchetto IP dal dispositivo dell'utente fino a Internet (e viceversa):

- **Radio Access Network (RAN)** — è la “rete d'accesso” radio: una collezione distribuita di **Base Station** (stazioni base) che gestiscono lo spettro radio condiviso fra i dispositivi all'interno della loro cella di copertura.
- **Backhaul Network** — interconnette la RAN con il Mobile Core. Tipicamente cablata in fibra ottica, ma sono in arrivo soluzioni wireless integrate (Integrated Access Backhaul, IAB).
- **Mobile Core Network** — fornisce connettività IP per dati e voce, garantisce i requisiti di **QoS**, traccia la mobilità per assicurare un servizio ininterrotto e gestisce billing & charging. Si chiama **Evolved Packet Core (EPC)** in 4G e **NG-Core** in 5G.



**Figura 3.1.** I tre blocchi dell'architettura cellulare. La RAN gestisce lo spettro, il backhaul è il “trasporto” verso l'interno della rete, il Mobile Core fa da gateway verso Internet e custodisce le funzioni di mobilità, autenticazione e charging.

### 3.4 Gli elementi dell'architettura 4G LTE

L'architettura 4G LTE è comunemente chiamata **all-IP Enhanced Packet Core (EPC)**. Vale la pena entrare nel dettaglio dei suoi componenti, perché molti concetti sopravvivono inalterati anche in 5G (cambiando solo nome o granularità).

#### 3.4.1 User Equipment (UE)

##### Definizione – UE — User Equipment

Con **UE** si intende qualsiasi dispositivo terminale dotato di radio 4G LTE: smartphone, tablet, laptop, dispositivo IoT. Implementa l'intero stack Internet a 5 livelli. La sua identità globale è l'**IMSI** (International Mobile Subscriber Identity), un codice a 64 bit memorizzato sulla **SIM** (Subscriber Identity Module). L'IMSI identifica univocamente l'abbonato nel sistema cellulare mondiale, codificando la nazione e il carrier “home”.

#### 3.4.2 Base Station (eNode-B)

La Base Station si trova al “margine” (*edge*) della rete del carrier: gestisce le risorse radio wireless e i dispositivi all'interno della propria **cella**, e coordina l'autenticazione del dispositivo interfacciandosi con gli altri elementi del core. Rispetto a un Access Point Wi-Fi, però, il suo ruolo è molto più attivo:

- coordina gli **handover** del dispositivo tra celle adiacenti, mantenendo le sessioni in corso;
- collabora con le Base Station vicine per **minimizzare le interferenze radio**;
- crea **tunnel IP specifici per ogni dispositivo** verso i gateway del core;
- gestisce direttamente la mobilità intercella.

In gergo LTE è chiamata **eNode-B** (evolved NodeB).

### 3.4.3 Mobility Management Entity (MME)

L'MME è il “cervello” del piano di controllo. Coordina l'**autenticazione bidirezionale** (dispositivo → rete e rete → dispositivo) interfacciandosi con l'HSS della rete *home* dell'utente. Si occupa inoltre della gestione del dispositivo: **handover** fra celle, **tracking/paging** della posizione, e setup dei percorsi (tunneling) verso S-GW e P-GW.

### 3.4.4 Home Subscriber Service (HSS)

#### Definizione – HSS — Home Subscriber Service

Database centrale che contiene tutte le informazioni sugli abbonati. Memorizza i dati dei dispositivi per i quali la rete in cui si trova l'HSS è la **home network** (“rete di casa”), e collabora strettamente con l'MME nelle procedure di autenticazione.

### 3.4.5 Serving Gateway (S-GW)

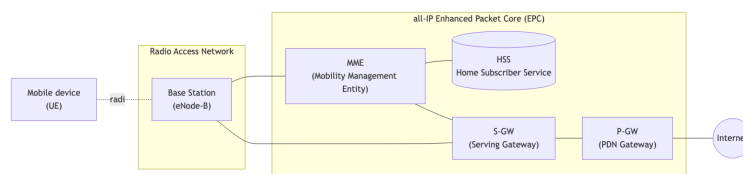
L'S-GW si trova **sul percorso dei dati** fra dispositivo mobile e Internet. Si occupa dell'inoltro dei pacchetti IP da/verso la RAN e funge da **Local Mobility Anchor**: quando un utente si sposta da un eNode-B a un altro all'interno della stessa rete, l'S-GW fa da “ancora locale” della sessione, cosicché l'handover risulti trasparente ai livelli superiori.

### 3.4.6 PDN Gateway (P-GW)

Il **Packet Data Network Gateway** connette il core 4G alle reti dati esterne — tipicamente Internet o reti aziendali private. È l'**ultimo elemento LTE** che un datagramma incontra prima di entrare nella Internet pubblica. Si comporta come un router gateway “normale”, ma con responsabilità aggiuntive: **NAT**, applicazione di policy, traffic shaping, charging.

#### Nota – S-GW e P-GW co-localati

Nelle implementazioni reali S-GW e P-GW possono essere co-localizzati sullo stesso nodo fisico. Le specifiche 3GPP permettono molte configurazioni: ad esempio un'unica coppia MME/P-GW può servire un'intera area metropolitana, mentre gli S-GW vengono distribuiti su ~10 siti edge cittadini, ciascuno a servizio di ~100 stazioni base.



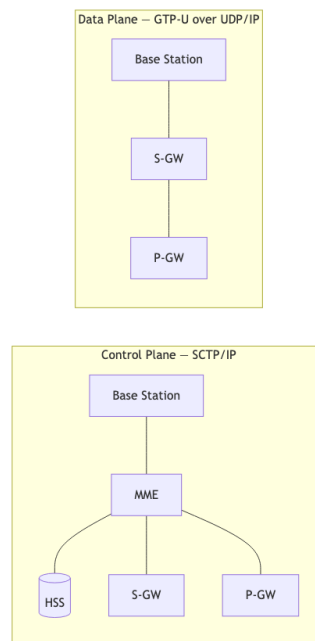
**Figura 3.2.** Topologia logica del 4G LTE. In rosso scuro le interfacce di controllo (verso MME e HSS), il piano dati attraversa BS → S-GW → P-GW → Internet.

### 3.5 Separazione tra Control Plane e Data Plane

Una caratteristica distintiva dell'architettura LTE è la **netta separazione fra Piano di Controllo e Piano Dati**. Questa scelta — ereditata e amplificata in 5G — riflette principi di Software Defined Networking: chi prende le decisioni (MME, HSS) è disaccoppiato da chi inoltra i pacchetti (S-GW, P-GW).

La Base Station si trova esattamente sull'incrocio dei due piani, e inoltra **sia pacchetti di controllo sia pacchetti utente** fra Mobile Core e UE.

- Nel **Piano di Controllo**, i pacchetti viaggiano tunnelizzati su **SCTP/IP** (Stream Control Transmission Protocol, RFC 4960). SCTP è un'alternativa affidabile a TCP, progettata per trasportare la **segnalazione** delle reti telefoniche; gestisce le procedure di registrazione, tracking e autenticazione.
- Nel **Piano Dati**, i pacchetti utente sono incapsulati nei tunnel **GTP-U** (General Packet Radio Service Tunneling Protocol — User plane), un protocollo 3GPP che opera **sopra UDP**.



**Figura 3.3.** Separazione Control Plane / Data Plane in LTE. Il control plane (sopra) trasporta segnalazione su SCTP fra BS, MME, HSS e gateway; il data plane (sotto) trasporta i pacchetti utente in tunnel GTP-U attraverso BS → S-GW → P-GW.

#### Tip – Perché la separazione è cruciale

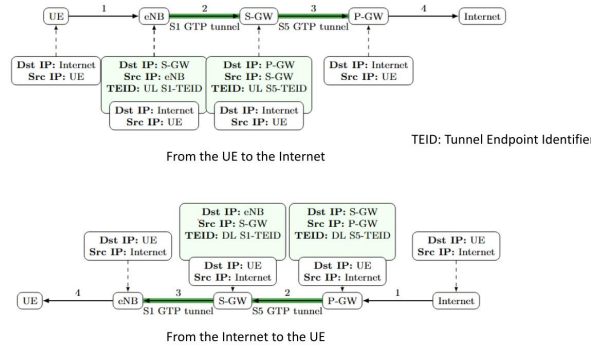
Disaccoppiare control e data plane permette di scalarli indipendentemente: si possono aggiungere S-GW per assorbire più traffico utente senza toccare MME/HSS. Inoltre apre la strada alla **virtualizzazione** delle funzioni di controllo (NFV), che in 5G diventa il principio architetturale dominante.

### 3.6 Il tunneling GTP: come la rete supporta la mobilità

Il datagramma originario emesso dall'UE viene **incapsulato con GTP** e spedito dentro un datagramma **UDP** verso l'S-GW; l'S-GW lo *re-incapsula* e lo invia al P-GW, che lo “scarta” l'incapsulamento e lo immette su Internet. Le sessioni vengono identificate dal **TEID** (Tunnel Endpoint Identifier).

### Tip – Perché il tunneling GTP abilita la mobilità

Il tunneling GTP risolve il problema della mobilità IP in modo molto elegante: incapsulando il datagramma dell'utente dentro un nuovo pacchetto UDP/GTP, gli **indirizzi IP visibili a Internet rimangono fissi** (quelli dell'S-GW e del P-GW). Quando l'utente cambia cella o si sposta in un'altra area, basta **aggiornare gli endpoint del tunnel** — il dispositivo mantiene il suo indirizzo IP e le sessioni TCP attive non vengono interrotte.

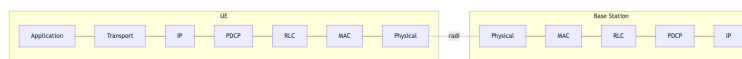


**Figura 3.4.** Composizione degli header GTP nel doppio senso del traffico. **UE → Internet:** il pacchetto originale  $Src=UE$ ,  $Dst=Internet$  viene incapsulato dall'eNB in un GTP+UDP+IP con  $Src=eNB$ ,  $Dst=S-GW$ ,  $TEID=UL-S1-TEID$  (tunnel S1); l'S-GW lo ri-incapsula con  $Src=S-GW$ ,  $Dst=P-GW$ ,  $TEID=UL-S5-TEID$  (tunnel S5); il P-GW rimuove gli incapsulamenti e immette il datagramma originale su Internet. **Internet → UE:** il P-GW riceve  $Src=Internet$ ,  $Dst=UE$ , lo incapsula con  $Src=P-GW$ ,  $Dst=S-GW$ ,  $TEID=DL-S5-TEID$ ; l'S-GW lo ri-incapsula verso l'eNB con  $Src=S-GW$ ,  $Dst=eNB$ ,  $TEID=DL-S1-TEID$ ; l'eNB consegna il datagramma all'UE via radio.

## 3.7 Lo stack del Data Plane LTE

A livello radio (cioè fra UE e Base Station), lo stack del piano dati LTE è composto da quattro livelli specifici, che si frappongono fra il livello IP “ordinario” e l'aria:

1. **Packet Data Convergence Protocol (PDCP)** — compressione degli header e crittografia.
2. **Radio Link Control (RLC)** — frammentazione e riassettaggio dei pacchetti IP, trasferimento affidabile a livello di collegamento (ACK/NACK).
3. **Medium Access Control (MAC)** — richiesta e uso degli slot di trasmissione radio, rilevazione e correzione degli errori.
4. **Physical Layer** — modulazione e accesso al canale: FDM e TDM, e in particolare **OFDM** (Orthogonal Frequency Division Multiplexing) in downlink, le cui sottoportanti “ortogonali” minimizzano l'interferenza fra canali. Ogni dispositivo attivo riceve **due o più time slot da 0.5 ms** distribuiti su **12 frequenze**; lo scheduler non è standardizzato (lo decide l'operatore) e rende possibili velocità nell'ordine di **centinaia di Mbps per dispositivo**.



**Figura 3.5.** Stack del data plane LTE sul primo hop UE BS. Sopra l'IP “classico” si innestano PDCP, RLC, MAC e Physical, che insieme forniscono compressione, crittografia, riassettaggio, scheduling e modulazione OFDM.

## 3.8 Sleep modes e procedura di associazione

Per preservare la batteria, i dispositivi LTE — analogamente a Wi-Fi e Bluetooth — possono mettere la radio in **sleep**:

- **Light sleep** dopo centinaia di millisecondi di inattività; il dispositivo si risveglia periodicamente (ogni ~100 ms) per controllare se ci sono trasmissioni in arrivo.
- **Deep sleep** dopo 5–10 secondi di inattività. In deep sleep il dispositivo potrebbe **cambiare cella**, quindi al risveglio deve **ristabilire l'associazione** per ricevere i messaggi di **paging** broadcastati dall'MME quando qualcuno cerca di contattarlo (chiamata, SMS, dato in arrivo).

### Nota – Cos'è il paging

Quando qualcuno tenta di chiamare o inviare un messaggio a un dispositivo, la rete deve prima localizzarlo. L'MME **broadcasta** messaggi di paging nell'area in cui il dispositivo è probabilmente presente (la “tracking area”). Il dispositivo, ricevuto il paging, esce dal deep sleep e si ri-associa per ricevere il dato.

La **procedura di associazione** a una rete avviene in più passi:

1. La BS broadcasta ogni 5 ms un **primary synch signal** su tutte le frequenze. Più carrier diversi possono star broadcastando in parallelo.
2. Il dispositivo individua il primary synch signal e poi il **secondary synch signal** sulla stessa frequenza.
3. Legge le informazioni broadcastate dalla BS: larghezza di banda, configurazioni, info sul carrier cellulare.
4. Il dispositivo **seleziona la BS** a cui associarsi (preferendo tipicamente il proprio carrier home).
5. Seguono altri passi per **autenticarsi**, stabilire stato e configurare il piano dati.

## 3.9 La rivoluzione del 5G: prestazioni e use case

Mentre il 4G mirava a circa 10 Mbps e 10 ms di latenza, il 5G alza drasticamente l'asticella (proiezioni GAO e ITU):

- **10× il bitrate di picco** — 100 Mbps di “User experienced data rate” sostenuti, con picchi potenziali > 10 Gbps;
- **10× più bassa la latenza** — fino a **1 ms**;
- **100× più capacità di traffico**, e una **densità di connessione** che supporta **1 milione di dispositivi per km<sup>2</sup>**.

La nuova radio **5G NR (New Radio)** non è **retrocompatibile** con il 4G. Opera su due bande principali:

- **FR1** — 450 MHz – 6 GHz (microonde “tradizionali”, buona copertura);
- **FR2** — 24 GHz – 52 GHz (**onde millimetriche**, mmWave).

Le frequenze millimetriche permettono velocità immense ma su distanze brevi: questo richiede antenne direzionali (**MIMO**, Multiple-Input Multiple-Output) e un'installazione **densissima** di nuove stazioni base, le **pico-cells**, con raggio di soli 10–100 m.

### 3.9.1 I tre macro-scenari d'uso

#### Tip – eMBB — Enhanced Mobile Broadband

Larghezza di banda incrementata per applicazioni multimediali pesanti: realtà aumentata e virtuale, video streaming 4K/360°. Si punta a *data rate* multi-Gbps di picco, 100+ Mbps sostenuti, e capacità aggregata di 10 Tbps per km<sup>2</sup>.

**Tip – URLLC — Ultra Reliable Low-Latency Communications**

Applicazioni mission-critical altamente sensibili alla latenza: automazione di fabbrica, **guida autonoma**. Richiede affidabilità “five nines” (99.999%), latenza **1 ms** e supporto della mobilità ad alta velocità (fino a 100 km/h).

**Tip – mMTC — Massive Machine Type Communications**

Accesso *narrowband* per pervasività sensoriale: metering, monitoring, IoT diffuso. Dispositivi **ultra-low-power** (10+ anni di batteria), ultra-low-complexity (decine di bit/s), ultra-high density (1M nodi/km<sup>2</sup>).

### 3.10 La 5G Core e la Service Based Architecture

A livello architetturale il 5G Core (NG-Core) è profondamente riprogettato per integrarsi con Internet e con i servizi cloud. L'approccio è **cloud-native**: la rete è organizzata come un set di **Network Functions (NF)** indipendenti, riutilizzabili e implementabili come macchine virtuali o container su hardware generico (**Network Function Virtualization, NFV**). Le NF possono essere lanciate e rilasciate dinamicamente in base al carico.

Ogni Network Function espone i propri servizi tramite una **Service-Based Interface (SBI)**, un'interfaccia REST ben definita su **HTTP/2**. È un cambio paradigmatico: le funzioni di rete diventano **microservizi** che si chiamano tra loro come fossero API web.

Alcuni concetti rimangono comunque ereditati dal 4G — primo fra tutti l'incapsulamento dei pacchetti via **GTP-U** sul piano utente. La separazione fra control e user plane viene portata all'estremo, e i server vengono distribuiti **fino all'edge** della rete per ridurre la latenza.

#### 3.10.1 Mapping 4G → 5G delle funzioni

| Componente 4G | Corrispettivo 5G          | Funzione                                                                                                                                                                                                                                 |
|---------------|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| eNB           | gNB                       | Radio 5G modulare e cloud-native, splittata in <i>Distributed Unit</i> e <i>Central Unit</i> per deployment flessibile                                                                                                                   |
| S-GW + P-GW   | UPF (User Plane Function) | Inoltra il traffico fra RAN e Internet; gestisce packet inspection, QoS e reporting; può essere distribuita per Multi-Access Edge Computing (MEC)                                                                                        |
| MME           | AMF + SMF                 | <b>AMF</b> (Access and Mobility Management Function): autenticazione, gestione connessione, mobilità, reachability. <b>SMF</b> (Session Management Function): gestione sessioni dati, allocazione IP, controllo QoS e routing user-plane |

| Componente 4G | Corrispettivo 5G  | Funzione                                                                                                                                                                 |
|---------------|-------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HSS</b>    | <b>AUSF + UDM</b> | <b>AUSF</b> (Authentication Server Function): procedure di autenticazione. <b>UDM</b> (Unified Data Management): identità degli utenti, credenziali, dati di abbonamento |
| <b>PCRF</b>   | <b>PCF</b>        | Policy Control Function — definisce le regole di policy che le altre NF di controllo applicano                                                                           |

Il 5G introduce inoltre **Network Function di supporto** (helper) totalmente nuove, che rendono i servizi *stateless* e quindi facilmente scalabili:

- **SDSF** (Structured Data Storage Network Function) e **UDSF** (Unstructured Data Storage Network Function) — servizi di archiviazione dati centralizzati: estraendo lo stato dalle altre NF, queste possono restare *stateless*.
- **NEF** (Network Exposure Function) — espone in modo controllato e sicuro le capacità della rete a servizi di terze parti.
- **NRF** (NF Repository Function) — registry per la *service discovery* delle NF disponibili.
- **NSSF** (Network Slicing Selector Function) — seleziona la **Network Slice** appropriata per ogni UE, permettendo di partizionare logicamente la rete e differenziare il servizio fra utenti/tenant.

#### Tip – La logica cloud-native

Avere storage separato (SDSF/UDSF) e un registry di servizi (NRF) non è un dettaglio tecnico: è esattamente lo stesso *pattern* di un'architettura a microservizi cloud (12-factor, Kubernetes-style). Permette di **scalare orizzontalmente** una NF aggiungendo istanze, di **sostituire un'istanza** in caso di guasto senza perdere stato, e di esporre a terzi (NEF) capacità di rete come API consumabili.

### 3.10.2 UPF e Multi-Access Edge Computing

Particolarmente importante è il ruolo dell'**UPF**: corrispondente unificato di S-GW + P-GW, può essere replicato e distribuito in molteplici punti della rete, anche **all'edge** vicino agli utenti. Questa flessibilità abilita il **Multi-Access Edge Computing (MEC)**, in cui le **VNF** (Virtual Network Functions) e i workload applicativi possono essere co-locati con l'UPF stesso, riducendo drasticamente la latenza per scenari URLLC.

## 3.11 Opzioni di deployment: Standalone vs Non-Standalone

Il passaggio da 4G a 5G non è atomico: gli operatori possono scegliere fra diverse strategie di deployment.

| Modalità             | RAN | Core | Note           |
|----------------------|-----|------|----------------|
| <b>Standalone 4G</b> | 4G  | EPC  | Rete 4G “pura” |

| Modalità                    | RAN     | Core         | Note                                                                                                                                                                                               |
|-----------------------------|---------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Non-Standalone (NSA)</b> | 4G + 5G | EPC (4G)     | Le BS 5G affiancano quelle 4G per dare un <i>boost</i> di capacità; il <b>piano di controllo passa attraverso le BS 4G</b> verso il Core 4G, mentre le BS 5G veicolano <b>solo traffico utente</b> |
| <b>NSA su NG-Core</b>       | 4G + 5G | NG-Core (5G) | Variante con BS 4G+5G ma core già migrato al 5G                                                                                                                                                    |
| <b>Standalone 5G</b>        | 5G      | NG-Core      | 5G “puro”: sia controllo che dati gestiti dal NG-Core                                                                                                                                              |

#### Attenzione – Adozione del 5G SA molto disomogenea

Secondo i dati Ookla 2025, l'adozione del 5G Standalone varia enormemente nel mondo: **80% in Cina, 52% in India, 24% negli Stati Uniti**, ma **solo 2% in Europa**. L'Europa è in forte ritardo sull'adozione del 5G “puro” e la maggioranza delle reti è ancora in modalità NSA.

### 3.12 La rete cellulare come “rete di reti IP”

Le reti cellulari globali formano oggi una complessa “rete di reti IP” interconnesse tramite punti di interscambio chiamati **IPX** (IP eXchange). Pur condividendo molte tecnologie con Internet (HTTP, DNS, TCP/UDP, NAT, separazione control/data, SDN, Ethernet, tunneling), conservano alcune **specificità**:

|                     | Internet cablata          | Reti cellulari 4G/5G                               |
|---------------------|---------------------------|----------------------------------------------------|
| Mobilità            | non nativa                | servizio di <b>prima classe</b>                    |
| Identità utente     | varia (IP, account, ecc.) | fortemente legata alla <b>SIM card</b>             |
| Modello di business | best-effort, peering      | <b>abbonamento</b> a un carrier specifico          |
| Home vs visited     | non significativo         | distinzione fondamentale, regola il <b>roaming</b> |
| Link di accesso     | tipicamente cablato       | <b>radio</b> , con tutti i suoi vincoli            |

Il **roaming** — l'uso della rete da parte di un abbonato fuori dalla propria home network — è regolato dal **rapporto di fiducia commerciale** fra rete home e rete visitata: è la home network (che possiede l'HSS con i dati dell'abbonato) a “garantire” per l'utente, mentre la rete visitata fornisce l'accesso e poi regola economicamente i costi con la home.

### 3.13 Sicurezza e autenticazione in 4G LTE: il protocollo AKA

Quando un dispositivo arriva in una rete (anche visitata), deve compiere **due operazioni distinte**:

1. **Associarsi alla BS** — stabilire la comunicazione sul canale radio 4G;



### 5G Architecture - User Plane

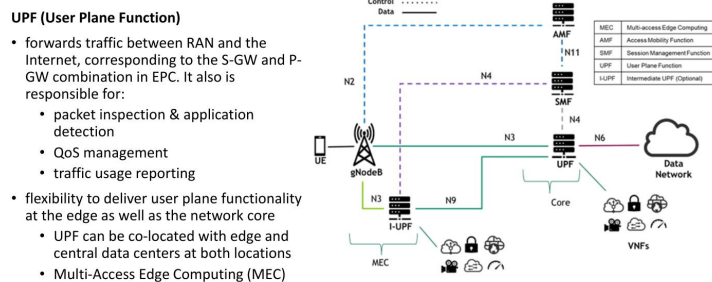
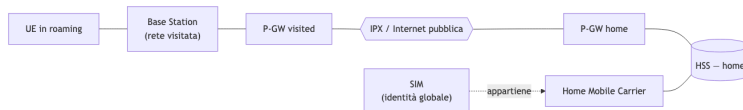


Image from: <https://developer.nvidia.com/blog/accelerating-iva-using-ultra-efficient-5g-core-with-mavenir-and-nvidia-edge-ai/>

**Figura 3.6.** Architettura 5G User Plane (immagine NVIDIA/Mavenir). Il **gNodeB** dialoga con un **I-UPF** (Intermedie UPF) all'edge tramite l'interfaccia **N3**, raggiungendo il core via **N9** fino all'**UPF** centrale che si interfaccia col Data Network (**N6**). Il control plane (linee tratteggiate) — **AMF**, **SMF** — comunica con i nodi user-plane via **N2**, **N4**, **N11**. Le **VNF** all'edge realizzano il MEC.



**Figura 3.7.** Roaming in una rete globale di carrier interconnessi via IPX. La SIM lega l'utente alla home network; quando è in roaming, la rete visitata si appoggia all'HSS della home per autenticare e autorizzare l'utente.

#### 2. Autenticarsi alla rete — e contemporaneamente autenticare la rete (mutua autenticazione).

A differenza del Wi-Fi, qui la **SIM card fornisce l'identità globale** dell'utente, e i servizi disponibili nella rete visitata dipendono da una **subscription pagata** nella home network. **MME nella rete visitata** e **HSS nella rete home** giocano insieme il ruolo dell'Authentication Server di un access point Wi-Fi: il vero “ultimate authenticator” è l'HSS, ma le decisioni passano dall'MME visitato.

#### Definizione – AKA — Authentication and Key Agreement

Protocollo di autenticazione **challenge-and-response** definito da 3GPP, basato su **chiave simmetrica** condivisa fra abbonato e home network. Garantisce **autenticazione** delle entità, **integrità** e **confidenzialità** dei messaggi. Dopo la mutua autenticazione, vengono derivate chiavi crittografiche per proteggere sia la signaling sia i dati del piano utente.

Vediamo passo-passo come funziona, con particolare attenzione al *perché* la mutua autenticazione regge.

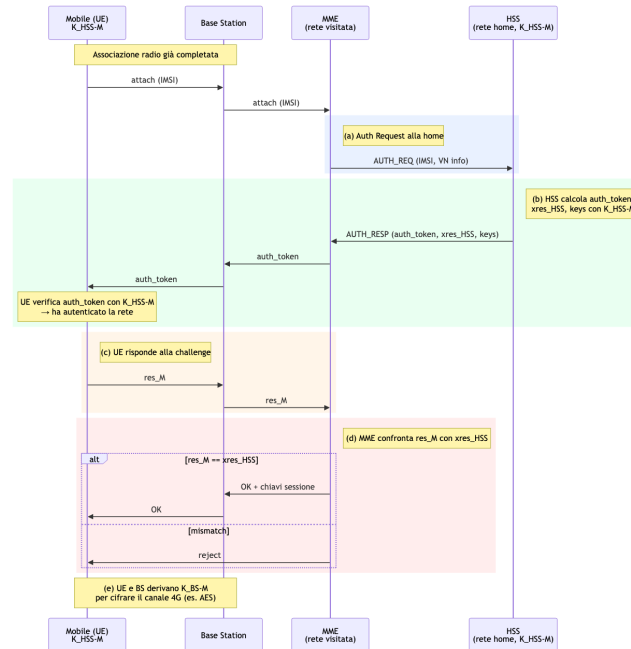
#### 3.13.1 I cinque passi di AKA, in dettaglio

(a) **Attach + AUTH\_REQ.** L'UE invia un messaggio di *attach* contenente il proprio **IMSI**. La BS lo inoltra all'MME della rete visitata, che a sua volta gira **IMSI + informazioni della rete visitata (VN info)** all'**HSS** nella home network del dispositivo. L'**IMSI** identifica esplicitamente la home network, quindi l'MME sa a chi rivolgersi.

(b) **HSS genera la challenge.** L'HSS, usando la **chiave segreta pre-condivisa**  $K_{HSS-M}$  (memorizzata in modo sicuro sulla SIM dal lato UE, e nel database dell'HSS dal lato rete), calcola:

- un **auth\_token** che contiene informazioni cifrate con  $K_{HSS-M}$ ;
- la **risposta attesa**  $xres_{HSS}$  (la “verifica” della challenge);
- le **chiavi di sessione**.

«««< HEAD Manda (**auth\_token**, **xres\_HSS**, **keys**) all'MME, che inoltra solo l'**auth\_token** all'UE attraverso la BS. L'UE, possedendo lui pure  $K_{HSS-M}$ , **decifra e verifica matematicamente** l'auth-token: se



**Figura 3.8.** Sequence diagram del protocollo AKA in 4G LTE. Le cinque fasi (a)...(e) corrispondono ai passi numerati delle slide.

torna, significa che l'altra parte conosce davvero la chiave segreta — quindi è la vera home network. **A questo punto l'UE ha autenticato la rete.** Nel frattempo l'MME tiene da parte  $xres_{HSS}$  per il passo (d). ===== Manda ( $auth\_token$ ,  $xres_{HSS}$ , keys) all'MME, che inoltra solo l' $auth\_token$  all'UE attraverso la BS. L'UE, possedendo lui pure  $K_{HSS-M}$ , **decifra e verifica matematicamente** l' $auth\_token$ : se torna, significa che l'altra parte conosce davvero la chiave segreta — quindi è la vera home network. **A questo punto l'UE ha autenticato la rete.** Nel frattempo l'MME tiene da parte  $xres_{HSS}$  per il passo (d). »»»»> origin/main

**(c) UE risponde.** L'UE esegue **lo stesso calcolo crittografico** che ha fatto l'HSS, usando la sua copia di  $K_{HSS-M}$ , ottenendo  $res_M$ . Lo invia all'MME via BS.

**(d) MME valida.** L'MME confronta  $res_M$  (calcolato dall'UE) con  $xres_{HSS}$  (calcolato dall'HSS). Se coincidono, **la rete ha autenticato l'UE** — perché solo chi possiede  $K_{HSS-M}$  poteva ottenere quel valore. L'MME informa la BS dell'avvenuta autenticazione e genera le chiavi di sessione per la BS.

**(e) Derivazione delle chiavi radio.** UE e BS, a partire dal materiale crittografico scambiato, derivano la chiave  $K_{BS-M}$  con cui cifrano dati e frame di controllo sul canale wireless 4G — tipicamente con **AES**.

#### Domanda – Perché il confronto

`text{res}_M =`

`text{xres}_{HSS}` basta a dimostrare l'identità dell'UE?

Perché la funzione che calcola la response è **deterministica e dipende dalla chiave segreta**  $K_{HSS-M}$ . Solo due entità la conoscono: la SIM dell'utente e l'HSS. Se il valore calcolato dall'UE coincide con quello calcolato dall'HSS, è praticamente impossibile (sotto le ipotesi crittografiche standard) che l'UE non possieda la chiave — quindi è autentico. La presenza di un *nonce* freschissimo nella challenge previene attacchi di replay.

### 3.14 Sicurezza in 5G: cosa cambia rispetto al 4G

Il 5G mantiene la struttura concettuale di AKA ma ne irrobustisce due aspetti chiave, entrambi rivolti a chiudere falle di privacy/fiducia presenti in 4G.

| Aspetto                            | 4G                                                      | 5G                                                                                                      |
|------------------------------------|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| <b>Decisione di autenticazione</b> | Preso dall' <b>MME nella rete visitata</b>              | Preso dalla <b>rete home</b> ; l'AMF visitato fa da "middleman" (può comunque rifiutare la connessione) |
| <b>Trasmissione dell'IMSI</b>      | Inviato <b>in chiaro</b> alla BS al momento dell'attach | Cifrato con <b>crittografia a chiave pubblica</b> prima di essere trasmesso                             |

#### Attenzione – L'IMSI in chiaro era un problema serio

In 4G l'IMSI viaggiava in cleartext sulla prima porzione del protocollo di attach: questo permetteva attacchi noti come **IMSI catcher** ("Stingray"), apparati che si fingono BS legittime per intercettare gli IMSI dei dispositivi vicini e tracciare gli utenti. Il 5G chiude questa falla cifrando l'identificativo con una **chiave pubblica della home network**: solo l'home network può decifrarlo, la rete visitata e gli intercettatori vedono solo un identificativo "schermato" (SUCI — SUBscription Concealed Identifier).

#### Nota – Centralizzare la decisione nella home

Spostare la decisione finale di autenticazione dalla rete visitata alla rete home riduce la superficie di attacco: anche se un MME visitato viene compromesso, da solo non può "creare" abbonati validi. La rete visitata mantiene comunque potere di veto sulla connessione (può rifiutare per policy locali), ma non può approvarla autonomamente.

#### Nota – Sintesi

L'evoluzione dalle reti cellulari analogiche al 5G è una trasformazione architetturale, non solo un aumento di velocità. Il cuore della trasformazione è il passaggio a un **All-IP core** (introdotto in 4G con l'EPC) e poi a una **Service Based Architecture cloud-native** (in 5G con l'NG-Core), con netta separazione di control plane e data plane e con le funzioni di rete organizzate come **microservizi** indipendenti. Il **tunneling GTP** è il pilastro che abilita la mobilità trasparente: gli IP del traffico restano fissi, cambiano solo gli endpoint dei tunnel quando l'utente si sposta. Il protocollo **AKA** garantisce mutua autenticazione UE rete sfruttando una chiave simmetrica condivisa fra SIM e home HSS; il 5G rafforza la sicurezza spostando la decisione finale alla home network e cifrando l'IMSI con crittografia a chiave pubblica per prevenire attacchi di tracciamento. Sul fronte radio, **5G NR** combina sub-6 GHz e onde millimetriche con MIMO e densità di celle elevate, abilitando i tre macro-scenari **eMBB**, **URLLC** e **mMTC**.

## Capitolo 4

# Mobile Networks — Mobilità nelle Reti Cellulari

La mobilità nelle reti di telecomunicazione moderne si estende lungo un ampio spettro: dall'assenza totale di spostamento fino a scenari in cui un dispositivo attraversa reti di operatori diversi mantenendo attive le proprie sessioni. Questo capitolo analizza le architetture che rendono possibile tale mobilità — in particolare nelle reti 4G/5G — esaminando i meccanismi di registrazione, i due approcci di routing (indiretto e diretto) e le procedure di *handover* tra stazioni base.

---

### 4.1 Lo Spettro della Mobilità e la Home Network

Dal punto di vista infrastrutturale, la mobilità non è una proprietà binaria ma uno spettro continuo. A un estremo si trovano dispositivi completamente statici; procedendo si incontrano dispositivi che cambiano rete solo tra una sessione e l'altra, o che si muovono tra Access Point dello stesso operatore. L'estremo di maggiore interesse è l'**alta mobilità**: un dispositivo che cambia rete mantenendo attive le connessioni in corso.

Per gestire questo livello di mobilità è indispensabile il concetto di **home network**: una fonte autorevole e centralizzata che registra la posizione attuale del dispositivo e dalla quale le altre entità di rete possono ottenerla. Senza di essa non esiste modo affidabile di recapitare traffico a un nodo in movimento.

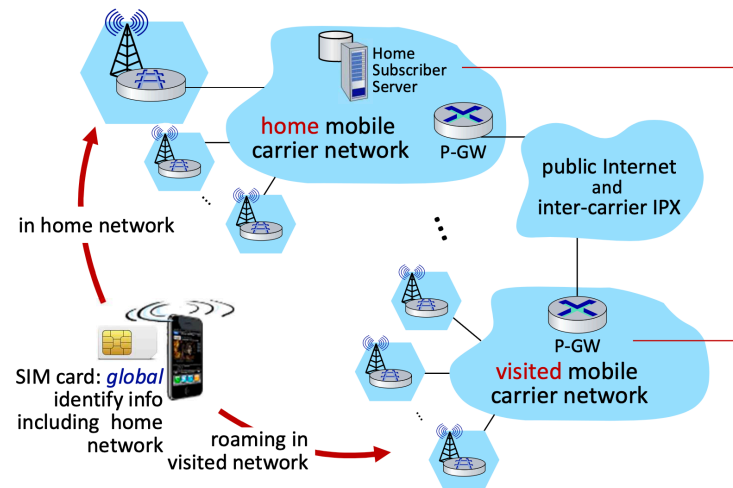
#### 4.1.1 Reti 4G/5G vs ISP/WiFi

Nelle reti cellulari **4G/5G** la home network corrisponde alla rete dell'operatore con cui l'utente ha sottoscritto il contratto (es. Verizon, Orange). Il database centrale che memorizza identità e servizi abilitati è l'**Home Subscriber Server (HSS)**. L'identità globale del dispositivo — inclusa la sua rete di appartenenza — è codificata nella **SIM card**.

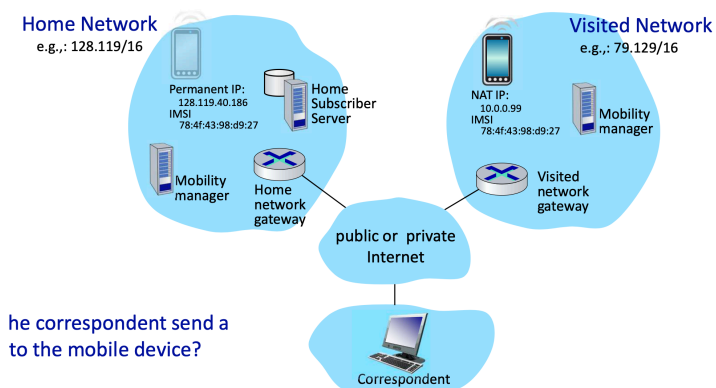
Quando il dispositivo lascia la copertura del proprio operatore entra in una **visited network**, condizione nota come *roaming*. La rete visitata ha accordi commerciali con altre reti per garantire accesso ai dispositivi in transito.

Le reti **ISP/WiFi** presentano un'architettura radicalmente diversa: manca una nozione globale di home. L'autenticazione avviene tramite server locali, le credenziali variano da rete a rete e la mobilità fluida è difficile da realizzare. Esistono eccezioni accademiche come *eduroam* e architetture teoriche come **Mobile IP**, ma quest'ultima non ha mai raggiunto un'adozione commerciale significativa.

---



**Figura 4.1.** Struttura della home network in un'architettura 4G/5G.



**Figura 4.2.** Relazione tra home network e visited network in uno scenario di roaming.

## 4.2 Approcci alla Gestione della Mobilità

I progettisti di rete hanno valutato due filosofie principali per permettere a un nodo corrispondente di raggiungere un dispositivo mobile.

Il primo approccio affida la mobilità direttamente ai router, che annuncerebbero la posizione del nodo mobile attraverso i consueti protocolli di routing. Tecnicamente Internet potrebbe supportarlo tramite il *longest prefix match*, ma l'approccio è stato scartato per mancanza di scalabilità: propagare la posizione di miliardi di dispositivi mobili nelle tabelle di routing globali è impraticabile.

Il secondo approccio, quello effettivamente implementato, sposta la complessità ai margini della rete (*edge*), lasciando che siano i sistemi terminali a gestirla. Questo dà origine a due tecniche: il **routing indiretto** e il **routing diretto**.

## 4.3 La Fase di Registrazione

Indipendentemente dalla tecnica di routing adottata, la home network deve sempre conoscere la posizione corrente del dispositivo. Questo avviene tramite la **registrazione**: il dispositivo si associa a un *mobility manager* nella rete visitata, il quale comunica la posizione aggiornata all'HSS domestico. Al termine della procedura, il mobility manager della rete visitata è a conoscenza della presenza del dispositivo e l'HSS domestico conosce la sua posizione remota.

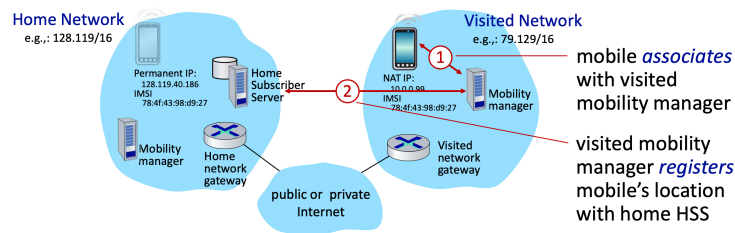


Figura 4.3. Procedura di registrazione tra visited network e home network.

Durante le operazioni in roaming il dispositivo mantiene il proprio indirizzo permanente associato alla home network, ma ottiene temporaneamente un indirizzo IP nel range della rete visitata, tipicamente assegnato tramite NAT.

## 4.4 Routing Indiretto e Routing Diretto

Nel **routing indiretto** — modalità standard per le reti 4G LTE e per Mobile IP — il nodo corrispondente invia il datagramma all'indirizzo permanente del dispositivo mobile. Il gateway della home network lo riceve, lo incapsula in un tunnel e lo inoltra al gateway della rete visitata, che decapsula il pacchetto, applica la traduzione NAT e lo consegna al dispositivo. Le risposte possono seguire il percorso inverso oppure essere inviate direttamente al corrispondente.

Perché questo meccanismo funzioni servono tre protocolli specifici:

1. **Protocollo di associazione** dispositivo–rete visitata: il dispositivo si associa alla rete visitata e si disassocia quando la lascia.
2. **Protocollo di registrazione** rete visitata–HSS domestico: la rete visitata comunica all'HSS la posizione del dispositivo.
3. **Protocollo di tunneling** tra i due gateway: il gateway domestico incapsula il datagramma originale; il gateway visitato lo decapsula, applica il NAT e lo consegna al dispositivo.

Questo schema genera il cosiddetto *triangle routing*: se corrispondente e dispositivo si trovano nella stessa rete, i dati compiono un percorso inutilmente lungo fino alla home network. Il vantaggio, però, è notevole: ogni cambio di rete visitata è completamente trasparente per il corrispondente — la nuova rete si registra presso l'HSS e il traffico viene reindirizzato nel nuovo tunnel senza azioni da parte del corrispondente.

**Nota – Continuità delle sessioni durante il cambio di rete**

Quando il dispositivo si sposta in una nuova rete visitata, possono andare persi alcuni datagrammi in transito. Tuttavia le sessioni TCP rimangono attive: dal punto di vista del corrispondente, la posizione del mobile è un dettaglio interno alla home network.

Nel **routing diretto** il corrispondente interroga l'HSS domestico per ottenere il *care-of-address* del dispositivo nella rete visitata e invia i datagrammi direttamente a quell'indirizzo, eliminando le inefficienze del triangle routing. Tuttavia l'approccio non è trasparente per il corrispondente, che deve gestire esplicitamente la richiesta dell'indirizzo. Inoltre, se il dispositivo cambia rete durante una sessione attiva, il problema è più complesso: il corrispondente ha interrogato l'HSS **solo all'inizio della sessione** e non conosce il nuovo indirizzo; sono quindi necessari meccanismi aggiuntivi per aggiornare dinamicamente il flusso dati, a differenza del routing indiretto dove è sufficiente cambiare l'endpoint del tunnel.

## 4.5 L'Architettura Pratica: Mobilità nelle Reti 4G

Quando un dispositivo entra in una rete 4G visitata, la gestione della mobilità si sviluppa attraverso quattro fasi distinte che coinvolgono progressivamente il piano di controllo, il piano dati e infine la transizione tra celle.

- Il punto di ingresso è l'**associazione alla Base Station (BS)**: il dispositivo si aggancia alla stazione radio base della rete visitata e si identifica tramite il proprio **IMSI** (*International Mobile Subscriber Identity*), che codifica sia l'identità del dispositivo sia la sua home network di appartenenza.
- A questo punto entra in gioco il **piano di controllo**: la BS coinvolge la **Mobility Management Entity (MME)** locale, che usa l'IMSI per contattare l'HSS domestico. L'HSS restituisce all'MME i parametri di autenticazione, crittografia e servizi abilitati per quell'utente, e al tempo stesso viene aggiornato sulla nuova posizione del dispositivo. È qui che avviene la registrazione vera e propria: da questo momento l'HSS domestico sa dove si trova l'utente.
- Una volta autenticato l'utente, l'MME configura il **piano dati** stabilendo i tunnel necessari a far fluire il traffico. Poiché si adotta routing indiretto, tutto il traffico deve transitare per la home network. Vengono creati due tunnel distinti, entrambi usando il protocollo **GTP** (*GPRS Tunneling Protocol*):
  - **Tunnel BS S-GW**: quando il dispositivo cambia Base Station, non occorre ricreare il tunnel — è sufficiente aggiornare l'indirizzo IP dell'endpoint sul lato BS.
  - **Tunnel S-GW P-GW** (home network): implementa il routing indiretto verso la home network.
- La quarta fase è l'**handover tra Base Station**, che si attiva quando il dispositivo si sposta e la qualità del segnale sulla BS corrente degrada. La procedura completa si articola in sette passi:
  1. La **BS sorgente** decide di avviare l'handover (per degrado del segnale o sovraccarico) e seleziona la *target BS*, inviandole una **Handover Request**.
  2. La **target BS** pre-allocava le risorse radio e risponde con un **Handover Request ACK** contenente le informazioni di configurazione per il dispositivo.
  3. La **BS sorgente** notifica il dispositivo del cambio imminente; da questo momento il dispositivo può già trasmettere tramite la nuova BS — l'handover sembra completato dal punto di vista del dispositivo.
  4. La **BS sorgente** smette di trasmettere al dispositivo e inizia a **forwardare** i datagrammi in arrivo verso la target BS (che li recapita al dispositivo via radio).
  5. La **target BS** informa l'MME di essere la nuova BS per il dispositivo.

6. L'MME istruisce lo **S-GW** di aggiornare l'endpoint del tunnel dati alla nuova target BS. La BS sorgente riceve conferma e può liberare le proprie risorse radio.
7. I datagrammi del dispositivo fluiscono ora attraverso il **nuovo tunnel** dalla target BS allo S-GW.

#### Tip – Perché la BS sorgente decide l'handover

Sia la decisione di avviare l'handover sia la scelta della target BS spettano alla **BS sorgente** (non all'MME). L'MME viene coinvolto solo nella fase finale per aggiornare il piano dati. Questo è un punto frequente nelle domande d'esame.

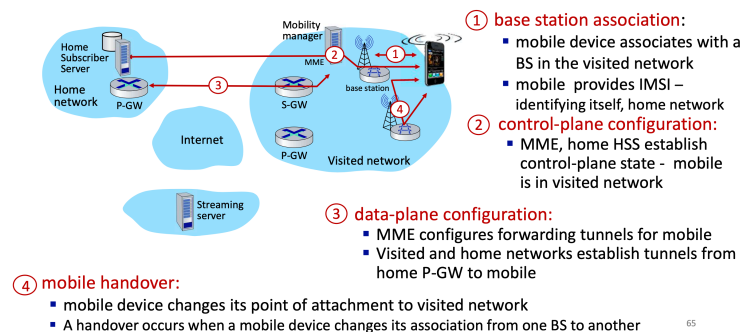


Figura 4.4. Sequenza di messaggi durante un handover tra due Base Station in 4G.

## 4.6 Mobile IP e Impatto sui Protocolli di Trasporto

L'architettura **Mobile IP** (RFC 5944), standardizzata circa vent'anni fa, anticipava molti dei concetti oggi presenti nel 4G: il *home agent* univa i ruoli dell'attuale HSS e P-GW, mentre il *foreign agent* corrispondeva all'accoppiata MME/S-GW. La registrazione avveniva tramite estensioni ICMP. All'epoca WiFi per i dati e reti 2G/3G per la voce erano considerati sufficienti, e Mobile IP non ha mai raggiunto una diffusione commerciale rilevante.

Sul fronte dell'impatto sui protocolli superiori, in teoria il layer wireless dovrebbe essere trasparente: il modello *best-effort* di IP rimane invariato e TCP/UDP girano regolarmente su reti mobili. In pratica le prestazioni degradano sensibilmente. Errori di bit sui link radio e interruzioni transitorie durante gli handover vengono interpretati da TCP come sintomi di congestione di rete, inducendolo a ridurre la *congestion window* in modo ingiustificato. A questo si aggiungono la limitatezza di banda dei link wireless e i ritardi che penalizzano il traffico in tempo reale.



## Capitolo 5

# Software Defined Networking (SDN)

Questo paradigma architetturale segna il definitivo passaggio da un sistema di protocolli puramente distribuiti a un framework di controllo di rete completamente programmabile.

### Definizione – Software Defined Networking

Il **Software Defined Networking** è un approccio alla gestione delle reti che sfrutta potenti meccanismi di astrazione per abilitare configurazioni e operazioni dinamiche ed efficienti a livello programmatico. La caratteristica distintiva è la separazione netta tra **control plane** e **data plane**.

## 5.1 L'Evoluzione dei Requisiti e la Complessità del Traffico

Sebbene il SDN sia spesso definito come un'idea radicalmente nuova, esso nasce per risolvere problemi concreti dettati dai moderni trend tecnologici: l'esplosione del **Cloud computing**, la gestione dei **Big data**, il massiccio aumento del traffico mobile e la diffusione pervasiva dell'**IoT** (Internet of Things). I dispositivi moderni possiedono CPU più potenti, buffer più capienti e link più veloci, ma il vero ostacolo non risiede nei puri volumi di traffico, bensì nella sua **variabilità spaziale e temporale**: i pattern di traffico sono diventati altamente dinamici, complessi e strettamente sensibili alle applicazioni che li generano, rendendo insufficiente la gestione statica.

La complessità e dinamicità del traffico sono alimentate da tre fenomeni strutturali. Le architetture applicative moderne sono fortemente distribuite: interrogano database multipli e server di backend, generando enormi volumi di traffico orizzontale tra server che si sommano al tradizionale traffico verticale client-server. La proliferazione di piattaforme di **Unified Communications (UC)** integra su un'unica infrastruttura servizi come chat, voce, video e gestione della presenza, obbligando la rete a sostenere flussi paralleli con requisiti di **QoS** radicalmente diversi. Infine, l'ascesa del cloud computing ha alterato la località geografica del traffico: dati un tempo confinati nel perimetro aziendale oggi attraversano ampie **WAN** verso cloud remoti, sottoponendo i router enterprise a carichi severi e imprevedibili.

La virtualizzazione aggrava ulteriormente il quadro: migrazioni, replicazioni e failover di **Virtual Machines** causano forti sbalzi di traffico orizzontale la cui posizione e intensità variano costantemente. I dispositivi mobili aggiungono frequenti cambi di access point e ritmi di utilizzo irregolari. Il risultato è che applicazioni real-time necessitano bassa latenza, backup di massa privilegiano la banda, e transazioni finanziarie richiedono garanzie di consegna — esigenze incompatibili con l'instradamento statico.

### Nota – Il dominio del traffico East-West

Oltre il 70% del traffico di rete odierno è di natura **East-West**, ovvero circoscritto alle comunicazioni interne tra server all'interno di un data center. I tradizionali data center a tre livelli (Access, Aggregation,

Core) sono però ottimizzati per il traffico **North-South** (richieste client-server). Gestire efficientemente il traffico East-West richiede selezione ultra-flessibile dei path, load balancing efficace e riconfigurazioni immediate.

## 5.2 Limiti Strutturali delle Architetture Tradizionali

Le reti convenzionali falliscono di fronte ai moderni scenari per tre ragioni fondamentali. L'**architettura statica e frammentata** è il primo problema: ogni protocollo risolve un problema isolato (routing, spanning tree, ACL) moltiplicando la complessità operativa senza una visione globale. L'**incoerenza delle policy** è il secondo: i parametri vengono elaborati localmente, obbligando gli operatori ad aggiornare manualmente ogni dispositivo con alto rischio di errata configurazione. Il **vendor lock-in** è il terzo: la logica di controllo è incapsulata in hardware proprietario privo di interfacce aperte, che impedisce la programmabilità dell'infrastruttura.

La forza storica di Internet è stata l'architettura a livelli gerarchici (Fisico, Link, Rete, Trasporto, Applicazione): la suddivisione in strati fornisce astrazioni precise dei servizi e consente a ciascun livello di evolversi indipendentemente. Tuttavia, queste astrazioni non sono state applicate in egual misura a tutti i piani. Mentre il data plane gode di logiche definite, il **control plane** tradizionale si è evoluto caoticamente aggiungendo protocolli su protocolli a ogni nuovo requisito tecnico. Costruito attorno a hardware chiusi con interfacce proprietarie, il network management non segue i principi solidi adottati, ad esempio, nei sistemi di database distribuiti: il risultato è un groviglio ingestibile di regole che sopprime l'innovazione.

## 5.3 La Separazione tra Data Plane e Control Plane

### Definizione – Data Plane e Control Plane

Il **Data Plane (Forwarding)** è l'atto meccanico e istantaneo di smistare ogni singolo pacchetto in transito su una porta d'uscita del router. Il **Control Plane (Routing)** è il processo decisionale, su scale temporali molto più dilatate, attraverso cui si mappa il tragitto completo sorgente-destinazione.

Nel tradizionale instradamento IP entrambi coesistono in un modello **per-router control plane** fortemente accoppiato: gli algoritmi risiedono in ogni nodo e calcolano localmente le tabelle di forwarding in modo decentralizzato. Tale struttura fu ideata per garantire resilienza, ma rende le moderne operazioni di *Traffic Engineering* estremamente difficili. I link weight sono l'unico “manopola di controllo” disponibile. Si considerino tre scenari concreti su una rete con nodi u, v, w, x, y, z:

### Tip – Tre scenari impossibili con il routing tradizionale

**Scenario 1 — Percorso specifico:** l'operatore vuole che il traffico da u a z segua il percorso  $u \rightarrow v \rightarrow w \rightarrow z$  anziché il percorso più breve  $u \rightarrow x \rightarrow y \rightarrow z$ . L'unica leva è ridefinire i link weight affinché l'algoritmo calcoli quella rotta — ma non esiste garanzia che il risultato sia quello voluto senza effetti collaterali sugli altri flussi.

**Scenario 2 — Load balancing:** l'operatore vuole dividere il traffico da u a z equamente tra i due percorsi  $u \rightarrow v \rightarrow w \rightarrow z$  e  $u \rightarrow x \rightarrow y \rightarrow z$ . Non è possibile con il routing basato su destinazione: l'algoritmo converge su un unico percorso minimo.

**Scenario 3 — Routing per-flusso:** il nodo w vuole instradare verso z in modo diverso il traffico blu (proveniente da u) rispetto al traffico rosso (proveniente da x). Non è possibile con il forwarding basato sulla sola destinazione (LS, DV): la decisione dipende unicamente dall'indirizzo di destinazione, non dal flusso.

## 5.4 Il Paradigma SDN: Centralizzazione Logica e Programmabilità

La grande innovazione del SDN è il **Logically Centralized Control Plane**: un controller interroga e supervisiona remotamente la topologia globale calcolando le tabelle di forwarding per tutti i nodi. Mentre nel modello classico ogni tabella è rigidamente calcolata come  $T_i = SPF(Topology, link\_weights)$  ignorando metriche esterne, nel modello SDN il controller calcola la funzione globale:

$$\mathcal{F} : S(t) \rightarrow \{T_1, T_2, \dots, T_n\}$$

dove le tabelle di uscita derivano dallo stato  $S(t)$  che ingloba in tempo reale topologia, traffico totale e vincoli di policy.

### Importante – Architettura SDN e astrazione Match-Action

Il SDN poggia su pilastri imprescindibili: una demarcazione netta tra data plane e control plane; il control plane definisce il comportamento strategico della rete mentre il data plane applica le direttive ai pacchetti fisici; l'uso di switch elementari ad alte prestazioni governati tramite l'astrazione **match-action** orientata ai flussi (es. OpenFlow); il ricorso totale a protocolli aperti e programmabili.

Il controller SDN comunica su due assi. La **Southbound API** (tipicamente OpenFlow) è l'interfaccia verso il basso tra il controller e gli switch del data plane, tramite cui vengono installate le regole di forwarding nell'hardware. La **Northbound API** è l'interfaccia verso l'alto che dialoga con le applicazioni di gestione (bilanciamento, routing avanzato, access control), sviluppabili autonomamente da terze parti indipendentemente dai produttori hardware.

### Attenzione – L'illusione del server singolo

Sebbene logicamente centralizzato, il *SDN Controller* è fisicamente implementato come un sistema fortemente distribuito. Ciò assicura tolleranza agli errori, elevata scalabilità e robustezza contro guasti singoli critici.

## 5.5 Architettura SDN: I Tre Componenti

L'architettura SDN si articola in tre livelli distinti con ruoli ben separati.

### 5.5.1 Data-Plane Switches

Gli switch del data plane sono dispositivi semplici e veloci, spesso basati su hardware commodity a basso costo. Il loro unico compito è applicare meccanicamente le regole di forwarding ai pacchetti in transito secondo l'astrazione **match-action**: confrontano i campi dell'intestazione del pacchetto con le regole nella flow table e applicano l'azione corrispondente. Le flow table vengono calcolate e installate da remoto dal controller, non localmente. Lo switch espone un'API standardizzata (tipicamente OpenFlow) che definisce cosa è controllabile dall'esterno e cosa no, e un protocollo per comunicare con il controller.

### 5.5.2 SDN Controller (Network Operating System)

Il controller SDN svolge il ruolo di **sistema operativo di rete**: mantiene una visione aggiornata dello stato globale della rete (topologia, link attivi, tabelle di forwarding). È il punto di mediazione tra le applicazioni che esprimono policy di alto livello e gli switch che le devono applicare. Interagisce verso il basso tramite la **Southbound API** con gli switch del data plane, e verso l'alto tramite la **Northbound API** con le applicazioni di controllo. Sebbene appaia come un'entità singola, è implementato come sistema distribuito per garantire performance, scalabilità, tolleranza ai guasti e robustezza.

### 5.5.3 Network-Control Applications

Le applicazioni di controllo sono il vero “cervello” della rete: implementano le funzioni di controllo (routing, access control, load balancing, ...) sfruttando i servizi e le API esposte dal controller. Il punto chiave è che sono **unbundled**: possono essere sviluppate e fornite da soggetti terzi — distinti sia dal produttore degli switch hardware sia dal produttore del controller SDN. Questo disaccoppiamento abilita un ecosistema aperto in cui l’innovazione a livello applicativo è indipendente dall’hardware sottostante.

#### Nota – Sintesi dell’architettura

| Livello | Componente                  | Ruolo                                                           |
|---------|-----------------------------|-----------------------------------------------------------------|
| Alto    | Network-control apps        | “Brains”: routing, load balancing, ACL — unbundled, terze parti |
| Medio   | SDN Controller (Network OS) | Stato globale della rete; northbound southbound                 |
| Basso   | Data-plane switches         | Match-action su flow table installate dal controller            |

## 5.6 Cronologia e Sviluppi Storici

| Periodo | Pietra Miliare          | Dettagli                                                                                                                                                                                       |
|---------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ~2004   | Genesi ricercativa      | Princeton, Stanford, Berkeley avviano la ricerca su nuovi paradigmi di gestione del networking.                                                                                                |
| 2008    | Nascita ufficiale SDN   | NICIRA presenta NOX; OpenFlow nasce in collaborazione con Stanford.                                                                                                                            |
| 2011    | Standardizzazione       | Fondazione dell’ <b>Open Networking Foundation (ONF)</b> con Google, Yahoo, Verizon, Cisco, HP, Dell.                                                                                          |
| 2013    | Consacrazione scalabile | Picco di interesse all’Open Networking Summit (1600 partecipanti). Google rende pubblica la propria implementazione SDN sulla WAN globale.                                                     |
| 2014    | Data center             | VMware NSX e Cisco ACI portano SDN nei data center globali.                                                                                                                                    |
| 2021+   | ML, 5G, P4              | Intent-Based Networking con AI/ML (Cisco DNA, Mist AI). SDN come base del 5G. Linguaggio <b>P4</b> (Programming Protocol-independent Packet Processors) per switch programmabili a line speed. |

---

## Capitolo 6

# Software Defined Networking (SDN) — Architettura

Il Software Defined Networking è un paradigma architetturale che separa nettamente il **piano di controllo** (*control plane*) dal **piano dati** (*data plane*). Questa separazione permette di programmare il comportamento della rete in modo centralizzato, eliminando la rigidità dei dispositivi di rete tradizionali che incorporano sia la logica di instradamento che la funzione di forwarding. La lezione approfondisce l'architettura interna di questi due piani e i meccanismi con cui interagiscono.

### 6.1 Motivazioni: Perché SDN?

Il Software Defined Networking non è nato per risolvere un semplice problema di capacità, ma per rispondere a una trasformazione strutturale delle reti moderne. La domanda di rete — alimentata da cloud computing, big data, traffico mobile e IoT — è cresciuta enormemente, ma anche l'offerta ha tenuto il passo: CPU più veloci, buffer più grandi, link ad alta capacità. Il vero problema, dunque, non è quantitativo ma qualitativo: **la complessità si è spostata dal volume alla variabilità**.

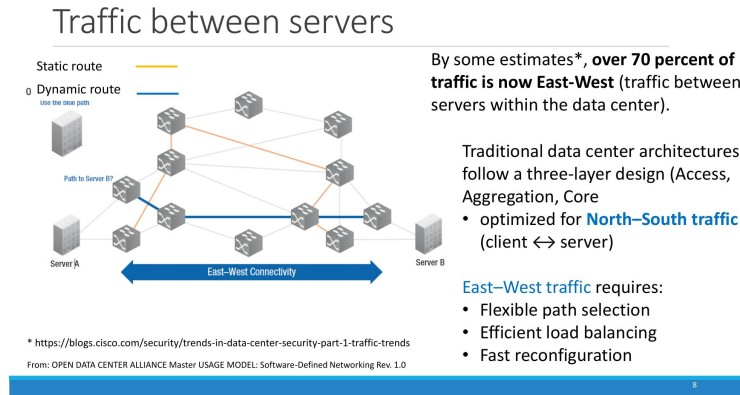
I pattern di traffico moderni sono *time-variant* (cambiano nel tempo), *spatially dynamic* (cambiano nei percorsi) e *application-sensitive* (dipendono dal tipo di applicazione). Questa variabilità richiede un controllo programmabile della rete, che le architetture tradizionali non sono in grado di offrire.

#### 6.1.1 Dinamiche del Traffico Moderno

Le applicazioni distribuite moderne accedono simultaneamente a molteplici database e server backend, generando significativo traffico “orizzontale” (*server-to-server*) in aggiunta al tradizionale traffico “verticale” (*client-server*). I servizi di Unified Communications (UC) — messaggistica, presenza, VoIP, videoconferenza — amplificano ulteriormente questa complessità, richiedendo alla rete di supportare flussi concorrenti con requisiti di QoS eterogenei.

L'adozione massiva del cloud computing ha spostato traffico che un tempo rimaneva all'interno delle reti enterprise verso data center remoti, rendendo i carichi sui router enterprise imprevedibili. La virtualizzazione dei server ha aggiunto un ulteriore strato: una singola macchina fisica ospita multiple VM tra cui avvengono significative comunicazioni (migrazione, replica, failover).

Il traffico mobile aggiunge un'ulteriore dimensione di imprevedibilità: i dispositivi mobili generano carichi rapidamente mutevoli e cambiano frequentemente il punto di connessione alla rete. Le applicazioni moderne richiedono trattamento differenziato: le applicazioni real-time (VoIP, videoconferenza) richiedono bassa latenza e alta priorità, mentre i trasferimenti bulk (backup) richiedono ampia banda ma tollerano ritardi.



**Figura 6.1.** 0 — Oltre il 70% del traffico nei data center moderni è East-West (tra server). Le architetture tradizionali a tre livelli (Access, Aggregation, Core) erano ottimizzate per il traffico North-South (client-server) e richiedono selezione flessibile dei percorsi, load balancing efficiente e riconfigurazione rapida.

### 6.1.2 Limitazioni delle Reti Tradizionali

Le tendenze descritte richiedono una gestione flessibile delle risorse di rete, ma le architetture tradizionali mostrano tre limitazioni strutturali:

**A) Architettura statica e complessa:** le reti tradizionali sono costruite come collezione di protocolli indipendenti, ognuno pensato per una funzione specifica e separata (routing, spanning tree, ACL, ecc.). Il risultato è un control plane frammentato con complessità operativa crescente.

**B) Inconsistenza delle policy e rigidità operativa:** le modifiche di configurazione devono essere applicate manualmente su ogni singolo dispositivo. Le policy vengono enforced localmente, non globalmente, con alto rischio di misconfiguration.

**C) Dipendenza dal vendor e innovazione lenta:** la logica di controllo è embedded in hardware proprietario, con interfacce chiuse. Questo rende lento il deployment di nuovi servizi e limita la programmabilità.

## 6.2 Il Paradigma SDN

### 6.2.1 Separazione Control Plane / Data Plane

Le funzioni di rete a livello network si dividono in due categorie temporalmente molto diverse: il **forwarding** (spostare pacchetti dall'input all'output appropriato) avviene su scala temporale dei pacchetti (*fast timescales*), mentre il **routing** (determinare il percorso dei pacchetti da sorgente a destinazione) avviene su scala degli eventi di controllo (*slow timescales*). In una rete tradizionale, queste due funzioni sono tightly coupled nello stesso dispositivo.

#### Definizione – Software Defined Networking (SDN)

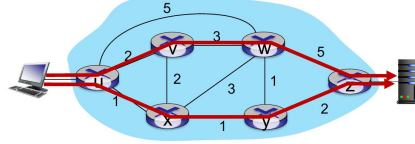
Approccio alla gestione di rete che usa l'astrazione per abilitare una configurazione e un'operazione della rete dinamica e programmaticamente efficiente. Separa il control plane dal data plane, centralizzando logicamente il primo in un controller remoto.

### Il Problema del Traffic Engineering Tradizionale

Nelle reti con routing distribuito, i router calcolano i percorsi indipendentemente usando algoritmi come Dijkstra (link-state) o Bellman-Ford (distance-vector). Il problema è che l'unico “knob” disponibile all'operatore

sono i pesi dei link: se si vuole forzare il traffico da u a z lungo il percorso uvwz invece di uxyz, bisogna ricalcolare i pesi, e questo può influenzare altri flussi in modo indesiderato.

### Traffic engineering: difficult with traditional routing



**Q:** what if network operator wants u-to-z traffic to flow along uvwz, rather than uxyz?

**A:** need to re-define link weights so traffic routing algorithm computes routes accordingly (or need a new routing algorithm)!

*link weights are only control “knobs”: not much control!*

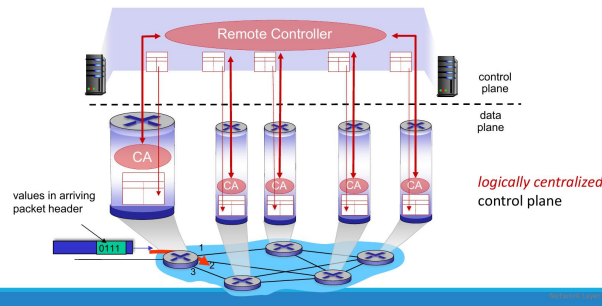
**Figura 6.2.** 0.1 — Con il routing tradizionale, i link weights sono gli unici “knob” di controllo. Non è possibile: (a) forzare percorsi specifici senza alterare il routing globale, (b) fare load balancing su più percorsi, (c) differenziare traffico di colori diversi sullo stesso nodo.

SDN risolve questo problema spostando la logica di routing in un controller centralizzato che ha visione globale della rete e può installare regole arbitrarie in ogni switch.

### Control Plane Logicamente Centralizzato

### Software-Defined Networking (SDN) control plane

Remote controller computes, installs forwarding tables in routers



**Figura 6.3.** 0.2 — Nel modello SDN, un Remote Controller logicamente centralizzato calcola le forwarding table e le installa nei router. I dispositivi del data plane (con etichetta CA, Controlled Agent) eseguono le istruzioni ricevute senza logica di routing autonoma.

In una rete tradizionale, ogni router calcola la propria forwarding table come:

$$T_i = \text{SPF}(\text{Topology, link weights})$$

Il comportamento globale emerge da computazioni locali non coordinate. Non vengono considerati la traffic demand (traffic matrix) né le policy applicative.

In SDN, un controller centralizzato calcola:

$$\mathcal{F} : S(t) \rightarrow \{T_1, T_2, \dots, T_n\}$$

dove  $S(t)$  include topologia, stato del traffico e policy. Le forwarding table vengono calcolate in modo **consistente** su tutta la rete.

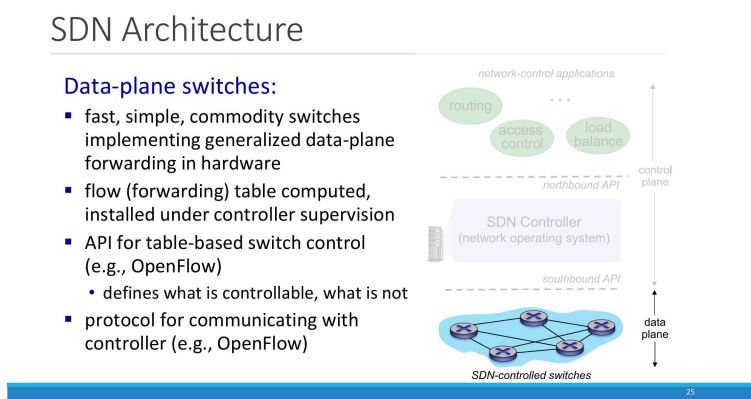


## I Quattro Pilastri di SDN

SDN si fonda su quattro idee principali:

1. **Generalized flow-based forwarding:** il forwarding non si basa solo sull'indirizzo IP di destinazione, ma può usare qualsiasi campo dell'header (OpenFlow)
2. **Separazione control plane / data plane:** i dispositivi del data plane eseguono il forwarding senza prendere decisioni di routing autonome
3. **Control plane esterno agli switch:** la logica di controllo risiede nel controller, non nei dispositivi di rete
4. **Programmabilità:** le applicazioni di controllo implementano funzioni di rete (routing, access control, load balancing) usando le API del controller

## 6.2.2 Architettura SDN a Tre Livelli



**Figura 6.4.** 0.3 — L'architettura SDN è organizzata in tre livelli: le applicazioni di controllo in cima, il controller (network OS) al centro, e gli switch programmabili in basso. Le NorthBound API collegano controller e applicazioni; le SouthBound API (es. OpenFlow) collegano controller e switch.

**Data-plane switches:** switch veloci e semplici (*commodity*) che implementano il generalized forwarding in hardware. Le flow table sono calcolate e installate sotto la supervisione del controller. L'API per il controllo (es. OpenFlow) definisce cosa è controllabile e cosa non lo è.

**SDN Controller** (*network operating system*): mantiene le informazioni di stato della rete, interagisce con le applicazioni via NorthBound API e con gli switch via SouthBound API. È implementato come sistema distribuito per performance, scalabilità, fault-tolerance e robustezza.

**Network-control applications:** il “cervello” del sistema, implementano le funzioni di controllo (routing, access control, load balancing) usando i servizi di basso livello offerti dal controller. Possono essere fornite da terze parti, separate dal vendor del controller o degli switch.

## 6.2.3 Breve Storia di SDN

| Anno  | Evento                                                                              |
|-------|-------------------------------------------------------------------------------------|
| ~2004 | Ricerca su nuovi paradigmi di gestione (Princeton, Stanford/Berkeley)               |
| 2008  | NOX Network OS (NICIRA), interfaccia OpenFlow (Stanford/NICIRA)                     |
| 2011  | Open Networking Foundation — board: Google, Yahoo, Verizon; membri: Cisco, HP, Dell |
| 2013  | 1600 partecipanti all'Open Networking Summit; SDN usato nel WAN di Google (B4)      |

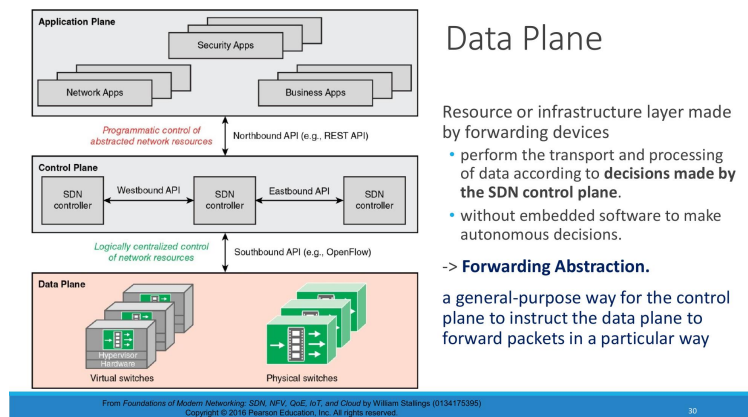
| Anno  | Evento                                                                                                                    |
|-------|---------------------------------------------------------------------------------------------------------------------------|
| 2014  | SDN nei data center: VMware NSX, Cisco ACI                                                                                |
| 2021+ | SDN guidato da AI/ML (Cisco DNA Center, Juniper Mist AI), SDN in reti 5G, IoT, edge; P4 per switch hardware programmabili |

#### Nota – Da protocolli distribuiti a controllo programmabile

La traiettoria di SDN riflette una tendenza più ampia nell'informatica: la separazione tra *policy* (cosa fare) e *meccanismo* (come farlo). Così come i sistemi operativi hanno astratto l'hardware dai programmi applicativi, SDN astrae l'infrastruttura di rete dalle applicazioni di controllo, abilitando innovazione indipendente a ogni livello.

## 6.3 SDN Data Plane

Il data plane, chiamato anche *resource layer* o *infrastructure layer*, è composto da dispositivi di forwarding — switch fisici e virtuali — che trasportano e processano i dati secondo le decisioni del control plane. L'elemento chiave che distingue un dispositivo SDN da un router tradizionale è l'assenza di software autonomo per prendere decisioni di instradamento: il dispositivo esegue soltanto le istruzioni che riceve dal controller.



**Figura 6.5.** 1 — L'architettura SDN stratificata: il Data Plane è il livello inferiore, composto da switch che eseguono il forwarding secondo le regole imposte dal Control Plane via SouthBound API (es. OpenFlow).

#### Definizione – Forwarding Abstraction

Un meccanismo generico e standardizzato con cui il control plane istruisce il data plane su come instradare i pacchetti. Non è legata a un tipo specifico di dispositivo: vale tanto per router quanto per switch o firewall.

### 6.3.1 Forwarding Generalizzato e Match+Action

Nel forwarding tradizionale basato sulla destinazione (*destination-based forwarding*), un router confronta l'indirizzo IP di destinazione del pacchetto con la propria tabella di routing e lo instrada di conseguenza. Il forwarding generalizzato di SDN generalizza radicalmente questo concetto: qualsiasi campo dell'header del pacchetto — a livello link, rete o trasporto — può essere usato come chiave di corrispondenza (*match*), e le azioni possibili sono molto più ricche del semplice forwarding.

**Tip – L’astrazione Match+Action**

Un pacchetto arriva allo switch → lo switch confronta i campi dell’header con le voci della **flow table** → se trova corrispondenza, esegue l’**azione** associata. Questa astrazione unifica in un unico modello dispositivi che in passato erano separati: router, switch, firewall, NAT.

Le azioni possibili includono: - **drop**: scarta il pacchetto (firewall) - **forward**: invia il pacchetto su una porta specifica (router/switch) - **modify**: modifica campi dell’header (NAT, VLAN rewriting) - **send to controller**: invia il pacchetto al controller per decisione centralizzata

**6.3.2 Flow Table e Astrazione di Flusso**

Un **flusso** (*flow*) è definito dall’insieme dei valori che i campi header possono assumere. La **flow table** contiene regole della forma:

| Componente      | Descrizione                                                                 |
|-----------------|-----------------------------------------------------------------------------|
| <b>Match</b>    | Pattern da confrontare con i campi header (wildcard * per qualsiasi valore) |
| <b>Action</b>   | Cosa fare con il pacchetto che corrisponde                                  |
| <b>Priority</b> | Priorità per risolvere ambiguità tra regole sovrapposte                     |
| <b>Counters</b> | Contatori di byte e pacchetti per statistica                                |

I campi su cui si può effettuare il match coprono tre livelli OSI contemporaneamente: porta di ingresso, MAC sorgente/destinazione, tipo Ethernet, VLAN ID, IP sorgente/destinazione, protocollo IP, ToS, porte TCP/UDP sorgente e destinazione. Questa flessibilità consente di implementare policy di rete complesse con un singolo meccanismo uniforme.

**Tip – Esempi di regole OpenFlow**

- **Forwarding IP classico**: IP Dst = 51.6.0.8 → forward(port 6) — tutti i datagrammi verso quell’IP vengono instradati sulla porta 6.
- **Firewall (blocca SSH)**: TCP dst-port = 22 → drop — blocca tutti i pacchetti TCP destinati alla porta 22.
- **Blocco per sorgente**: IP Src = 128.119.1.1 → drop — blocca tutto il traffico da quell’host.
- **L2 forwarding**: Eth Dst = 22:A7:23:11:E1:02 → forward(port 3) — forwarding basato su MAC address.

**6.3.3 OpenFlow**

**OpenFlow** è il protocollo standard che implementa la *SouthBound API* di SDN. Definisce come il controller comunica con i dispositivi di forwarding per installare, modificare e rimuovere regole nella flow table. Il primo paper che lo ha introdotto è di Nick McKeown et al. (2008) e ha dato origine a un intero ecosistema gestito oggi dalla *Open Networking Foundation (ONF)*.

**Definizione – OpenFlow**

Protocollo di comunicazione nelle architetture SDN che permette ai controller di consegnare *forwarding entries* ai dispositivi di data plane (switch e router, fisici e virtuali). È la principale implementazione della SouthBound API.

Sebbene l’adozione in ambienti di produzione rimanga limitata, OpenFlow ha un valore fondamentale per la ricerca e la didattica, avendo ispirato soluzioni più avanzate come **P4** (p4.org) per la programmazione generalizzata del data plane.

### Struttura di una Flow Entry OpenFlow

Ogni voce nella flow table OpenFlow ha tre campi fondamentali:

1. **Match:** i campi header da confrontare (tutti i livelli)
2. **Action:** le operazioni da eseguire (forward, drop, modify header, encapsulate e invia al controller)
3. **Stats:** contatori di pacchetti e byte per monitoraggio

### OpenFlow come Astrazione Unificante

#### Tip – Un solo modello per tutti i dispositivi

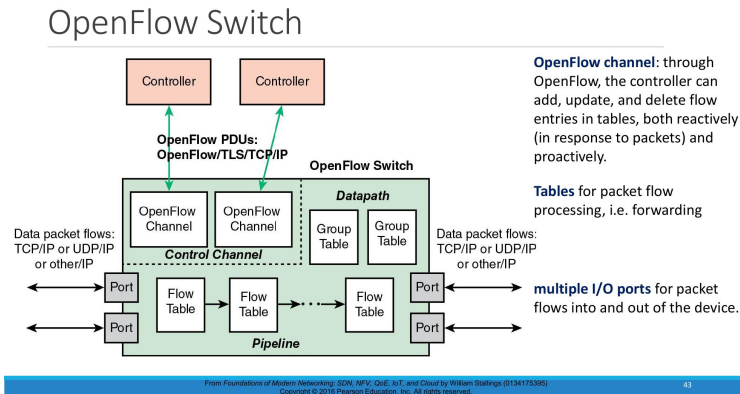
| Dispositivo | Match                    | Action                 |
|-------------|--------------------------|------------------------|
| Router      | Longest prefix IP dst    | Forward su link        |
| Switch L2   | MAC address destinazione | Forward o flood        |
| Firewall    | IP + TCP/UDP port        | Permit o deny          |
| NAT         | IP address e port        | Rewrite address e port |

Tutti questi dispositivi, storicamente separati, diventano istanze della stessa astrazione match+action con flow table diverse.

### 6.3.4 OpenFlow Switch e Flow Table Pipeline

Un OpenFlow switch ha tre componenti principali:

1. **Porte I/O:** le interfacce fisiche attraverso cui entrano ed escono i pacchetti
2. **Flow Tables (Datapath):** tabelle organizzate in pipeline per il processing dei pacchetti
3. **OpenFlow Channel:** il canale di comunicazione sicuro (TCP, opzionalmente TLS) con il controller



**Figura 6.6.** 2 — L'OpenFlow Switch: il canale OpenFlow collega lo switch al controller, che può aggiungere, aggiornare e rimuovere flow entries sia in modo reattivo (in risposta ai pacchetti) che proattivo.

### Flow Table Pipeline

Uno switch può contenere **una o più flow table** organizzate in **pipeline**. Le tabelle sono numerate sequenzialmente a partire da 0. Il pacchetto viene processato prima dalla tabella 0, e in base al risultato del match può essere diretto ad altre tabelle, a una group table, a una meter table, o infine all'uscita.

Packet in → [Table 0] → [Table 1] → ... → [Table N] → Packet out

L'uso di più tabelle in pipeline offre al controller notevole flessibilità: permette, ad esempio, di separare le regole di sicurezza da quelle di routing in tabelle distinte, applicandole in sequenza.

### Processing Pipeline

Quando un pacchetto viene processato da una flow table: - Se c'è **corrispondenza** con una o più entry, viene eseguita la regola con **priorità più alta**. Le istruzioni possono aggiornare l'action set, aggiornare metadata, eseguire azioni, o redirigere il pacchetto con **Goto** a un'altra tabella. - Se c'è corrispondenza solo con la **table-miss entry** (entry speciale per pacchetti senza match), si può: (a) inviare al controller, (b) redirigere a un'altra tabella, (c) scartare. - Se **non c'è alcun match** e non esiste table-miss entry, il pacchetto viene **scartato**.

### 6.3.5 Group Tables

Oltre alle flow table, OpenFlow definisce le **group table**. Un pacchetto che corrisponde a una flow entry può essere diretto non a una singola porta di uscita, ma a un *group entry*, che specifica come il pacchetto deve essere gestito su più porte.

Ogni group entry ha: - Un **Group ID** univoco - Un **Group Type** che definisce la semantica - Uno o più **action bucket** con le azioni possibili

| Group Type    | Comportamento                                                                      |
|---------------|------------------------------------------------------------------------------------|
| <b>ALL</b>    | Il pacchetto viene inviato a tutti i bucket (multicast/broadcast)                  |
| <b>SELECT</b> | Il pacchetto viene inviato a un singolo bucket scelto per load balancing o hashing |

#### Tip – Group Table in azione

- Group 1 (ALL): Output su Port 2 e Port 3 → flooding su entrambe le porte
- Group 2 (SELECT): Output su Port 4 o Port 5 → bilanciamento del carico tra due link

## 6.4 SDN Control Plane

Il control plane SDN è il livello di intelligenza della rete. Centralizza la logica di instradamento, il calcolo dei percorsi e la gestione delle policy in un componente software chiamato **controller SDN**, che mantiene una visione globale e consistente dello stato della rete.

### 6.4.1 Componenti del Controller SDN

Il controller SDN è organizzato in tre strati funzionali:

**Strato di comunicazione (*Communication Layer*)**: gestisce la comunicazione bidirezionale tra il controller e i dispositivi del data plane. I protocolli utilizzati sono OpenFlow (SouthBound API), SNMP, e altri.

**Strato di gestione dello stato di rete (*Network-Wide State Management*)**: mantiene un database distribuito con le informazioni sullo stato dell'intera rete: topologia dei link, informazioni sugli host, flow table degli switch, statistiche. È il cuore del controller, poiché fornisce la *global network view* necessaria per decisioni di routing ottimali.

**Strato di interfaccia verso le applicazioni (*Interface Layer*):** espone API (*NorthBound API*) alle applicazioni di controllo (routing, access control, load balancing, orchestrazione). Queste API permettono alle applicazioni di interagire con la rete in modo astratto, senza conoscere i dettagli del protocollo SouthBound.

### 6.4.2 Protocollo OpenFlow — Messaggi

OpenFlow opera tra controller e switch tramite **TCP** (opzionalmente cifrato con TLS). I messaggi sono divisi in tre classi:

#### Controller → Switch:

| Messaggio                     | Funzione                                                                        |
|-------------------------------|---------------------------------------------------------------------------------|
| <b>features</b>               | Il controller interroga lo switch sulle sue capacità, lo switch risponde        |
| <b>configure</b>              | Il controller legge o imposta parametri di configurazione dello switch          |
| <b>modify-state (FlowMod)</b> | Aggiunge, elimina o modifica flow entries nelle tabelle OpenFlow                |
| <b>packet-out</b>             | Il controller invia un pacchetto incapsulato a una porta specifica dello switch |

#### Switch → Controller:

| Messaggio           | Funzione                                                              |
|---------------------|-----------------------------------------------------------------------|
| <b>packet-in</b>    | Lo switch trasferisce un pacchetto (e il suo controllo) al controller |
| <b>flow-removed</b> | Notifica che una flow entry è stata eliminata dallo switch            |
| <b>port-status</b>  | Informa il controller di un cambiamento di stato su una porta         |

#### Attenzione – I network operator non programmano OpenFlow direttamente

In pratica, gli operatori usano astrazioni di alto livello fornite dal controller (es. intent framework in ONOS). La scrittura diretta di messaggi OpenFlow è riservata allo sviluppo interno del controller.

### 6.4.3 Interazione Control Plane / Data Plane — Esempio

Un esempio concreto illustra la collaborazione tra i componenti. Si consideri un guasto su un link della rete:

1. Lo switch **S1** rileva il guasto e invia al controller un messaggio **port-status** per notificare il cambiamento di stato del link.
2. Il **controller SDN** riceve il messaggio e aggiorna il proprio database di stato dei link.
3. L'applicazione di routing **Dijkstra** (che si è registrata per ricevere notifiche sui cambiamenti di link) viene invocata.
4. Dijkstra accede al grafo della rete e alle informazioni di link-state nel controller e calcola i **nuovi percorsi ottimali**.
5. L'applicazione di routing interagisce con il componente di calcolo delle flow table nel controller per determinare le nuove regole da installare.
6. Il controller invia messaggi **FlowMod** via OpenFlow agli switch interessati per installare le nuove flow table.

**Nota – SDN vs routing distribuito tradizionale**

In una rete tradizionale, i router eseguono OSPF o BGP e calcolano i percorsi autonomamente scambiandosi informazioni. In SDN, questa logica è centralizzata nel controller: questo semplifica il coordinamento, ma richiede robustezza e alta disponibilità del controller stesso.

#### 6.4.4 Implementazioni di Controller SDN

| Controller                | Note                                                         |
|---------------------------|--------------------------------------------------------------|
| <b>NOX</b>                | Primo controller SDN, riferimento storico                    |
| <b>Ryu</b>                | Framework Python per SDN                                     |
| <b>OpenDaylight (ODL)</b> | Controller open source enterprise-grade                      |
| <b>ONOS</b>               | Open Network OS, forte enfasi su distribuzione e scalabilità |
| <b>TeraFlow SDN</b>       | Controller per reti cloud-native, progetto ETSI              |

##### OpenDaylight (ODL)

OpenDaylight è uno dei controller SDN più diffusi in ambienti enterprise. La sua architettura è stratificata:

- **Northbound API:** REST/RESTCONF/NETCONF per le applicazioni (Traffic Engineering, Firewalling, Load Balancing, Orchestration)
- **Basic Network Functions:** moduli interni per topology processing, switch management, statistiche, host tracking, gestione regole di forwarding
- **Service Abstraction Layer (SAL):** il bus interno che interconnette applicazioni, servizi interni e protocolli SouthBound
- **Southbound API:** OpenFlow, NETCONF, SNMP, OVSDB

**Tip – SAL in ODL**

Il Service Abstraction Layer è l'elemento architetturale chiave di ODL: agisce da mediatore tra il mondo delle applicazioni (Northbound) e i protocolli di rete specifici (Southbound), rendendo il controller agnostico rispetto al protocollo SouthBound utilizzato.

##### ONOS

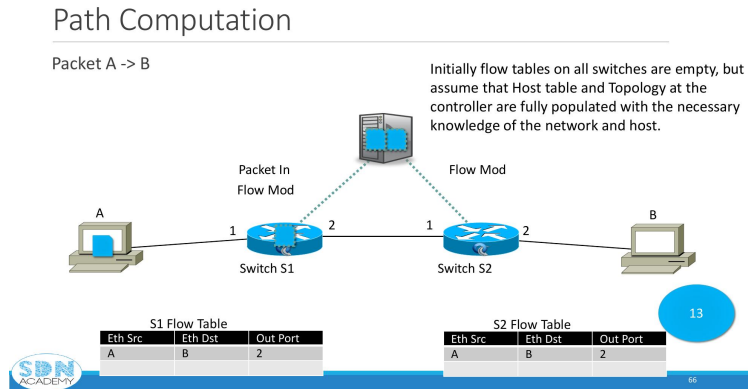
ONOS (*Open Network Operating System*) enfatizza la separazione delle applicazioni di controllo dal core del controller e la robustezza distribuita:

- Le **applicazioni di controllo** (Traffic Engineering, Firewalling, Load Balancing) sono separate dal controller core e comunicano via Northbound API
- L'**intent framework** permette alle applicazioni di specificare *cosa* si vuole ottenere (es. connettività tra due host) piuttosto che *come* (es. quale percorso usare): il controller si occupa di tradurre l'intent in flow rules concrete
- Il **distributed core** garantisce alta disponibilità e scalabilità attraverso replicazione e consensus distribuito

## 6.5 Forwarding e Topology Discovery

Un aspetto cruciale in una rete SDN è come il controller costruisce la conoscenza della topologia e come gestisce il forwarding dei pacchetti prima che le flow table siano popolate.

### 6.5.1 Path Computation



**Figura 6.7.** 3 — Path computation: le flow table degli switch sono inizialmente vuote. Il primo pacchetto da A a B genera un Packet-In; il controller calcola il percorso e installa le regole via FlowMod.

Quando un pacchetto arriva a uno switch e non trova corrispondenza nella flow table (*table-miss*), lo switch lo invia al controller tramite un messaggio **Packet-In**. Il controller, disponendo di una Host Table e di una Topology Table già popolate, calcola il percorso ottimale da sorgente a destinazione e invia messaggi **FlowMod** agli switch lungo il percorso.

#### Attenzione – Ordine di installazione delle flow entries

I messaggi FlowMod vengono inviati in **ordine inverso**: prima allo switch più vicino alla destinazione, poi risalendo verso la sorgente. Questo garantisce che quando il pacchetto arriva al secondo switch, la regola sia già installata, evitando un secondo Packet-In al controller.

Una volta installate le regole, tutti i pacchetti successivi dello stesso flusso vengono gestiti direttamente dagli switch, senza passare dal controller.

### 6.5.2 Routing Centralizzato

In una rete tradizionale, la funzione di routing è distribuita tra i router, ognuno dei quali mantiene la propria tabella di routing e partecipa a protocolli come OSPF. In SDN, è naturale centralizzare questa funzione nel controller, che:

- Ha una visione **globale e coerente** dello stato della rete
- Può implementare **policy application-aware** (es. preferire certi link per certi tipi di traffico)
- Libera gli switch dal burden computazionale e di storage del routing, migliorandone le prestazioni

Il routing centralizzato svolge due funzioni distinte:

1. **Link/Topology Discovery**: il controller deve conoscere i link tra gli switch del data plane (non standardizzata in OpenFlow, ogni implementazione ha il proprio approccio)
2. **Topology Manager**: mantiene la topologia aggiornata e calcola i percorsi più brevi tra nodi



### 6.5.3 Inizializzazione degli Switch

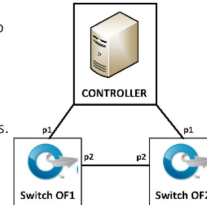
#### Switch Initialization

When the switch is initialized:

- it attempts to establish a connection with the controller
- the controller sends a message (**FEATURE REQUEST MESSAGE**) to the switch
- the switch responds with a message (**FEATURE REPLY MESSAGE**)
- it informs the controller of relevant parameters for the discovery of the links like the **Switch ID**, a **list of active ports**, among others.

At the end of the initial handshake, the controller knows the exact number of active ports on the OF switches

But it doesn't know the physical connections between switches, so it needs to build it -> **Topology discovery**



**Figura 6.8.** 4 — *Switch Initialization: al boot, lo switch stabilisce una connessione TCP con il controller. Lo scambio Feature Request / Feature Reply comunica al controller i parametri dello switch (Switch ID, porte attive).*

Quando uno switch OpenFlow viene acceso, esegue una procedura di handshake con il controller:

1. Lo switch tenta di stabilire una **connessione TCP** con l'indirizzo del controller (pre-configurato)
2. Il controller invia un messaggio **FEATURE REQUEST**
3. Lo switch risponde con un messaggio **FEATURE REPLY** contenente: Switch ID, lista delle porte attive, e altri parametri rilevanti

Al termine dell'handshake, il controller sa esattamente quante porte attive ha ogni switch, ma **non conosce ancora le connessioni fisiche tra switch**. Per questo è necessaria la topology discovery.

### 6.5.4 Topology Discovery con LLDP

La topology discovery in reti OpenFlow si basa sul **Link Layer Discovery Protocol (LLDP)**, standardizzato da IEEE nel 2005 e 2009. LLDP è un protocollo *vendor-neutral* che opera al Livello 2 del modello OSI e permette a un dispositivo di annunciare la propria identità e le proprie capacità ai dispositivi adiacenti.

Gli switch OpenFlow hanno due configurazioni iniziali che abilitano la topology discovery: 1. **Indirizzo del controller**: ogni switch ha pre-configurato l'IP e la porta TCP del controller, a cui si connette al boot 2. **Flow rule per LLDP**: ogni switch ha una regola pre-installata che invia al controller (via Packet-In) qualsiasi frame LLDP ricevuto

#### Struttura del Frame LLDP

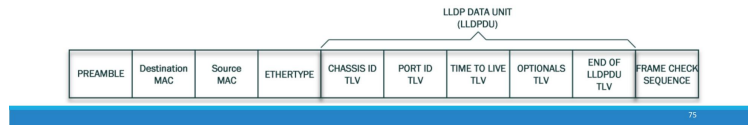
Il frame LLDP usa Ethernet come protocollo di trasporto (EtherType 0x88cc) e contiene i seguenti campi TLV (*Type-Length-Value*):

| TLV           | Tipo | Contenuto                                                        |
|---------------|------|------------------------------------------------------------------|
| Chassis ID    | 1    | Identificatore dello switch che invia il frame LLDP              |
| Port ID       | 2    | Identificatore della porta attraverso cui il frame viene inviato |
| Time to Live  | 3    | Validità in secondi delle informazioni contenute nel frame       |
| End of LLDPDU | 4    | Marcatore di fine payload                                        |

## LLDP frame

Typically, LLDP uses Ethernet as its "transport" protocol. The Ethernet type for LLDP is 0x88cc.

- Chassis ID (Type 1): contains the identifier of the switch that sends the LLDP packet.
- Port ID (Type 2): contains the identifier of the port through which the LLDP packet is sent.
- Time to Live (Type 3): time in seconds during which the information received in the LLDP packet is going to be valid.
- End of LLDPDU (Type 4): indicates the end of the payload in the LLDP frame



**Figura 6.9.** 5 — Struttura del frame LLDP. Il tipo Ethernet è 0x88cc. I TLV (Type-Length-Value) fondamentali sono: Chassis ID (switch mittente), Port ID (porta mittente), TTL, End of LLDPDU.

### Processo di Topology Discovery

Il processo si svolge in quattro passi:

1. Il controller genera un **Packet-Out** per ogni porta attiva su ogni switch scoperto, incapsulando al suo interno un frame LLDP con il Chassis ID e il Port ID dello switch sorgente.
2. Quando uno switch OF riceve un messaggio Packet-Out con un frame LLDP, lo **forwarda** sulla porta specificata nel Port ID verso lo switch adiacente.
3. Lo switch adiacente, ricevendo il frame LLDP su una porta non di controllo, lo **incapsula in un Packet-In** indirizzato al controller, includendo i propri Switch ID e Port ID come metadata.
4. Il controller riceve il Packet-In ed **estrae le informazioni**: conosce ora lo Switch ID e Port ID del frame LLDP (mittente originale) e lo Switch ID e Port ID del Packet-In (ricevente). Da questi dati deduce che esiste un link tra quei due switch su quelle specifiche porte.

#### Tip – Scalabilità della topology discovery

In una rete con  $S$  switch e un totale di  $P$  porte attive, il numero di messaggi Packet-Out inviati dal controller è pari a  $P$  (uno per porta attiva su ciascuno switch). L'intero processo viene eseguito periodicamente per rilevare cambiamenti nella topologia.

#### Nota – Topology Discovery non standardizzata

Il meccanismo di topology discovery non è standardizzato in OpenFlow: ogni implementazione di controller (ODL, ONOS, Ryu) può adottare varianti proprie. La procedura basata su LLDP descritta qui è quella tipicamente usata.

### 6.5.5 Host Reachability

La scoperta della raggiungibilità degli host avviene in modo simile alla topology discovery, ma è **event-driven**: viene tipicamente scatenata da traffico sconosciuto che entra nel dominio SDN da un host collegato o da un router vicino.

Quando un host A invia il primo pacchetto verso B e lo switch non ha una regola corrispondente, invia un Packet-In al controller. Il controller: 1. Registra nella **Host Table** l'indirizzo Ethernet di A, lo switch a cui è collegato, e la porta 2. Calcola il percorso verso B (se B è già nella Host Table) oppure avvia una procedura di discovery

La Host Table cresce progressivamente man mano che gli host comunicano, costruendo una mappa completa della rete.

## Capitolo 7

# SDN Applications e P4 Language

Questa lezione affronta tre casi d'uso concreti dell'[[Lezione 15 - (Lab) SDN Architecture]]: il caso reale di **B4**, la rete WAN globale di Google; il linguaggio **P4** per la programmazione del data plane; e il paradigma del **Service Function Chaining**. Il filo conduttore è la progressione da una rete tradizionale — rigida, sovradimensionata, costosa — verso un'infrastruttura completamente programmabile, dove sia il piano di controllo che il piano dati sono definiti via software.

---

### 7.1 B4 — La WAN Software-Defined di Google

#### 7.1.1 Il problema: WAN tradizionali e loro limiti

Le reti WAN tradizionali nascono attorno a un principio di conservatività: i link intercontinentali sono costosi, la perdita di pacchetti è inaccettabile, e dunque si impiegano router di fascia altissima con un significativo sovrapprovisionamento della capacità. Il risultato è che i link WAN vengono utilizzati mediamente al **30–40%** della loro capacità, richiedendo un overprovisioning di 2–3x. Questo approccio tende inoltre a trattare tutto il traffico allo stesso modo: tutte le applicazioni sono considerate ugualmente importanti, senza distinzione di priorità o sensibilità alla latenza.

Google si è trovata davanti a una proiezione di costo insostenibile: crescere seguendo il modello tradizionale significava pagare 2–3 volte il costo di una WAN pienamente utilizzata. La soluzione è stata ridisegnare completamente la rete inter-datacenter.

#### 7.1.2 Cos'è B4

**B4** è la rete WAN globale proprietaria di Google, che connette i suoi datacenter tra loro — non è la rete Internet-facing per il traffico degli utenti. Google gestisce in realtà due backbone separati: **B2**, che porta il traffico Internet verso gli utenti (e cresce alla velocità di Internet), e **B4**, che gestisce il traffico inter-datacenter (cresciuto di 10x in 3,5 anni al momento della pubblicazione del paper originale).

#### 7.1.3 Caratteristiche del traffico B4

Il traffico che percorre B4 non è omogeneo. Le slide distinguono tre categorie principali, ordinate per sensibilità alla latenza e volume:

| Tipo di traffico                                 | Volume | Sensibilità latenza | Priorità |
|--------------------------------------------------|--------|---------------------|----------|
| Replicazione dati utente verso datacenter remoti | Basso  | Alta                | Alta     |

| Tipo di traffico                                         | Volume | Sensibilità latenza | Priorità |
|----------------------------------------------------------|--------|---------------------|----------|
| Accesso remoto a storage per computazione distribuita    | Medio  | Media               | Media    |
| Sincronizzazione di stato tra datacenter (bulk transfer) | Alto   | Bassa               | Bassa    |

Questa eterogeneità è cruciale: il traffico di tipo 3 è elastico — tollera riduzioni temporanee di banda e persino fallimenti — e costituisce la maggior parte del volume. Questo rende B4 **adatto al traffic engineering aggressivo**: si può rimandare o rallentare il traffico a bassa priorità per fare spazio a quello critico.

#### Tip – L’insight chiave di B4

Il traffico inter-datacenter è prevalentemente *bulk* ed *elastico*. A differenza del traffico utente, non ha bisogno di garanzie di bassa latenza costante. Questo apre la strada a ottimizzazioni aggressive della banda che sarebbero impossibili su una WAN tradizionale.

### 7.1.4 Caratteristiche di progetto di B4

B4 si distingue dalla WAN tradizionale per quattro proprietà fondamentali.

La prima è la **domanda di banda elastica**: le applicazioni tollerano failure temporanei e riduzioni di banda, il che permette di ottimizzare l’utilizzo senza sacrificare l’esperienza utente.

La seconda è il **numero moderato di siti**: poche decine di siti significa che le tabelle di routing restano piccole e gestibili — non serve la scalabilità da centinaia di migliaia di rotte tipica di un router ISP.

La terza è il **controllo fine-grained delle applicazioni**: le applicazioni stesse comunicano le loro priorità e possono controllare i burst, eliminando la necessità di overprovisioning dei link.

La quarta è la **sensibilità al costo**: i link intercontinentali privati sono estremamente costosi e devono essere utilizzati al 100% della capacità disponibile. Non ci si può permettere il lusso del 30–40% di utilizzo tipico delle WAN tradizionali.

### 7.1.5 Architettura SDN di B4

La transizione di Google verso SDN è avvenuta in tre stadi progressivi, che illustrano perfettamente la metodologia di migrazione graduale.

#### Stadio 1 — Rete di partenza

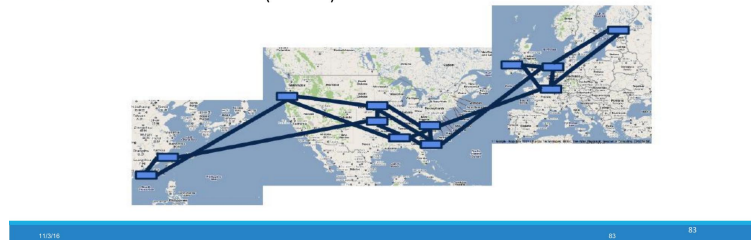
Nella configurazione iniziale, i *site* (nodi WAN) comunicano tra loro tramite protocolli distribuiti tradizionali: iBGP e ISIS per il routing interno ai siti, eBGP tra i siti. I *cluster* (reti datacenter) sono collegati ai siti tramite router BGP. Il controllo è interamente distribuito — nessun controller centralizzato.

#### Stadio 2 — Deployment parziale di OpenFlow

Nella fase intermedia, viene introdotto per alcuni switch il controller OpenFlow, affiancato da Quagga (il software di routing BGP/ISIS) e da un modulo “Glue” che coordina i due sistemi. Il consenso tra istanze ridondanti del controller è gestito tramite **Paxos**, garantendo fault tolerance. La rete è in uno stato ibrido: alcuni switch sono già controllati da OpenFlow, altri ancora dal piano di controllo distribuito tradizionale.

## B4 — Google's software defined WAN

- Google's globally deployed WAN, datacenter backbone
- private WAN connecting its data centers (NOT: Internet-facing backbone for user traffic)
- Modest number of sites (~ dozen)



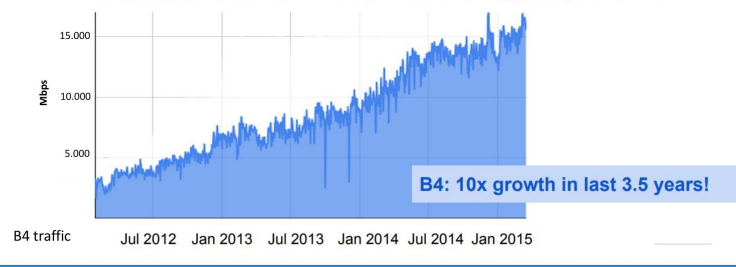
**Figura 7.1.** La rete B4 connette una dozzina di siti (datacenter) in tutto il mondo tramite link privati intercontinentali.

## Two Backbones



Two separate backbones:

- B2: Carries Internet facing traffic → Growing faster than the Internet
- B4: Inter-datacenter traffic → More traffic than B2, growing faster than B2

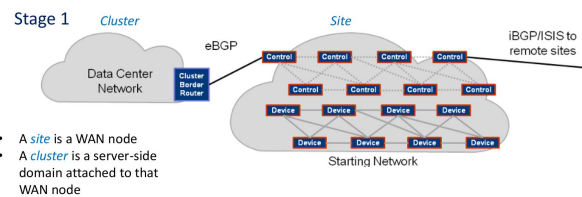


**Figura 7.2.** Traffico B4 dal 2012 al 2015: crescita di 10x in 3.5 anni, con B4 che supera B2 per volume.

## SDN Architecture

Traditional WAN and SDN:

Google's SDN migration path moved in stages from a fully distributed monolithic control and data plane hardware architecture to a physically decentralized (though logically centralized) control plane architecture

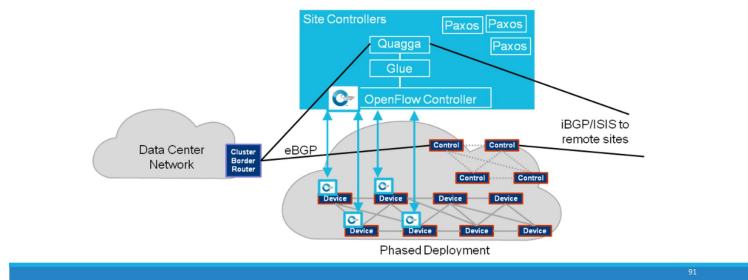


- A *site* is a WAN node
- A *cluster* is a server-side domain attached to that WAN node

**Figura 7.3.** Stage 1: la rete di partenza, con controllo distribuito basato su BGP/ISIS. Un sito è un nodo WAN; un cluster è il dominio server collegato a quel nodo.

## SDN Architecture

Stage 2



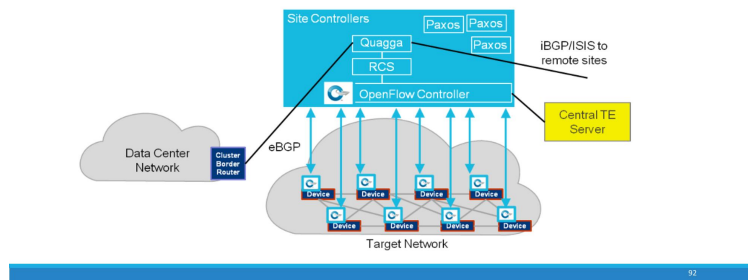
**Figura 7.4.** Stage 2: deployment graduale. I Site Controllers con OpenFlow coesistono con i nodi di controllo tradizionale. Paxos garantisce la fault tolerance del controller.

### Stadio 3 — Architettura target

Nella configurazione finale, tutti gli switch sono controllati da OpenFlow. Viene aggiunto il **Central TE Server** (Traffic Engineering Server) come applicazione globale che ottimizza l'intera rete. Il layer di controllo è fisicamente distribuito ma logicamente centralizzato.

## SDN Architecture

Stage 3



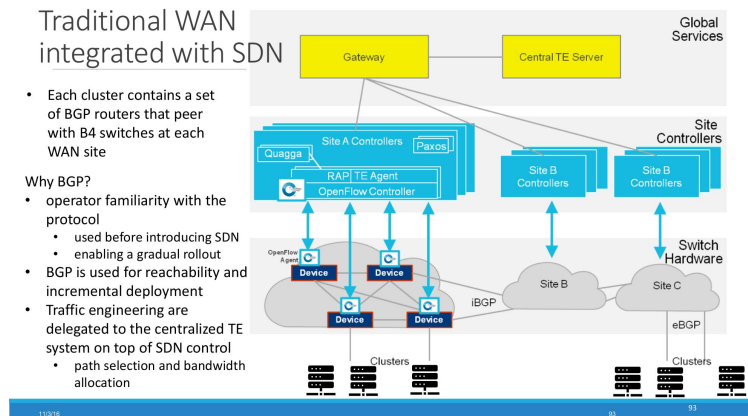
**Figura 7.5.** Stage 3: rete target. Tutti i dispositivi sono controllati via OpenFlow. Il Central TE Server fornisce ottimizzazione globale del traffico.

### Integrazione con BGP e piano di controllo tradizionale

Anche nell'architettura finale, BGP non scompare completamente. Ogni cluster mantiene router BGP che si appoggiano agli switch B4 tramite eBGP. BGP viene mantenuto per due ragioni pratiche: la familiarità degli operatori con il protocollo (già in uso prima di SDN) e la necessità di un deployment graduale e reversibile. BGP gestisce la *raggiungibilità* (reachability), mentre il **traffic engineering** — ovvero la selezione dei percorsi e l'allocazione della banda — è delegato interamente al sistema centralizzato SDN.

#### Nota – Perché mantenere BGP?

BGP non è la scelta tecnicamente “più pulita” in un sistema SDN puro, ma è mantenuto per due motivi pragmatici: (1) gli operatori lo conoscono bene e lo usavano già prima di SDN, facilitando una transizione graduale; (2) permette un rollout incrementale — si può migrare un sito alla volta senza interrompere il servizio.



**Figura 7.6.** L'architettura completa: BGP per la raggiungibilità, TE Server centralizzato per l'ottimizzazione del traffico. Gateway, Site Controllers e Switch Hardware interagiscono tramite OpenFlow.

### 7.1.6 Panoramica dell'architettura B4

L'architettura B4 si articola su tre layer sovrapposti.

Lo **Switch Hardware** è hardware custom di Google, progettato con silicio commodity ma senza software di controllo complesso. La riduzione del numero di siti elimina la necessità di grandi tabelle di forwarding; il controllo applicativo fine-grained riduce la necessità di grandi buffer.

Il **Control Layer** è composto da *Network Control Servers* (NCS) che ospitano sia gli **OpenFlow Controllers** (OFC) sia le **Network Control Applications** (NCA). Gli OFC mantengono lo stato della rete in base alle direttive delle NCA e agli eventi degli switch, e istruiscono gli switch ad aggiornare le tabelle di forwarding. Per la fault tolerance, un'istanza Paxos per sito elegge il controller primario tra più repliche fisiche su server differenti.

Il **Global Layer** contiene le applicazioni logicamente centralizzate: l'**SDN Gateway** e il **TE Server**. L'SDN Gateway astrae i dettagli di OpenFlow e dell'hardware degli switch per il TE Server — ogni sito B4 può avere centinaia di porte fisiche, ma il Gateway le aggrega in un'astrazione a livello di sito. Il TE Server opera su questa visione globale per calcolare path e allocazioni di banda.

### 7.1.7 Traffic Engineering in B4

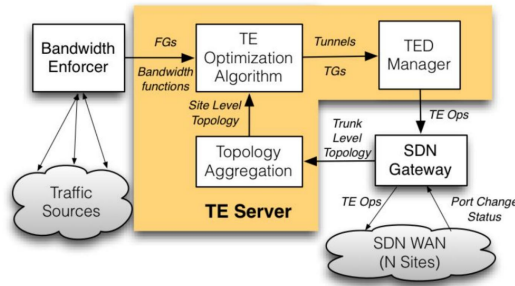
Il Traffic Engineering (TE) è il meccanismo attraverso cui B4 realizza l'utilizzo vicino al 100% dei link. L'obiettivo del TE Server è **condividere la banda tra applicazioni concorrenti**, possibilmente usando percorsi multipli.

Il TE Server lavora su due astrazioni principali.

In **input** riceve: (1) la *topologia di rete* come grafo di siti e connessioni sito-a-sito, che riduce enormemente la dimensione del problema rispetto a una topologia a livello di porta; (2) i *flow group* (FG), ciascuno definito come una tupla composta da sito sorgente, sito destinazione e classe QoS. Il TE non lavora a livello di singola applicazione — aggrega il traffico per sito e classe di servizio. Il **Bandwidth Enforcer** è un componente esterno che specifica le funzioni di banda per ogni applicazione, derivate da pesi statici configurati dall'amministratore.

In **output** produce: *Tunnel* — percorsi a livello di sito come sequenze di siti (es. A→B→C), implementati tramite IP-in-IP encapsulation — e *Tunnel Group* (TG), che mappano i flow group sui tunnel disponibili.

## B4 – Traffic Engineering



**Figura 7.7.** Il TE Server riceve Flow Groups e funzioni di banda dal Bandwidth Enforcer, ottimizza e produce Tunnels/Tunnel Groups che vengono poi installati nella rete via SDN Gateway e OFC.

### Definizione – Tunnel in B4

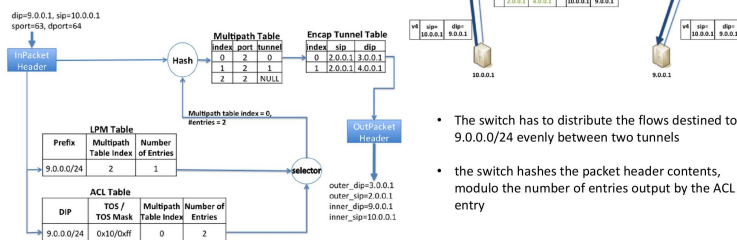
Un Tunnel rappresenta un percorso a livello di sito nella rete. È una sequenza di siti (es. Amsterdam → Frankfurt → Warsaw) implementata con incapsulamento IP-in-IP. Non coincide con un link fisico: può attraversare più switch e link interni a ogni sito.

I Tunnel Group e i relativi Tunnel e Flow Group vengono inviati dall'SDN Gateway agli OFC, che li installano negli switch via OpenFlow.

### Forwarding Multipath

Per sfruttare pienamente la banda disponibile, B4 utilizza il **forwarding multipath**: un flusso destinato a una subnet può essere distribuito su più tunnel in parallelo. Lo switch usa una funzione di hash sugli header del pacchetto (modulo il numero di entry ACL) per distribuire equamente i flussi tra i tunnel disponibili, garantendo che pacchetti dello stesso flusso seguano lo stesso percorso (evitando riordinamento).

### B4 – Multipath WAN Forwarding Example



**Figura 7.8.** Esempio di forwarding multipath: i flussi verso 9.0.0.0/24 vengono distribuiti equamente tra due tunnel tramite hash degli header del pacchetto.

### Nota – Risultati di B4

Grazie all'architettura SDN, Google ha raggiunto un utilizzo dei link vicino al **100%** per i carichi elastici — contro il 30–40% delle WAN tradizionali. Questo ha permesso di ridurre costi e complessità dell'infrastruttura, eliminando il sovrapprovisionamento da 2–3x. Ulteriori miglioramenti successivi hanno introdotto topologie gerarchiche e algoritmi TE decentralizzati per migliorare scalabilità e disponibilità.



## 7.2 Verso un Data Plane Programmabile

### 7.2.1 Limiti di OpenFlow

OpenFlow ha dimostrato il valore della separazione tra control plane e data plane, ma presenta limiti intrinseci al crescere delle esigenze. Il protocollo è nato semplice — circa 12 header field — ma la specifica è cresciuta fino a ~40 campi, aumentando la complessità. Soprattutto, i datacenter moderni richiedono nuove forme di encapsulamento dei pacchetti (VXLAN, NVGRE, ecc.) che OpenFlow non supporta in modo flessibile.

Il problema più profondo è concettuale: la vera programmabilità di rete richiederebbe la capacità di definire e modificare sia gli algoritmi del control plane sia quelli del **data plane**. Gli operatori di rete dovrebbero poter definire entrambi autonomamente, senza coinvolgere i progettisti originali dell'hardware. OpenFlow permette di *installare regole* nel data plane, ma non di *riprogrammare* il data plane stesso.

#### Tip – La distinzione cruciale

OpenFlow controlla *cosa fanno* le tabelle di un data plane fisso. P4 permette di definire *come è fatto* il data plane stesso — quali header riconoscere, come processarli, quali azioni eseguire.

## 7.3 P4 Language

### 7.3.1 Cos'è P4

**P4** (*Programming Protocol-Independent Packet Processors*) è un linguaggio di programmazione *domain-specific* progettato per dispositivi di rete. Il suo nome cattura l'essenza del progetto: permette di programmare il processamento dei pacchetti in modo indipendente dal protocollo sottostante.

P4 estende il paradigma match-action introdotto da OpenFlow, ma lo porta a un livello superiore: invece di installare regole in tabelle predefinite dall'hardware, con P4 il programmatore **definisce le tabelle stesse**, i campi degli header da estrarre, le azioni possibili e il flusso di controllo tra le tabelle.

#### Definizione – P4 — Proprietà fondamentali

Un programma P4 descrive integralmente il comportamento atteso nel data plane: - **Headers**: tutti gli header dei pacchetti e i campi rilevanti per il matching - **Actions**: l'insieme di tutte le azioni possibili supportate dalle tabelle - **Tables**: l'insieme delle tabelle di matching e il flusso di controllo tra esse  
In aggiunta agli header standard, i progettisti di rete possono definire header per protocolli personalizzati.

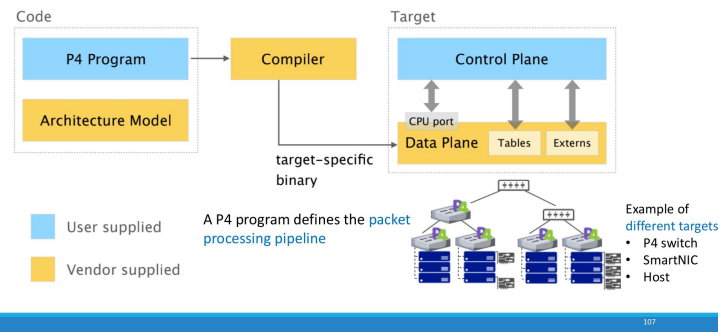
P4 è stato concepito nel 2013 da un team accademico-industriale e presentato all'ACM SIGCOMM 2014. La standardizzazione è coordinata dal **P4 Language Consortium** (specifiche su [p4.org/specs](http://p4.org/specs)).

### 7.3.2 Vantaggi e applicazioni di P4

P4 permette di **personalizzare il processamento dei pacchetti senza modifiche hardware**, accelerando drasticamente il ciclo di innovazione dei protocolli di rete. Le applicazioni principali includono: SDN avanzato, monitoring e telemetria di rete, implementazioni custom di sicurezza e firewall.

Il vantaggio competitivo rispetto all'approccio tradizionale è la separazione tra il *cosa* (il programma P4, scritto dall'utente) e il *come* (l'hardware o il compilatore, forniti dal vendor): lo stesso programma può essere compilato per switch P4, SmartNIC, o software su host.

## Programming a P4 target



**Figura 7.9.** Il programmatore scrive un P4 Program; il compilatore vendor-specifico genera una configurazione del data plane e un'API per il control plane (P4Runtime).

### 7.3.3 Compilazione e target P4

Un programma P4 viene scritto dal programmatore in termini dell'**Architecture Model** — una specifica astratta del target, fornita dal vendor. Il **compilatore P4**, anch'esso vendor-specifico, prende in input il programma e il modello architetturale e produce:

- Una **configurazione del data plane** che implementa la logica di forwarding descritta nel programma.
- Un'**API** per gestire lo stato degli oggetti del data plane dal control plane.

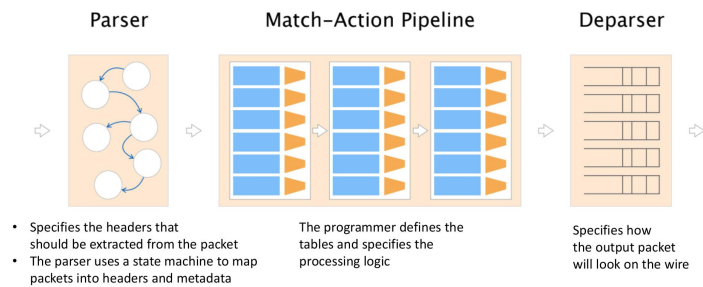
L'interazione a runtime tra control plane e data plane avviene tramite **P4Runtime**, una specifica standard per controllare gli elementi del data plane definiti da un programma P4. Il control plane popola le tabelle con le regole; quando arriva un match, l'azione selezionata viene eseguita con i parametri specificati dalla regola corrispondente.

I target supportati includono switch P4, SmartNIC e host software — la stessa logica di forwarding può essere eseguita su hardware diversissimi semplicemente ricompilando.

### 7.3.4 Struttura di un programma P4

Un programma P4 è composto da tre componenti principali, che corrispondono alle tre fasi del ciclo di vita di un pacchetto nel data plane.

## P4 program: main parts



**Figura 7.10.** Pipeline P4: il Parser estrae gli header (state machine), la Match-Action Pipeline applica le regole, il Deparser ricostruisce il pacchetto per il wire.

## Parser

Il **Parser** specifica come estrarre gli header dai pacchetti in ingresso. Usa una **macchina a stati** per mappare il flusso di byte in strutture header e metadata. Per esempio, si parte dallo stato `start`, si estrae l'header Ethernet, e in base al campo `eth_type` si transita allo stato `parse_ipv4` (per pacchetti IP) oppure si accetta direttamente il pacchetto.

```
header ethernet_t {
    bit<48> dst_addr;
    bit<48> src_addr;
    bit<16> eth_type;
}

header ipv4_t {
    bit<4>  version;
    bit<4>  ihl;
    bit<8>  diffserv;
    bit<16> totalLen;
    bit<16> identification;
    bit<3>  flags;
    bit<13> fragOffset;
    bit<8>  ttl;
    bit<8>  protocol;
    bit<16> hdrChecksum;
    bit<32> srcAddr;
    bit<32> dstAddr;
}

parser MyParser(packet_in packet,
                 out headers_t hdr,
                 inout metadata_t meta) {

    state start {
        packet.extract(hdr.ethernet);
        transition select(hdr.ethernet.eth_type) {
            0x0800: parse_ipv4;
            default: accept;
        }
    }

    state parse_ipv4 {
        packet.extract(hdr.ipv4);
        transition accept;
    }
}
```

Gli header vengono dichiarati con campi a larghezza fissa (es. `bit<48>` per un MAC address). La struttura `headers_t` li raggruppa in un unico oggetto che attraversa tutta la pipeline.

## Match-Action Pipeline

Il programmatore definisce le **tabelle di matching** e la logica di processing. Ogni tabella specifica i campi su cui fare match, le azioni possibili, e la dimensione massima. Quando un pacchetto arriva, viene confrontato con le regole della tabella (installate dal control plane); se trova un match, l'azione associata viene eseguita con i parametri specificati dalla regola.

**Nota – Differenza rispetto a OpenFlow**

In OpenFlow, le tabelle sono fisse nell'hardware: l'operatore può solo popolarle con regole. In P4, le tabelle stesse sono definite nel programma: il programmatore decide quante tabelle ci sono, su quali campi fare match, e quali azioni sono disponibili. Questo abilita protocolli completamente nuovi senza modificare l'hardware.

**Deparser**

Il **Deparser** specifica come ricostruire il pacchetto per l'output sul wire. Definisce l'ordine in cui gli header modificati vengono serializzati nel flusso di byte in uscita, permettendo anche di aggiungere o rimuovere header (es. per encapsulamento/decapsulamento).

**Nota – Sintesi del modello P4**

P4 porta la programmabilità al data plane con un modello coerente: Parser (deserializzazione) → Match-Action Pipeline (processamento) → Deparser (serializzazione). Il control plane (via P4Runtime) si occupa solo di popolare le tabelle; tutta la logica di parsing e azione è definita staticamente nel programma P4 e compilata nell'hardware.

## 7.4 Service Function Chaining

### 7.4.1 Il problema: comporre funzioni di rete

Le reti moderne non applicano ai pacchetti una singola trasformazione, ma una sequenza di elaborazioni: un pacchetto che entra da Internet può dover attraversare un firewall, un sistema di intrusion detection (IDS), e un load balancer prima di raggiungere il server applicativo. Ciascuna di queste elaborazioni è una **Network Function** — una capacità di processamento dei pacchetti fornita da un dispositivo di rete.

**Definizione – Service Function Chaining (SFC)**

Il **Service Function Chaining** è il paradigma per definire e gestire servizi di rete come catene ordinate di funzioni di rete. Una catena definisce come un flusso di traffico deve essere processato per raggiungere obiettivi di sicurezza, performance o QoS.

Esempi di funzioni di rete: NAT, firewall, IDS, DNS, caching, DDoS protection, load balancer.

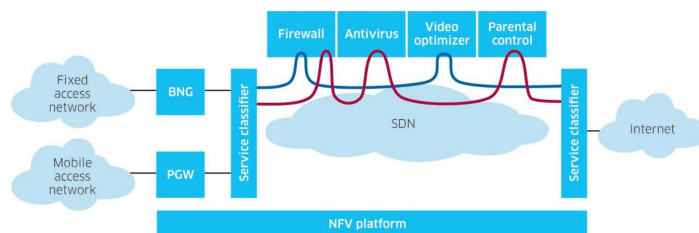
Nelle reti tradizionali, le service chain sono implementate topologicamente: i dispositivi fisici sono cablati in sequenza, e il traffico li attraversa nell'ordine fisico. Questo rende l'aggiornamento o il cambiamento delle chain estremamente rigido e costoso.

### 7.4.2 SFC con SDN

SDN risolve il problema della rigidità. Con SDN, le service chain sono **definite programmaticamente**: il controller SDN configura le regole di forwarding affinché i pacchetti attraversino le funzioni desiderate nell'ordine corretto, indipendentemente dalla topologia fisica. Un **Service Classifier** identifica i flussi e li instrada sulla chain appropriata; alla fine della chain, un secondo classificatore rilascia il traffico verso la destinazione finale.

Questo approccio si integra naturalmente con la **Network Function Virtualization (NFV)**: le funzioni di rete non devono essere implementate su hardware dedicato, ma possono girare come istanze software su una piattaforma NFV. SDN + NFV insieme abilitano chain completamente virtuali, modificabili in tempo reale senza intervento fisico.

## Service Function Chaining



SDN supports the dynamic definition of service chains allowing traffic flows to be steered through the desired chain

© <https://www.sdncentral.com/sdn/network-virtualization/definitions/what-is-network-service-chaining/>

114

**Figura 7.11.** SDN permette la definizione dinamica di service chain: il traffico viene instradato dal Service Classifier attraverso le funzioni desiderate (Firewall, Antivirus, Video Optimizer, Parental Control) su una piattaforma NFV, senza richiedere ricablaggio fisico.

### Tip – SDN + NFV: perché si completano

NFV virtualizza le funzioni di rete (le rende software). SDN controlla il forwarding e permette di comporre queste funzioni in catene dinamiche. Insieme, eliminano sia la dipendenza dall'hardware dedicato sia la rigidità topologica delle reti tradizionali.

## 7.5 Sintesi

### Nota – Punti chiave della lezione

Il routing tradizionale offre un controllo troppo grossolano: i pesi dei link e il forwarding destination-based non bastano per traffic engineering fine-grained e controllo delle policy.

**B4** dimostra che SDN consente utilizzo vicino al 100% dei link WAN inter-datacenter, contro il 30–40% delle WAN tradizionali — abbattendo costi e complessità. L'architettura è fisicamente distribuita ma logicamente centralizzata, con un TE Server globale che ottimizza l'intera rete.

**OpenFlow** è uno dei possibili meccanismi southbound di SDN — non è SDN stesso. I suoi limiti (data plane fisso, header predefiniti) hanno portato allo sviluppo di P4.

**P4** estende la programmabilità al data plane: non solo installa regole, ma permette di definire il parser, le tabelle, le azioni e il deparser. Il risultato è un data plane completamente personalizzabile, compilato per hardware diversi tramite un compilatore vendor-specifico.

**Service Function Chaining** con SDN permette di comporre funzioni di rete (firewall, IDS, load balancer...) in catene dinamiche, senza ricablaggio fisico — specialmente potente in combinazione con NFV.

## Capitolo 8

# Network Function Virtualization

La **Network Function Virtualization** (NFV) rappresenta uno dei cambiamenti di paradigma più significativi nell'ingegneria delle reti degli ultimi anni. L'idea fondamentale è semplice ma rivoluzionaria: spostare le funzioni di rete — firewall, bilanciatori di carico, gateway — dall'hardware proprietario dedicato a software eseguito su infrastruttura virtualizzata generica. Questa lezione ripercorre il problema che NFV risolve, i suoi benefici, la sua architettura e come i servizi di rete vengono effettivamente composti e distribuiti.

### 8.1 Il problema: middlebox e rigidità hardware

#### 8.1.1 Un mondo di middlebox

Le reti tradizionali non sono composte solo da router e switch. Accanto a questi elementi “trasparenti” proliferano i **middlebox**: dispositivi che eseguono funzioni di elaborazione avanzata oltre al semplice inoltro dei pacchetti. Firewall, sistemi di rilevamento delle intrusioni (NIDS), media gateway, load balancer, proxy, VPN gateway, WAN optimizer — in una grande impresa con oltre 80.000 utenti su decine di siti, questi apparati possono essere addirittura più numerosi dei router tradizionali.

#### Tip – Dati da una grande enterprise (Sekar et al., 2011)

| Tipo di apparato        | Numero     |
|-------------------------|------------|
| Firewall                | 166        |
| NIDS                    | 127        |
| Media gateway           | 110        |
| Load balancer           | 67         |
| Proxy                   | 66         |
| VPN gateway             | 45         |
| WAN Optimizer           | 44         |
| <b>Totale middlebox</b> | <b>636</b> |
| Totale router           | ~900       |

I middlebox sono quasi pari in numero ai router: la rete è molto più di semplice forwarding.

#### 8.1.2 Il modello tradizionale e i suoi limiti

Nell'industria delle telecomunicazioni tradizionale, ogni funzione di rete corrisponde a un **apparato hardware proprietario** dedicato. Il firewall è un box fisico, il load balancer è un altro box fisico, e così via. Questo modello impone vincoli severi:

- Le funzioni devono essere concatenate in un ordine preciso, riflesso nella topologia fisica della rete (es. il traffico deve passare prima per il firewall e poi per il load balancer — non si può cambiare senza ricablare).
- I fornitori devono garantire alti standard di qualità, performance e conformità ai protocolli, con cicli di sviluppo prodotto molto lunghi.
- Il risultato è una **scarsa agilità**: aggiungere una nuova funzione o scalare una esistente richiede acquisto di nuovo hardware, deployment manuale e formazione del personale.

Contemporaneamente, gli utenti richiedono servizi sempre più diversificati e di breve durata. Ogni nuovo servizio richiede nuovo hardware, deployment manuale e nuove competenze. Le reti diventano complesse, difficili da scalare e costose da operare: i costi **CAPEX** (*Capital Expenditure*, l'investimento in hardware e infrastruttura fisica) e **OPEX** (*Operational Expenditure*, i costi operativi di energia, manutenzione e personale) crescono, ma i ricavi no.

#### Attenzione – Il nodo del problema

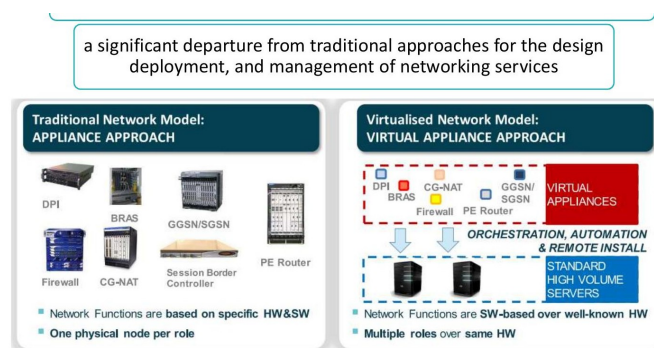
Il modello hardware-based è per natura *lento e rigido*. I provider di servizi Internet (Google, Amazon, Facebook) hanno invece dimostrato che un approccio *software-based* è *veloce e flessibile*. NFV nasce dalla volontà di portare questo modello software dentro le reti di telecomunicazione.

## 8.2 NFV: l'idea fondamentale

Nel 2012, i principali operatori globali (AT&T, Telefónica, Verizon e altri) avviarono un'iniziativa presso **ETSI** (European Telecommunications Standards Institute), dando vita all'Industry Specification Group for NFV. L'obiettivo: trasformare le reti attraverso la virtualizzazione.

### 8.2.1 Decoupling tra funzioni e hardware

Il principio cardine di NFV è il **disaccoppiamento** (*decoupling*) tra le funzioni di rete e l'hardware su cui girano. Le funzioni non sono più implementate in appliance fisiche dedicate, ma come **software** eseguito su infrastruttura virtualizzata standard (macchine virtuali o container su server COTS — *Commercial Off-The-Shelf*).



**Figura 8.1.** A sinistra, il modello tradizionale: ogni funzione (DPI, Firewall, BRAS, CG-NAT...) richiede HW specifico e un solo ruolo per nodo. A destra, il modello NFV: le stesse funzioni girano come software su server standard ad alto volume, orchestrate in modo automatico e remoto.

Nel modello tradizionale le funzioni di rete sono basate su HW e SW proprietari, con un nodo fisico per ogni ruolo. Nel modello NFV le funzioni sono implementate in software su hardware commodity, e più ruoli possono coesistere sullo stesso server fisico.

## 8.3 Vantaggi di NFV

NFV introduce tre famiglie di benefici fondamentali:

**Flessibilità:** le funzioni possono essere deployate ovunque nell'infrastruttura e spostate dinamicamente in base alle esigenze, grazie all'allocazione flessibile delle risorse hardware condivise (*hardware pool*).

**Scalabilità ed efficienza:** la capacità dedicata a ciascuna funzione può essere modificata dinamicamente (*scaling up/down*) in base al carico effettivo, migliorando l'utilizzo delle risorse. Non è più necessario acquistare hardware ridondante per gestire i picchi: si alloca più capacità software quando serve e la si riduce quando non serve.

**Innovazione accelerata:** il ciclo di sviluppo di un nuovo servizio di rete si accorcia drasticamente, perché non richiede procurement hardware ma soltanto deployment software.

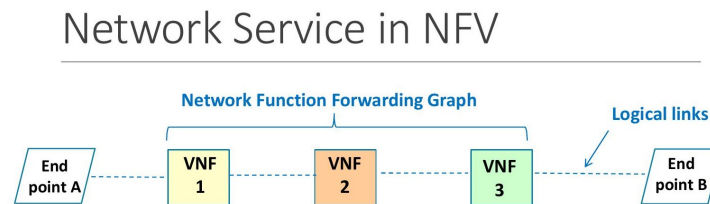
### Nota – NFV introduce anche nuove sfide

La virtualizzazione porta con sé problemi nuovi: gestione del ciclo di vita delle funzioni, orchestrazione, performance (overhead della virtualizzazione), sicurezza dell'infrastruttura condivisa, e il complesso problema del *placement* delle funzioni virtualizzate.

## 8.4 Da funzioni di rete a servizi di rete

### 8.4.1 Composizione di funzioni

Una singola funzione (es. un firewall) non è sufficiente a realizzare un servizio reale. I servizi richiedono la **composizione di più funzioni** applicate in sequenza al traffico. In un servizio tipico, un utente che accede a Internet deve attraversare: firewall → antivirus → video optimizer → parental control. Sequenze diverse possono applicarsi a traffici diversi.



**Figura 8.2.** Un servizio di rete minimo: il traffico dell'utente attraversa in sequenza Firewall, NAT e Load Balancer prima di raggiungere il Web Service.

In un'infrastruttura reale, non tutto il traffico segue la stessa catena: il servizio non è una pipeline fissa ma una composizione flessibile di funzioni, dipendente dai requisiti del flusso. Questo pone la domanda: come modificare le rotte per applicare trattamenti differenziati a flussi diversi? La risposta è [[Software Defined Networking]] (SDN), che fornisce il piano di controllo per il service chaining.

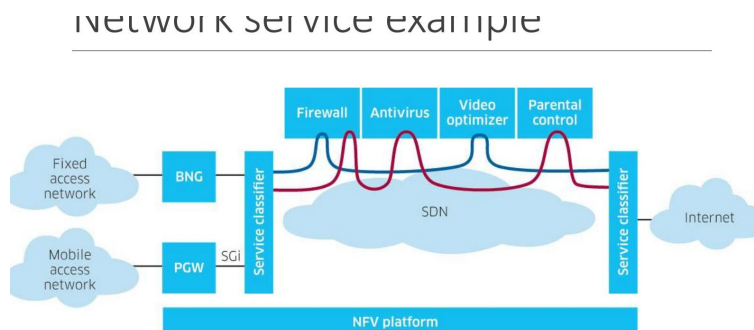
### 8.4.2 Network Function Forwarding Graph (NF-FG)

In NFV, un servizio di rete end-to-end è formalizzato come un **grafo di forwarding** di funzioni di rete e punti terminali. Questo è ciò che un operatore fornisce ai propri clienti.

### Definizione – VNF — Virtualized Network Function

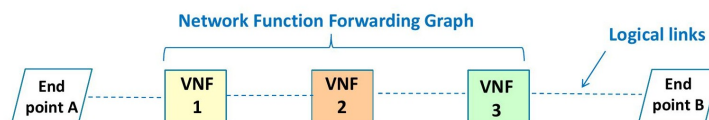
Una **Virtualized Network Function (VNF)** è l'implementazione software di una funzione di rete tradizionalmente realizzata in hardware dedicato (firewall, load balancer, NAT, ecc.), eseguita su risorse virtualizzate (VM o container).





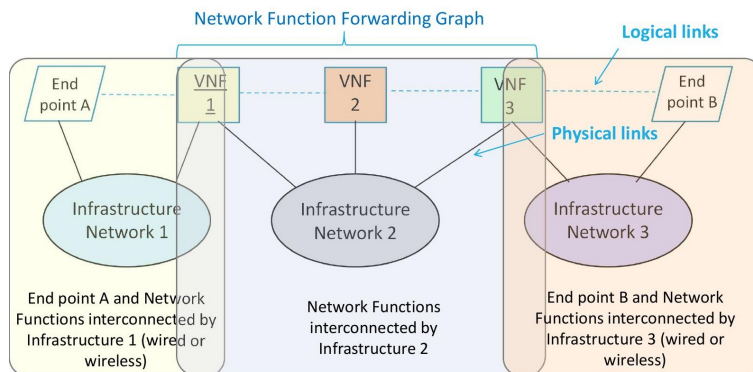
**Figura 8.3.** Le reti fissa e mobile convergono su una piattaforma NFV. Un service classifier instrada il traffico attraverso funzioni diverse (Firewall, Antivirus, Video optimizer, Parental control) via SDN, prima di raggiungere Internet.

## Network Service in NFV



**Figura 8.4.** Un servizio end-to-end è un grafo: l'endpoint A è connesso all'endpoint B attraverso VNF 1, VNF 2 e VNF 3 via link logici.

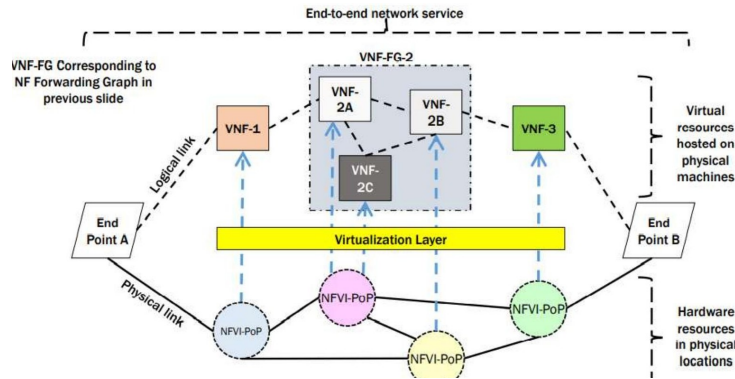
La separazione fondamentale introdotta da NF-FG è tra: - la **definizione logica del servizio**: il grafo descrive quali funzioni servono e come sono connesse (link logici) - il **deployment fisico**: i link logici sono supportati da percorsi fisici attraverso reti di infrastruttura distinte



**Figura 8.5.** Il grafo logico del servizio (VNF 1, VNF 2, VNF 3 con link tratteggiati) è mappato su tre reti di infrastruttura fisiche distinte. La separazione logico/fisico è il principio chiave.

### 8.4.3 VNF-FG nidificati e PoP

Le VNF girano su nodi fisici chiamati **PoP** (*Points of Presence*). I grafi di forwarding possono essere nidificati: un VNF-FG può essere costruito componendo altri VNF-FG (esempio: VNF-FG-2 composto da VNF-2A, VNF-2B e VNF-2C), consentendo la definizione di servizi gerarchici e riutilizzabili.



**Figura 8.6.** Il servizio end-to-end (tra End Point A e B) è un VNF-FG composto da VNF-1, dal sotto-grafo VNF-FG-2 (con VNF-2A, VNF-2B, VNF-2C) e VNF-3. Le VNF girano su PoP fisici (NFVI-PoP) tramite un livello di virtualizzazione.

## 8.5 Il problema del VNF Placement

Dato un servizio di rete composto da più VNF, occorre decidere **dove deployare ciascuna funzione** nell'infrastruttura fisica. Questo è il problema del *VNF chain placement*, uno dei problemi di ottimizzazione centrali in NFV.

Gli obiettivi di ottimizzazione tipici sono: - minimizzare la latenza end-to-end - minimizzare i costi di deployment - massimizzare la banda residua - risparmiare energia

Soggetti ai vincoli di: - capacità computazionale dei nodi - banda disponibile sui link - requisiti di QoS (latenza massima, banda minima)

### Tip – VNF placement e 5G Network Slicing

Il VNF placement è particolarmente rilevante nel contesto del [[5G]] e del **network slicing**: ogni slice deve avere una catena di VNF dedicata con garanzie di isolamento e QoS. L'orchestratore deve risolvere il problema di placement per ogni slice contemporaneamente, spesso con obiettivi conflittuali.

Gli approcci risolutivi spaziano dalla **programmazione matematica** (ILP, MILP) alle **euristiche** e **metaeuristiche** (Algoritmi Genetici), fino al **Deep Reinforcement Learning** per scenari dinamici.

### 8.5.1 Nuove sfide: hardware programmabile

Le VNF possono essere deployate non solo su server, ma anche su **dispositivi di rete programmabili**, ciascuno con un profilo prestazionale diverso:

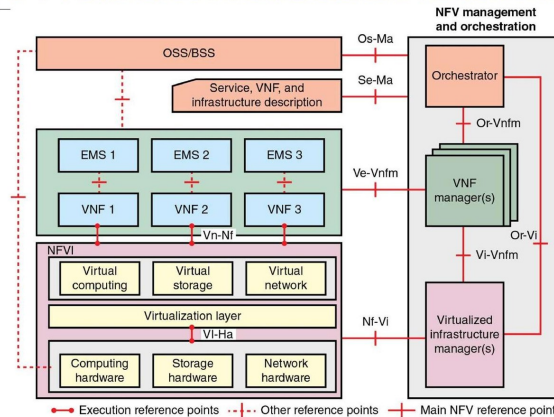
| Dispositivo         | Latenza per pacchetto | Caratteristiche                                 |
|---------------------|-----------------------|-------------------------------------------------|
| Programmable switch | 1–2 $\mu$ s           | Velocissimo, ma memoria e flessibilità limitate |
| Server (COTS)       | 1–10 $\mu$ s          | Flessibile, ma più lento                        |
| SmartNIC            | 10–100+ $\mu$ s       | Soluzione intermedia                            |

La scelta del dispositivo su cui deployare ciascuna VNF aggiunge una dimensione ulteriore al problema di placement.

## 8.6 L'architettura NFV

L'architettura di riferimento NFV, standardizzata da ETSI, è organizzata in tre blocchi principali: l'**infrastruttura** (NFVI), le **funzioni virtualizzate** (VNF) e il **sistema di gestione e orchestrazione** (NFV-MANO).

**NFV Reference Architectural Framework**



**Figura 8.7.** Il framework architetturale di riferimento NFV secondo ETSI/Stallings. Le frecce indicano i reference point: Or-Vi, Or-Vnfm, Ve-Vnfm, Vi-Vnfm, Nf-Vi, Se-Ma, Os-Ma.

### 8.6.1 NFV Infrastructure (NFVI)

L'**NFVI** è la totalità delle risorse hardware e software che costituiscono l'ambiente di esecuzione delle VNF. Virtualizza risorse fisiche di computing, storage e networking raggruppandole in *resource pool*.

È composta da tre domini:

- **Compute domain:** fornisce server COTS ad alto volume e storage.
- **Hypervisor domain:** media le risorse del compute domain alle VM (o container) delle VNF, fornendo un'astrazione dell'hardware. Oggi le VNF sono spesso eseguite in container → **Cloud Native Functions (CNF)**.
- **Infrastructure network domain:** switch generici ad alto volume interconnessi in una rete configurabile per erogare servizi di rete.

L'**NFVI** può distribuirsi su più **NFVI-PoP** (*Network Function Virtualization Infrastructure Point of Presence*): posizioni fisiche dove sono deployate le risorse di infrastruttura NFV. Due tipi di rete interconnettono i PoP: - **NFVI-PoP network:** connette le risorse all'interno di un PoP. - **Transport network:** interconnette i vari NFVI-PoP tra loro e verso l'esterno.

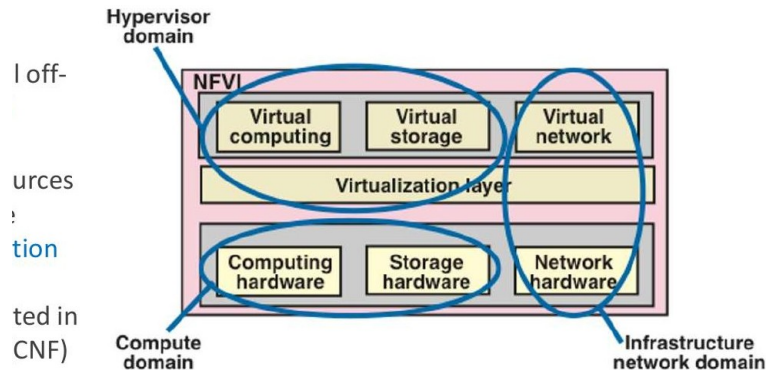
#### vSwitch

All'interno di un PoP, le VM sono collegate a interfacce virtuali (**VIF**). Un **virtual switch** (vSwitch) è un programma software che emula uno switch di livello 2, fornendo connettività tra le VIF e le interfacce fisiche (PIF), oltre a gestire il traffico tra VIF sullo stesso host fisico. L'esempio più noto è **Open vSwitch (OVS)**, che supporta OpenFlow e può integrarsi con switch fisici.

#### Opzioni di virtualizzazione

Le VNF possono essere eseguite in **macchine virtuali** (VM) o in **container**. La differenza fondamentale:

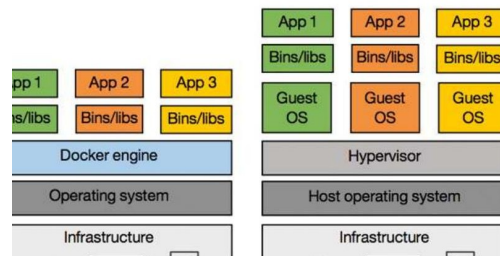
employed, managed and executed.  
 putting, storage, and networking and places them



itches interconnected into a

**Figura 8.8.** L'NFVI si stratifica in tre livelli: hardware fisico (compute, storage, network hardware), virtualizzazione (hypervisor/virtualization layer) e risorse virtuali (virtual computing, storage, network).

## ) Virtualization options



**Figura 8.9.** A sinistra i container: condividono il kernel dell'host, isolati in userspace tramite Docker engine. A destra le VM: includono un guest OS completo sopra l'hypervisor.

**Definizione – Container vs VM**

I **container** includono l'applicazione e le sue dipendenze ma condividono il kernel dell'host OS, girando come processi isolati in userspace. Le **VM** includono invece l'applicazione, le librerie necessarie e un intero guest OS, con un overhead maggiore ma un isolamento più forte.

## 8.6.2 NFV Management and Orchestration (NFV-MANO)

Il **MANO** è il framework che gestisce e orchestra tutte le risorse nell'ambiente NFV. Comprende:

### NFV Orchestrator (NFVO)

Responsabile di installare e configurare nuovi **Network Service (NS)** e pacchetti VNF, gestire il ciclo di vita dei servizi di rete e le risorse globali. Opera su due piani: - **Network services orchestration**: gestisce/coordina la creazione di servizi end-to-end tra VNF diverse, può istanziare nuovi VNFM. - Esempi di implementazioni open source: **OSM MANO** (ETSI) e **ONAP** (Linux Foundation).

### VNF Manager (VNFM)

Gestisce il **ciclo di vita delle istanze VNF**: - istanziazione (con configurazione iniziale se richiesta dal deployment template) - scaling out/in e up/down - raccolta di misure di performance dell'NFVI e correlazione con eventi/fault VNF - healing automatico o assistito - terminazione dell'istanza

### Virtualized Infrastructure Manager (VIM)

Il **VIM** controlla e gestisce l'interazione delle VNF con le risorse fisiche (computing, storage, network) e la loro virtualizzazione, tipicamente all'interno del dominio infrastrutturale di un singolo operatore. Piattaforme IaaS come **OpenStack** fungono da VIM. Un MANO può orchestrare più VIM.

## 8.6.3 Repository MANO

Il sistema MANO utilizza quattro repository:

| Repository                      | Contenuto                                                                   |
|---------------------------------|-----------------------------------------------------------------------------|
| <b>Network services catalog</b> | Template di deployment dei network service in termini di VNF e virtual link |
| <b>VNF catalog</b>              | VNF Descriptor (VNFD): describe deployment e behavior di ogni VNF           |
| <b>NFVI resources</b>           | Lista delle risorse NFVI utilizzate per i servizi NFV attivi                |
| <b>NFV instances</b>            | Dettagli sulle istanze di network service e VNF in esecuzione               |

## 8.7 Network Service Descriptor: un esempio

Il deployment di un network service è descritto tramite un **Network Service Descriptor (NSD)**, un documento strutturato (tipicamente JSON/YAML) che specifica la composizione del servizio.

**Tip – NSD per un servizio iperf (client-server)**

```
{
  "name": "iperf-NSD",
  "vendor": "fokus",
  "version": "0.1-ALPHA",
  "vnfd": [ ... ],
  "vld": [ { "name": "private" } ],
  "vnf_dependency": [
    {
      "source": { "name": "iperf-server" },
      "target": { "name": "iperf-client" },
      "parameters": ["private"]
    }
  ]
}
```

| Campo          | Significato                                                                    |
|----------------|--------------------------------------------------------------------------------|
| name           | Nome del Network Service                                                       |
| vendor         | Provider del servizio                                                          |
| vnfd           | Lista delle VNF che compongono il servizio                                     |
| vld            | Virtual Link: connettività tra VNF                                             |
| vnf_dependency | Dipendenza tra VNF: l'iperf-server deve fornire il proprio IP all'iperf-client |

Il VNFD per ogni VNF specifica: immagine VM, risorse (flavour), virtual link, lifecycle events (script da eseguire all'istanziamento: `install.sh`, `start-srv.sh`) e configurazione.

## 8.8 Casi d'uso

NFV è applicabile sia alle funzioni del **piano dati** che a quelle del **piano di controllo** nelle reti mobili e fisse:

- **Traffic analysis:** DPI (*Deep Packet Inspection*), misurazione QoE
- **Application-level optimization:** cache server, load balancer, application accelerator
- **Sicurezza:** firewall, virus scanner, IDS, spam protection, gateway IPsec/SSL VPN
- **Nodi di rete mobile:** HLR/HSS, MME, SGSN, PDN-GW, eNodeB (sia il core EPC che la RAN possono essere in larga parte virtualizzati)
- **Infrastruttura di rete:** broadband gateway, CG-NAT, router
- **Edge & Home:** home router, set-top box
- **Controllo & Gestione:** AAA server, policy control, sistemi di charging

## 8.9 Sintesi

**Nota – NFV in sintesi**

Nelle reti tradizionali, ogni dispositivo è deployato su piattaforme proprietarie/chiose. Gli elementi di rete sono box chiusi, l'hardware non può essere condiviso, e ogni device richiede hardware aggiuntivo per aumentare la capacità — hardware che rimane inattivo quando il sistema è sotto carico.

Con NFV, gli elementi di rete sono **applicazioni indipendenti** deployate in modo flessibile su una

piattaforma unificata di server standard, storage e switch commodity. Software e hardware sono disaccoppiati, e la capacità per ogni applicazione viene aumentata o ridotta aggiungendo o riducendo risorse virtuali.

I benefici principali: **flessibilità, innovazione più rapida**, possibilità di **scalare i servizi rapidamente** su/giù secondo le necessità.

## Capitolo 9

# Lezione 21 — (Lab) Network Emulator con ComNetsEmu

### 9.1 Laboratorio: Network Emulator con ComNetsEmu

Questa lezione di laboratorio introduce **ComNetsEmu**, un emulatore di reti progettato per sperimentare ambienti [\[\[SDN\]\]](#) ([\[\[Software-Defined Networking\]\]](#)) e [\[\[NFV\]\]](#) ([\[Network Function Virtualization\]](#)) su un singolo computer. Prima di avviare gli esperimenti è necessario installare e configurare l'ambiente: questa guida accompagna passo per passo dall'installazione fino all'esecuzione del primo esempio funzionante.

---

#### 9.1.1 Cos'è ComNetsEmu

**ComNetsEmu** è un ambiente software-based progettato come *testbed* e emulatore di reti, sviluppato da TU Dresden e Università di Trento come progetto open source. Il suo obiettivo principale è rendere possibile l'emulazione di scenari significativi di SDN e NFV su un singolo laptop, abbassando la barriera di accesso alla sperimentazione su reti programmatiche.

##### Nota – Caratteristiche chiave

- **Semplicità:** setup leggero, rapido e automatizzato
- **Riproducibilità e condivisibilità:** usa template human-friendly (Vagrantfile)
- **Tecnologie state-of-the-art senza complessità aggiuntiva:** permette di testare concetti avanzati in modo accessibile
- **Estensibilità:** progettato per essere ampliato con nuovi esempi e applicazioni

ComNetsEmu è costruito come un'estensione di **Mininet**, il popolare emulatore di reti per Linux, al quale aggiunge il supporto per container Docker come unità di elaborazione virtuali. La combinazione di Mininet (per l'emulazione del piano dati) e Docker (come soluzione NFVI) in un unico framework integrato è il tratto distintivo di ComNetsEmu.

---

#### 9.1.2 Fondamenti: Mininet

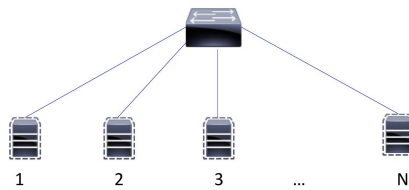
Prima di installare ComNetsEmu è utile capire come funziona Mininet, su cui si basa.

**Mininet** è un emulatore di rete che fa girare un insieme di host, switch, router e link su un singolo kernel Linux. Utilizza la virtualizzazione leggera per fare sembrare una macchina singola come una rete completa. Supporta [\[OpenFlow\]](#) per l'emulazione di reti SDN.



### Mininet – example topology

- SimpleSwitchTopology: one switch s1 and N nodes connected to s1



**Figura 9.1.** Il sito ufficiale di Mininet (<http://mininet.org/>) con la sua documentazione, walkthrough e API di riferimento.

### Come funziona internamente Mininet

Gli host di Mininet sono **processi** che condividono lo stesso kernel OS, gli stessi PID e file system. Ogni host ha però uno stack di rete **indipendente**, realizzato tramite i *network namespace* di Linux.

#### Definizione – Network Namespace

Un *network namespace* fornisce uno stack di rete isolato. I processi al suo interno hanno il proprio set di risorse di rete: interfacce virtuali, tabelle ARP, tabelle di routing. Un nodo virtuale di rete è semplicemente una shell in un network namespace.

### Topologie in Mininet

Una topologia Mininet è composta da entità come **Host**, **Switch**, **Controller** e **Link**. È possibile creare topologie custom tramite script Python estendendo la classe `mininet.topo.Topo` e sovrascrivendo il metodo `build()`.

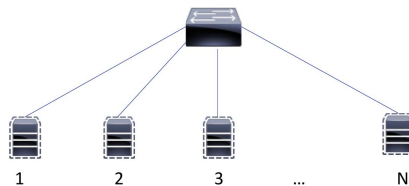
Il comando più semplice per avviare Mininet con una topologia di default è:

```
sudo mn
```

Questo inizializza una topologia predefinita e lancia la CLI interattiva.

### Mininet – example topology

- SimpleSwitchTopology: one switch s1 and N nodes connected to s1



**Figura 9.2.** SimpleSwitchTopology: uno switch centrale connesso a N host tramite link individuali.

### Esempio: SimpleSwitchTopology

**Tip – Principali classi e metodi Mininet**

- `Topo.addSwitch()` — aggiunge uno switch alla topologia
- `Topo.addHost()` — aggiunge un host alla topologia
- `Topo.addLink()` — aggiunge un link bidirezionale
- `Mininet.start()` / `Mininet.stop()` — avvia/ferma la rete
- `Mininet.pingAll()` — testa la connettività fra tutti i nodi
- Documentazione API: <http://mininet.org/api/annotated.html>

**CLI di Mininet**

La **CLI** di Mininet è uno strumento interattivo molto utile durante gli esperimenti. Si invoca passando l'oggetto rete al costruttore `CLI(net)`. I comandi principali sono:

```
mininet> net                # mostra la topologia
mininet> pingall             # testa la connettività tra tutti i nodi
mininet> h1 ping h2          # ping da h1 verso h2
mininet> h1 ip a             # mostra le interfacce di h1
mininet> iperf h1 h2         # misura il throughput tra h1 e h2
mininet> links               # mostra tutti i link attivi
mininet> h1 ip link show type veth # mostra le interfacce veth di h1
mininet> sh ovs-vsctl show    # mostra la configurazione del virtual switch
```

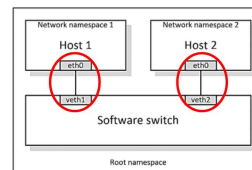
**Veth pairs e piano dati**

Ogni host Mininet può avere un'interfaccia Ethernet virtuale chiamata **veth** (*Virtual Ethernet Device*). I dispositivi veth vengono sempre creati in coppie interconnesse: i pacchetti trasmessi su un'interfaccia sono immediatamente ricevuti dall'altra. Si può pensare a un veth come a un cavo Ethernet reale che collega due dispositivi.

**Mininet characteristics**

- Each host may feature a Ethernet virtual interface (GNU/Linux Virtual Ethernet Device – **veth**)
  - veth devices are always created in interconnected pairs - packets transmitted on one device in the pair are immediately received on another device
  - A veth device is a **virtual ethernet device that you can think of as a real ethernet cable connecting two other devices**

veth can be connected to a virtual switch  
(e.g., OpenvSwitch)  
Virtual link parameters (bandwidth, delay) are  
configurable



**Figura 9.3.** Architettura delle veth pairs: Host 1 e Host 2 hanno ciascuno il proprio network namespace con interfacce veth collegate tramite un software switch nel Root Namespace.

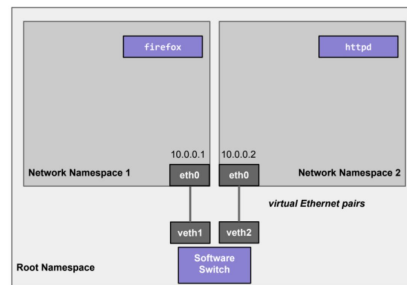
Le coppie veth possono essere connesse a un virtual switch (ad esempio **OpenvSwitch**). I parametri del link virtuale come bandwidth e delay sono configurabili.

**Linux Traffic Control (tc)** Mininet usa **tc** (*traffic control*) per configurare le caratteristiche dei link virtuali:

```
## Aggiunge 100ms di delay + jitter di ±10ms sull'interfaccia eth0
tc qdisc add dev eth0 root netem delay 100ms 10ms
```

```
## Da dentro Mininet, su h1:
```

## Mininet characteristics



This configuration could be setup manually through the `ip` utility

Mininet provides a simple command-line tool and APIs to do that for us automatically

It allow us to easily create topologies of varying size

SIGCOMM 2014 Tutorial: Teaching Computer Networking with Mininet

15

**Figura 9.4.** Il piano dati di Mininet: due namespace separati collegati tramite virtual Ethernet pairs e un Software Switch nel Root Namespace. Mininet gestisce automaticamente questa configurazione al posto dell'utilità `ip`.

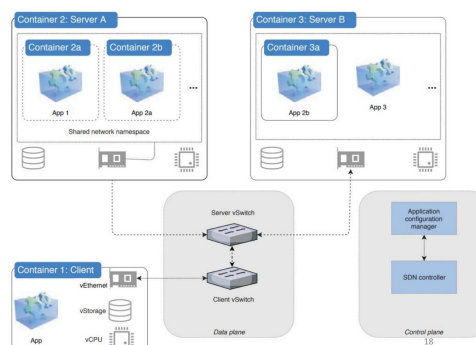
```
h1 tc qdisc add dev h1-eth0 root netem delay 100ms 10ms
h1 tc qdisc show dev h1-eth0
h1 tc qdisc del dev h1-eth0 root
```

### 9.1.3 Architettura di ComNetsEmu

ComNetsEmu estende Mininet sostituendo il tipo di host di default con **container Docker**. Questo approccio si chiama *Docker-in-Docker* (o *sibling containers*): si ha un **DockerHost** con container Docker interni che emulano host fisici che eseguono applicazioni containerizzate.

- It replaces the default host type in Mininet with **Docker containers**
- It uses Docker-in-Docker (or sibling containers)
  - Nested virtualization
  - A **DockerHost with deployed internal Docker containers**
    - replaces the default host type in Mininet
    - emulates a physical host running container applications
- **APPContainerManager** class orchestrates these internal Docker containers

## ComNetsEmu - main features



**Figura 9.5.** Architettura ComNetsEmu: i container rappresentano applicazioni su host virtuali; il piano dati è gestito da virtual switch; il control plane è affidato a un SDN controller.

La classe `APPContainerManager` orchestra i container Docker interni. Ogni `DockerHost` rimpiazza il tipo host di Mininet ed emula un host fisico che gira applicazioni containerizzate.

### Struttura del repository

| Cartella/File            | Contenuto                                 |
|--------------------------|-------------------------------------------|
| <code>comnetsemu/</code> | Modulo Python che estende Mininet         |
| <code>examples/</code>   | Esempi per iniziare con le API ComNetsEmu |

| Cartella/File    | Contenuto                             |
|------------------|---------------------------------------|
| app/             | Esempi di network slicing, SDN, ecc.  |
| util/            | Script per la gestione dell'ambiente  |
| test_containers/ | Dockerfile per i programmi di esempio |
| Vagrantfile      | File Vagrant per il setup della VM    |

### 9.1.4 Setup dell'ambiente

#### Attenzione – Prerequisiti da installare prima del laboratorio

L'installazione richiede circa **15 minuti** la prima volta. Completala prima della lezione di laboratorio.

#### Dipendenze

ComNetsEmu viene eseguito in una VM gestita da **Vagrant**. Le dipendenze da installare sul proprio sistema host sono:

- **Vagrant** >= v2.2.5 — <https://developer.hashicorp.com/vagrant/downloads>
- **VirtualBox** >= v6.0 — <https://www.virtualbox.org/wiki/Downloads>

Una volta avviata la VM, al suo interno saranno già presenti: - Mininet + OpenVswitch + Wireshark - Ryu SDN controller - Docker Engine

#### Installazione su Linux e Windows

Segui le istruzioni ufficiali: <https://stevetorenz.github.io/comnetsemu/installation.html#option-1-install-in-a-vagrant-managed-vm-highly-recommended>

```
cd ~
git clone https://git.comnets.net/public-repo/comnetsemu.git
cd ./comnetsemu
```

Prima di avviare la VM, è necessario modificare un Dockerfile.

#### Modifica obbligatoria: Dockerfile.dev\_test

#### Attenzione – Passo critico — da non saltare

Prima di eseguire **vagrant up**, devi modificare il file **Dockerfile.dev\_test** aggiungendo **netcat** alle dipendenze. Senza questa modifica il client TCP dell'esempio non funzionerà.

```
cd test_containers
## Apri il file Dockerfile.dev_test e aggiungi la riga "netcat \\" come mostrato nell'immagine
```

Il file modificato deve includere **netcat** \ nella lista dei pacchetti **apt-get install**. Dopo la modifica, salva il file e torna nella directory principale.

#### Avvio della VM

```
cd ..
vagrant up comnetsemu
## Attendi circa 15 minuti la prima volta
```

Quando la VM è pronta, il banner di ComNetsEmu viene stampato a schermo. A quel punto:

### Dockerfile.dev\_test

\$ cd comnetseму/test\_containers  
Open and modify the file **Dockerfile.dev\_test** as follows

```
# - telnet: Used by running iperf with Mininet's CLI
RUN apt-get -qy update && \
    DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
    bash \
    bash-completion \
    curl \
    iperf \
    iperf3 \
    iproute2 \
    iputils-ping \
    net-tools \
    stress-ng \
    tcpdump \
    netcat \
    telnet && \
    rm -rf /var/lib/apt/lists/*
CMD ["bash"]
```

**Figura 9.6.** Il file *Dockerfile.dev\_test*: aggiungere la riga *netcat* \\ prima di *telnet* \\ nella lista dei pacchetti da installare con *apt-get*.

## Accedi alla VM via SSH

```
vagrant ssh comnetseму
```

## Quando hai finito, ferma la VM sul terminale locale

```
vagrant halt
```

### Installazione su Mac

#### Nota – Mac — Percorso alternativo con Multipass

L'installazione con Vagrant potrebbe non funzionare su host Mac. In quel caso usa **Multipass**, un gestore di VM leggero sviluppato da Canonical. Segui le istruzioni al punto **B** di: <https://www.granelli-lab.org/researches/relevant-projects/comnetseму-labs>

### Installazione su Windows

Per host Windows, usa **MobaXterm** come console. Se l'apertura di xterm nell'emulatore non funziona:

## Salva la configurazione SSH in un file

```
vagrant ssh-config > vagrant-ssh
```

## Connettiti con forwarding X11 abilitato

```
ssh -F vagrant-ssh -Y comnetseму
```

## Testa il forwarding X11

```
xeyes
```

## oppure

```
xterm
```

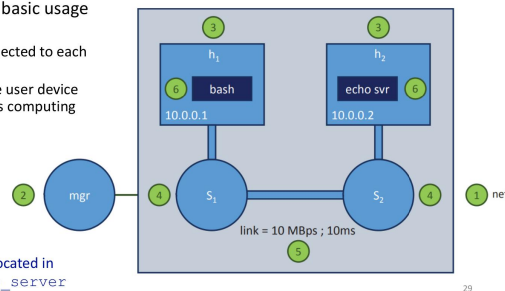
### 9.1.5 Primo esempio: Echo Server

L'esempio *echo\_server* mostra l'uso base dell'emulatore: due host emulati connessi tra loro tramite due switch, dove uno fa da client e l'altro ospita un server TCP che rimanda indietro ciò che riceve.

## Echo server

- example showcasing the basic usage of the emulator
  - two emulated hosts connected to each other
  - these hosts represent the user device and the service provider's computing infrastructure

the files of this example are located in the folder `examples/echo_server`



**Figura 9.7.** Topologia dell'echo server: `h1` (10.0.0.1) con container `bash` e `h2` (10.0.0.2) con container `echo_server`, collegati da due switch con link da 10 Mbps e 10ms di delay.

## Struttura del progetto

```
cd ~/comnetseму/examples
mkdir echoMCPS
cd echoMCPS
```

Il progetto richiede quattro file:

| File                               | Scopo                                  |
|------------------------------------|----------------------------------------|
| <code>server.py</code>             | Applicazione server TCP (echo)         |
| <code>Dockerfile</code>            | Containerizza l'applicazione server    |
| <code>build_docker_image.sh</code> | Script per costruire l'immagine Docker |
| <code>topology.py</code>           | Definisce la topologia ComNetsEmu      |

### File 1: `server.py`

Il server apre un socket TCP sulla porta 65000, accetta connessioni e rimanda indietro ogni segmento ricevuto (echo):

```
import socket

## Crea un socket TCP/IP
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

## Bind sulla porta 65000 (su tutte le interfacce)
server_address = ("", 65000)
print("starting up on {} port {}".format(*server_address))
sock.bind(server_address)

## In ascolto per connessioni in ingresso
sock.listen(1)

while True:
    print("waiting for a connection")
    connection, client_address = sock.accept()
    try:
        print("connection from", client_address)
```

```

while True:
    data = connection.recv(16)
    print("received {!r}".format(data))
    if data:
        print("sending data back to the client")
        connection.sendall(data)
    else:
        print("no data from", client_address)
        break
finally:
    connection.close()

```

## File 2: Dockerfile

```

FROM python:3.6-alpine3.9
COPY ./server.py /home/server.py
CMD python /home/server.py

```

## File 3: build\_docker\_image.sh

```
docker build -t echo_server --file ./Dockerfile .
```

Per costruire l'immagine:

```
bash build_docker_image.sh
```

## File 4: topology.py

```

from comnetsemu.net import Containernet, VNFMManager
from comnetsemu.cli import spawnXtermDocker
from mininet.node import Controller
from mininet.link import TCLink
from mininet.log import info
from mininet.cli import CLI

net = Containernet(controller=Controller, link=TCLink, xterms=False)
mgr = VNFMManager(net)

try:
    info("*** Add controller\n")
    net.addController("c0")

    info("*** Creating hosts\n")
    h1 = net.addDockerHost("h1", dimage="dev_test",
        ip="10.0.0.1", docker_args={"hostname": "h1"})
    h2 = net.addDockerHost("h2", dimage="dev_test",
        ip="10.0.0.2", docker_args={"hostname": "h2"})

    info("*** Adding switch and links\n")
    switch1 = net.addSwitch("s1")
    switch2 = net.addSwitch("s2")
    net.addLink(switch1, h1, bw=10, delay="10ms")
    net.addLink(switch1, switch2, bw=10, delay="10ms")

```

```

net.addLink(switch2, h2, bw=10, delay="10ms")

info("\n*** Starting network\n")
net.start()

# Avvia i container applicativi
srv1 = mgr.addContainer("srv1", "h1", "dev_test", "bash", docker_args={})
srv2 = mgr.addContainer("srv2", "h2", "echo_server",
                        "python /home/server.py", docker_args={})

# spawnXtermDocker("srv1")
CLI(net)

finally:
    info("*** Cleanup\n")
    mgr.removeContainer("srv1")
    mgr.removeContainer("srv2")
    net.stop()
    mgr.stop()

```

#### Nota – dev\_test vs echo\_server

h1 usa l'immagine `dev_test` (che ha `bash` e `netcat`) come client. h2 usa `echo_server` (che ha solo Python e il server). Per questo non è possibile aprire un xterm su `srv2`: l'immagine non include `bash`.

### Avvio dell'emulazione

```
sudo python3 topology.py
```

#### 9.1.6 Test dell'echo server

Una volta avviata la topologia, si apre un xterm sul container `srv1` (il client). Da lì è possibile eseguire i test.

#### Verifica connettività

```

h1:/# ip a                # mostra le interfacce del container
h1:/# ping 10.0.0.2       # verifica che h2 sia raggiungibile

```

#### Test con netcat

**netcat** (`nc`) è uno strumento da riga di comando che legge e scrive dati attraverso la rete. Lo usiamo come client TCP:

```

h1:/# nc 10.0.0.2 65000
Hello server
Hello server          # il server rimanda indietro il messaggio

```

#### Tip – Come funziona

`nc` si connette all'indirizzo IP 10.0.0.2 sulla porta TCP 65000, invia il testo digitato e stampa qualsiasi risposta ricevuta. Se tutto funziona correttamente, la parola "Hello server" comparirà due volte: una per l'input locale e una per l'echo del server.



## Cattura del traffico

Per ispezionare il traffico TCP generato, si può usare `tcpdump` direttamente nel container:

```
## Trova il container_id (h1 o h2)
docker ps -a

## Accedi al container
docker exec -it <container_id> bash

## Cattura il traffico TCP
tcpdump tcp

#Comando d'oro per cancellare pod e pulire la memoria e rifar partire topology
sudo mn -c && sudo docker rm -f $(sudo docker ps -aq) && sudo python3 topology.py
```

### 9.1.7 Uscire e spegnere la VM

Quando hai finito con gli esperimenti:

```
## 1. Esci dalla CLI di Mininet
mininet> exit

## 2. Esci dalla VM (shell SSH)
$ exit

## 3. Ferma la VM sul terminale locale
$ vagrant halt
```

### 9.1.8 Comandi Docker utili

Durante i laboratori è utile avere familiarità con i comandi base di Docker.

| Comando                                            | Descrizione                                        |
|----------------------------------------------------|----------------------------------------------------|
| <code>ip link list</code>                          | Mostra le interfacce di rete disponibili sull'host |
| <code>sudo lsns -t net</code>                      | Elenca i network namespace attivi                  |
| <code>sudo ip netns add &lt;name&gt;</code>        | Crea un nuovo namespace                            |
| <code>ip netns list</code>                         | Elenca i namespace                                 |
| <code>docker pull &lt;image&gt;</code>             | Scarica un'immagine dal registry                   |
| <code>docker run -it -d &lt;image&gt;</code>       | Crea ed avvia un container                         |
| <code>docker ps -a</code>                          | Elenca tutti i container (anche fermi)             |
| <code>docker exec -it &lt;id&gt; bash</code>       | Accede a un container in esecuzione                |
| <code>docker stop &lt;id&gt;</code>                | Ferma un container in esecuzione                   |
| <code>docker kill &lt;id&gt;</code>                | Ferma immediatamente un container                  |
| <code>docker images</code>                         | Elenca tutte le immagini locali                    |
| <code>docker rm &lt;id&gt;</code>                  | Elimina un container fermo                         |
| <code>docker rmi &lt;image-id&gt;</code>           | Elimina un'immagine locale                         |
| <code>docker build &lt;path&gt;</code>             | Costruisce un'immagine da un Dockerfile            |
| <code>docker commit &lt;id&gt; &lt;name&gt;</code> | Crea una nuova immagine da un container modificato |

## Capitolo 10

# Multi-Access Edge Computing (MEC)

### 10.1 Dal Cloud all'Edge

Il paradigma del **cloud computing** si basa sull'idea che le risorse computazionali possano essere fisicamente delocalizzate e connesse remotamente all'utente finale. L'**edge computing** estende questo paradigma portando le risorse elaborative ai margini della rete, fisicamente più vicine all'utente. Con l'edge computing, il traffico utente viene terminato in prossimità dell'utente stesso, creando un ambiente caratterizzato da latenza ridotta e throughput più elevato.

#### 10.1.1 Perché la latenza conta: la formula di Mathis

Per capire perché la vicinanza fisica all'utente sia importante, occorre capire come la latenza influenzi le prestazioni TCP. La **formula di Mathis** (1997) stabilisce che, per un tasso di perdita inferiore all'1%, il throughput massimo raggiungibile da una connessione TCP è limitato da:

*Mathis equation states that, for a small loss rate (less than 1%), the maximum achievable throughput of a TCP connection is limited by:*

$$\text{Throughput} \leq \frac{MSS}{RTT \sqrt{P_{loss}}} \quad \text{Mathis formula for throughput}^3$$

**Figura 10.1.** La formula di Mathis: il throughput TCP è inversamente proporzionale all' $RTT$  e alla radice quadrata della probabilità di perdita.

dove  $MSS$  è la *Maximum Segment Size*,  $RTT$  è il *Round-Trip Time* e  $P_{loss}$  è la probabilità di perdita dei pacchetti. La formula mostra come il throughput TCP sia limitato da due fattori strettamente legati alla rete: non basta ridurre il ritardo, ma serve anche migliorare il tasso di errore del canale. Questo motiva l'adozione di meccanismi di **QoS** implementati come elaborazione del traffico all'edge, finalizzati a ridurre la perdita di pacchetti.

#### Tip – Intuizione chiave

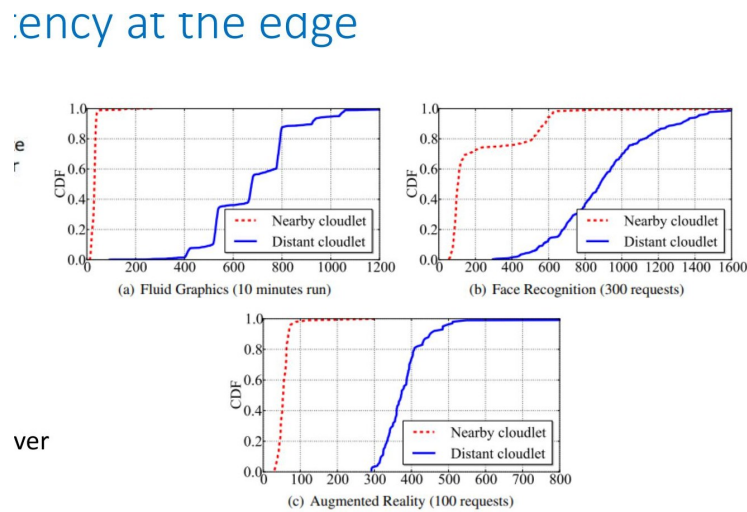
Portare il server vicino all'utente riduce l' $RTT$ ; terminare il traffico radio all'edge riduce anche  $P_{loss}$ . Entrambi i fattori si combinano per migliorare il throughput TCP in modo significativo.

#### 10.1.2 La latenza nella pratica: misurazioni reali

Un aspetto spesso controintuitivo è che la latenza end-to-end non è necessariamente proporzionale alla distanza geografica. Misurazioni effettuate in abitazioni distanti meno di 1 miglio quadrato mostrano differenze

enormi di latenza, specialmente quando sono coinvolti ISP diversi.

I grafici CDF (Cumulative Distribution Function) illustrano il confronto tra *nearby cloudlet* (bordo della rete, scenario MEC) e *distant cloudlet* (cloud tradizionale) per tre diverse applicazioni:



**Figura 10.2.** CDF del tempo di risposta per Fluid Graphics, Face Recognition e Augmented Reality: il cloudlet vicino (linea rossa tratteggiata) mostra latenze significativamente inferiori.

#### Nota – Evidenza empirica

Il risultato chiave è che la latenza end-to-end dipende dall'infrastruttura di rete (numero di hop, ISP attraversati) molto più che dalla distanza fisica. Questo rende l'edge computing essenziale per applicazioni sensibili alla latenza.

## 10.2 MEC: definizione e standard

**ETSI MEC** (*Multi-Access Edge Computing*) è il principale standard internazionale nell'area dell'edge computing. La definizione ufficiale ETSI recita:

#### Definizione – Multi-Access Edge Computing (MEC)

MEC offre a sviluppatori di applicazioni e provider di contenuti capacità di cloud computing e un ambiente di servizi IT al margine della rete.

Il termine “**multi-access**” indica che lo standard è agnostico rispetto alla tecnologia di accesso: è applicabile in linea di principio a reti 4G/5G, Wi-Fi o accesso fisso. Il “core business” dello standard ETSI MEC è la pubblicazione di **API standardizzate** che le applicazioni virtualizzate all'edge possono utilizzare per accedere a informazioni di rete.

### 10.2.1 Standard di riferimento

Esistono due principali corpi di standardizzazione nel dominio dell'edge computing:

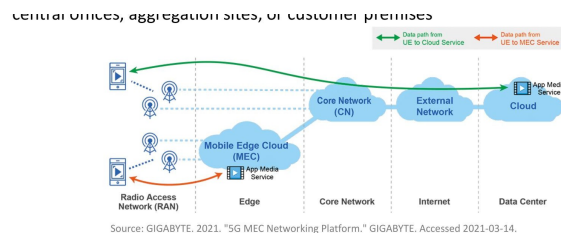
| Standard        | Focus                                             | Note                                                 |
|-----------------|---------------------------------------------------|------------------------------------------------------|
| <b>ETSI MEC</b> | Edge computing da prospettiva IT, access-agnostic | Definisce le API e le specifiche per piattaforme MEC |

| Standard    | Focus                           | Note                                              |
|-------------|---------------------------------|---------------------------------------------------|
| <b>3GPP</b> | Tecnologie radio e core network | Integra edge computing dal Release 15 (5G) in poi |

## 10.3 Il concetto di “edge”: dove si trova?

Il concetto di *edge* non è definito in modo rigido dallo standard ETSI, proprio per poter includere molte opzioni di deployment basate su requisiti diversi (costi, traffico, presenza di utenti, tariffe). L’edge può essere:

- **Base Station** (stazione radio base)
- **WiFi Access Point**
- **Data center** o micro-data center a uffici centrali, siti di aggregazione
- **Locali del cliente** (*customer premises*)



**Figura 10.3.** Confronto tra il percorso dati UE→Cloud (verde, attraverso core network, internet e data center) e UE→MEC Service (arancione, terminato al Mobile Edge Cloud). Fonte: GIGABYTE, 2021.

## 10.4 Benefici di MEC

I due abilitatori chiave forniti da MEC sono:

1. **Prossimità agli utenti finali:** riduzione della latenza end-to-end a livello del piano utente (UP – *User Plane*).
2. **Accesso alle informazioni radio locali** (piano di controllo, CP – *Control Plane*): permette di implementare meccanismi per migliorare il tasso di errore del canale.

A questi si aggiungono benefici supplementari:

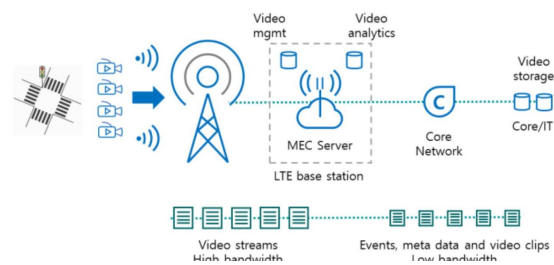
- possibilità di eseguire elaborazioni locali sfruttando grandi quantità di dati locali (sensori, dispositivi);
- supporto al **task offloading** da dispositivi finali con capacità di elaborazione limitata;
- facilitazione delle garanzie di **sicurezza e privacy**, ad esempio anonimizzando informazioni personali all’edge e trasmettendo solo dati aggregati al cloud centrale.

## 10.5 Scenari di servizio MEC

### 10.5.1 Analisi di flussi video

Un caso d’uso emblematico è il sistema di monitoraggio video con algoritmi di riconoscimento immagini, ad esempio per il riconoscimento di targhe o il controllo degli accessi in aree urbane.

## MEC SERVICE SCENARIOS: VIDEO STREAM ANALYSIS

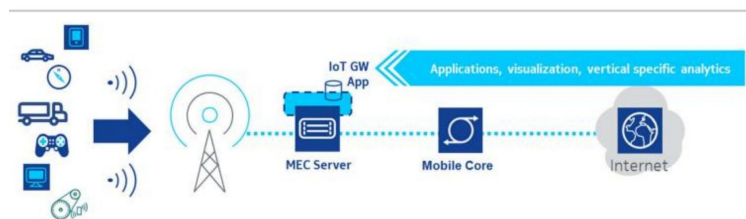


**Figura 10.4.** Il MEC Server, co-locato con la base LTE, esegue video management e analytics localmente. Verso la core network vengono trasmessi solo eventi, metadati e clip rilevanti (bassa banda), anziché stream video ad alta banda.

Eseguire il video analytics sul **MEC host** consente di: - evitare la trasmissione di stream video ad alta banda attraverso la core network verso servizi cloud remoti; - inviare alla rete solo i dati estratti dall'elaborazione (risultati, metadati, clip rilevanti); - ottenere prestazioni migliori rispetto sia a processare il video sulle telecamere stesse sia a processarlo su un server remoto.

### 10.5.2 IoT Gateway

MEC può fungere da **gateway IoT** intelligente, aggregando il traffico proveniente da dispositivi eterogenei (veicoli, sensori, attuatori industriali, dispositivi consumer) e applicando analytics verticali prima di trasmettere dati verso la core network e Internet.



**Figura 10.5.** Un'applicazione IoT GW App gira sul MEC Server, raccogliendo traffico da dispositivi IoT eterogenei via radio. Verso la core network vengono propagate solo analisi aggregate e eventi rilevanti.

## 10.6 Architettura MEC

### 10.6.1 Il sistema MEC

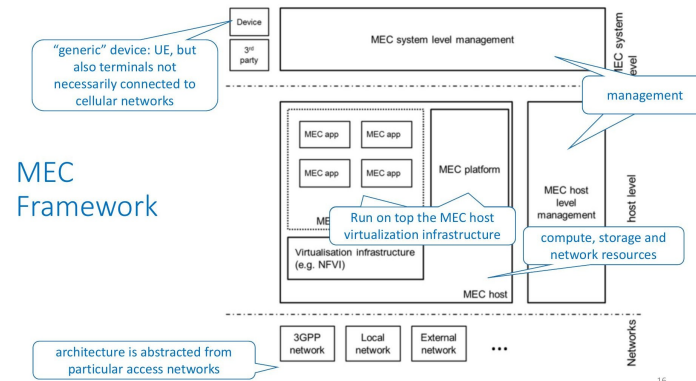
Un **sistema MEC** include i MEC host e la gestione MEC necessaria per eseguire applicazioni MEC all'interno di una rete operatore.

### 10.6.2 Il MEC Framework

I componenti principali dell'architettura sono:

#### Definizione – MEC Host

Include una **MEC platform** e una **virtualization infrastructure** che fornisce risorse di calcolo, storage e rete per le MEC application. È il server fisico che ospita la piattaforma e le applicazioni.



**Figura 10.6.** Il MEC Framework: i MEC host ospitano la virtualization infrastructure (NFV) con le MEC app e la MEC platform. La gestione è strutturata su due livelli: system-level e host-level.

#### Definizione – MEC Platform

Raccolta delle funzionalità essenziali per eseguire le MEC application su una particolare infrastruttura di virtualizzazione, abilitandole a consumare e offrire **MEC services**. La piattaforma può essa stessa offrire servizi.

#### Definizione – MEC Application

Applicazione che gira sopra la virtualization infrastructure fornita dal MEC host. Può interagire con la MEC platform per consumare e fornire MEC services.

#### Definizione – MEC Management

Divisa in *system-level* e *host-level*, gestisce l'orchestrazione dell'intero sistema MEC.

L'architettura è volutamente **astratta dalla tecnologia di accesso specifica**: le reti sottostanti possono essere 3GPP (4G/5G), Wi-Fi, rete fissa o qualsiasi combinazione. I dispositivi terminali non sono necessariamente UE cellulari, ma qualsiasi terminale connesso all'infrastruttura d'accesso.

## 10.7 Categorie di casi d'uso

I casi d'uso MEC si raggruppano in tre categorie principali:

| Categoria                              | Esempi                                                                                                           |
|----------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Consumer-oriented services             | Gaming, remote desktop, assistenza cognitiva, servizi in tempo reale per stadi/retail, offloading computazionale |
| Operator and third-party services      | Tracciamento posizione dispositivi, big data, video analytics, veicoli connessi, sicurezza                       |
| Network performance & QoE improvements | Content/DNS caching, ottimizzazione prestazioni, ottimizzazione e accelerazione video                            |

## 10.8 MEC Service APIs

MEC standardizza un insieme di **API di servizio** che consentono alle applicazioni virtualizzate all'edge di accedere a informazioni di rete e utente dal nodo locale. Le API principali sono:

| API                                     | Funzionalità                                                                |
|-----------------------------------------|-----------------------------------------------------------------------------|
| <b>Radio Network Info API (RNIS)</b>    | Condizioni radio attuali, Cell ID, intensità segnale, bearer UE             |
| <b>Location API</b>                     | Posizione geografica/logica degli UE, lista UE in un'area, movimenti in/out |
| <b>UE Identity API</b>                  | Identificazione dei terminali                                               |
| <b>Bandwidth Management API</b>         | Gestione della banda                                                        |
| <b>App Lifecycle Management API</b>     | Ciclo di vita delle applicazioni                                            |
| <b>WLAN Info API</b>                    | Informazioni sulle reti Wi-Fi                                               |
| <b>V2X Info Service API</b>             | Comunicazioni vehicle-to-everything                                         |
| <b>Service Discovery API</b>            | Scoperta dei servizi disponibili                                            |
| <b>Mobility API</b>                     | Gestione della continuità di servizio durante la mobilità                   |
| <b>MultiAccess Traffic Steering API</b> | Steering, splitting e duplicazione del traffico su accessi multipli         |

### 10.8.1 Radio Network Information API (RNIS)

La RNIS fornisce informazioni aggiornate sulle condizioni radio: Cell ID, intensità del segnale, misurazioni relative al piano utente, contesto degli UE connessi ai nodi radio associati al MEC host. È utile sia per le MEC app che vogliono ottimizzare i propri servizi (ad esempio guidare il throughput video adattativamente) sia per la MEC platform per ottimizzare le procedure di mobilità.

### 10.8.2 Location API

La Location API espone informazioni di localizzazione con due tipi di rappresentazione: - **Geolocalizzazione**: coordinate geografiche GPS. - **Localizzazione logica**: Cell ID, identificativi di settore radio.

Le informazioni includono: posizione di UE specifici, lista degli UE in un'area geografica, notifiche di ingresso/uscita da un'area, posizione dei nodi radio associati all'host.

## 10.9 MultiAccess Traffic Steering (MTS) API

Nessuna singola tecnologia di accesso wireless è in grado di soddisfare simultaneamente tutti i requisiti di comunicazione umana e machine-type: massima velocità di trasmissione da un lato, massima affidabilità dall'altro. La **MTS API** nasce per sfruttare la presenza di connessioni multiple combinandole intelligentemente.

### 10.9.1 Il trade-off fondamentale

L'idea di base è semplice: se si dispone di  $N$  link wireless, ciascuno con data rate  $R$  Mbps e affidabilità  $p$ , combinando i link si può ottenere:

- **Split traffic** (traffico suddiviso): data rate aggregato  $NimesR$  Mbps, stessa affidabilità  $p$  — si guadagna in throughput ma non in affidabilità.
- **Duplicate traffic** (traffico duplicato): stesso data rate  $R$ , affidabilità  $p^N$  — si guadagna in affidabilità ma non in throughput.

**Attenzione – Trade-off esclusivo**

Il miglioramento del data rate e il miglioramento dell'affidabilità non si ottengono contemporaneamente. Le due strategie (split e duplicate) sono mutuamente esclusive in termini di beneficio primario.

La tabella seguente mostra un esempio numerico con  $N$  link da 100 Mbps e affidabilità  $10^{-3}$ :

Reliability

- IDEA: performance can be improved if multiple connections are combined to deliver traffic
- E.g. improvement in data rate **OR** reliability: NOT at the same time!

|             | N = 1     | N = 2     | N = 3     | N = 4      |                 |
|-------------|-----------|-----------|-----------|------------|-----------------|
| Data rate   | 100 Mbps  | 200 Mbps  | 300 Mbps  | 400 Mbps   | Split traffic   |
| Reliability | $10^{-3}$ | $10^{-6}$ | $10^{-9}$ | $10^{-12}$ | Duplicate traff |

**Figura 10.7.** Con  $N=4$  link: data rate aggregato 400 Mbps (split) oppure affidabilità  $10^{-12}$  (duplicate). Le due colonne “Split traffic” e “Duplicate traffic” evidenziano il trade-off.

### 10.9.2 Operazioni MTS supportate

| Operazione             | Logica                                                          | Caso d'uso tipico           |
|------------------------|-----------------------------------------------------------------|-----------------------------|
| <b>Low Cost</b>        | Usa la rete non misurata (Wi-Fi) quando disponibile             | Email, backup in background |
| <b>Low Latency</b>     | Usa la connessione con latenza minore                           | Controllo remoto, gaming    |
| <b>High Throughput</b> | Usa la rete con throughput maggiore, o più reti simultaneamente | Video streaming HD          |
| <b>Redundancy</b>      | Duplica i pacchetti su più connessioni                          | Controllo robotico (URLLC)  |

**Tip – Esempi pratici di MTS**

- **Email/Dropbox:** applicazione data-centric in background, non sensibile a latenza né throughput → preferisce Wi-Fi (Low Cost).
- **Controllo robot (URLLC):** richiede altissima affidabilità e bassa latenza → Redundancy, pacchetti duplicati su più connessioni simultaneamente.
- **Video HD real-time:** richiede alto throughput → High Throughput con bandwidth aggregation (traffic split su più link).

### 10.9.3 Esercizio: applicabilità MTS



## Capitolo 11

# Teoria dei Segnali: Introduzione e Serie di Fourier

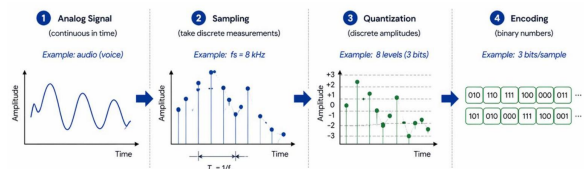
### 11.1 Perché la Teoria dei Segnali?

I sistemi moderni — reti mobili 4G/5G, dispositivi IoT, sistemi audio — elaborano **segnali** per rappresentare e trasmettere informazione. Capire come questi segnali sono strutturati, come vengono disturbati dal canale trasmissivo, e come possono essere analizzati e trasformati è il fondamento dell'ingegneria delle comunicazioni digitali.

La teoria dei segnali serve a quattro scopi pratici fondamentali. Il primo è **estrarre informazione**: identificare frequenze, pattern ed eventi rilevanti (es. riconoscimento vocale, analisi di segnali biomedici). Il secondo è **rimuovere effetti indesiderati**: filtrare il rumore, compensare distorsioni del canale, mitigare il fading multipath nelle reti wireless. Il terzo è **trasmettere efficientemente l'informazione**: codificare i segnali per il canale (es. OFDM in 4G/5G che distribuisce i dati su più frequenze). Il quarto è **abilitare l'elaborazione digitale**: convertire segnali analogici in forma digitale tramite campionamento e quantizzazione.

Why do we need signal theory?

- **Efficiently transmit information**
  - encode signals for transmission
  - adapt signals to channel constraints
  - **Examples:**
    - OFDM in 4G/5G splits data across frequencies
    - compression reduces bandwidth usage
- **Enable digital processing**
  - convert analog signals into digital form
  - process signals using algorithms
  - **Examples:**
    - sampling and FFT in receivers
    - edge processing in MEC systems



**Figura 11.1.** Dalla sorgente analogica ai bit: campionamento (discretizzazione nel tempo), quantizzazione (discretizzazione dell'ampiezza), codifica binaria.

## 11.2 Classificazione dei Segnali

### Definizione – Segnale

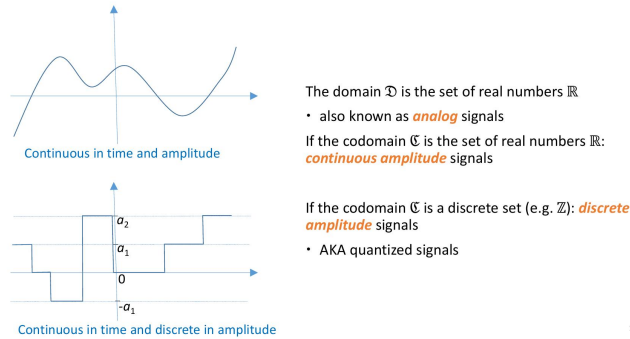
Un segnale è una variazione di una grandezza fisica che porta informazione. Formalmente è una funzione  $f(t) : \mathcal{D} \rightarrow \mathcal{C}$  dove il dominio  $\mathcal{D}$  può essere  $\mathbb{R}$  (tempo continuo) o  $\mathbb{Z}$  (tempo discreto), e il codominio  $\mathcal{C}$  può essere  $\mathbb{R}$  (ampiezza continua) o un insieme discreto (ampiezza quantizzata).

Ci concentriamo su **segnali deterministici monodimensionali**: la variabile indipendente è il tempo, e il segnale è noto prima di essere prodotto (al contrario dei segnali aleatori, analizzati con metodi probabilistici).

### 11.2.1 Segnali a Tempo Continuo

Quando il dominio è  $\mathbb{R}$ , il segnale è detto **analogico** (*analog*). Se anche il codominio è  $\mathbb{R}$ , l'ampiezza è continua; se il codominio è un insieme discreto (es.  $\mathbb{Z}$ ), l'ampiezza è quantizzata.

#### Continuous-time signals



**Figura 11.2.** In alto: segnale analogico, continuo in tempo e ampiezza. In basso: segnale continuo in tempo ma quantizzato in ampiezza (segnale a gradini).

### 11.2.2 Segnali a Tempo Discreto

Quando il dominio è  $\mathbb{Z}(T) = \{nT, \forall n \in \mathbb{Z}, T \in \mathbb{R}\}$ , il segnale è detto **discreto**. Ad esempio  $\mathbb{Z}(2) = \{\dots, -4, -2, 0, 2, 4, \dots\}$ . Un segnale discreto con ampiezza presa da un insieme finito di simboli è detto **digitale** (*digital*), o sequenza simbolica.

#### Discrete-time signals

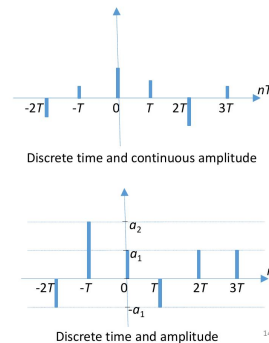
The domain  $\mathcal{D}$  is the set of integers  $\mathbb{Z}(T)$

- where  $\mathbb{Z}(T) = \{nT, \forall n \in \mathbb{Z} \text{ and } T \in \mathbb{R}\}$
- they are also called **discrete** signals
- for example,  $\mathbb{Z}(2) = \{\dots, -4, -2, 0, 2, 4, \dots\}$

The amplitude can still be continuous, or it can be discrete quantized

A discrete signal is called **digital** when it takes values from a finite set of symbols

- a digital signal is also called a symbolic sequence
- e.g. text  $\rightarrow$  sequence of characters (symbols)



**Figura 11.3.** In alto: segnale discreto con ampiezza continua. In basso: segnale digitale, discreto sia nel tempo che nell'ampiezza.

### 11.2.3 Tabella Riepilogativa

| Tempo                        | Ampiezza                  | Tipo                |
|------------------------------|---------------------------|---------------------|
| Continuo ( $\mathbb{R}$ )    | Continua ( $\mathbb{R}$ ) | Segnale analogico   |
| Continuo ( $\mathbb{R}$ )    | Discreta ( $\mathbb{Z}$ ) | Segnale quantizzato |
| Discreto ( $\mathbb{Z}(T)$ ) | Continua ( $\mathbb{R}$ ) | Segnale discreto    |
| Discreto ( $\mathbb{Z}(T)$ ) | Discreta (insieme finito) | Segnale digitale    |

### 11.2.4 Bitrate di una Sorgente Digitale

Quando un segnale analogico viene convertito in digitale con frequenza di campionamento  $f_c$  campioni/secondo e ogni campione è codificato con  $M$  bit:

$$\text{Bitrate} = f_c \cdot M \quad [\text{bit/secondo}]$$

Maggiori  $f_c$  e  $M$  producono migliore fedeltà ma richiedono maggiore capacità trasmissiva.

## 11.3 Trasmissione e Canale

### 11.3.1 Il Modello del Canale

Nella trasmissione di informazione, un trasduttore alla sorgente converte un messaggio in un segnale fisico (es. un'antenna converte energia elettrica in onde elettromagnetiche), il segnale si propaga attraverso il mezzo, e un trasduttore alla destinazione converte il segnale di ritorno in messaggio. Il **canale** non è un mezzo ideale: agisce come un sistema che modifica il segnale.

Le principali forme di degrado sono la **distorsione** (variazione della forma d'onda, es. attenuazione differenziata per frequenza), la **propagazione multipath** (il segnale raggiunge il ricevitore attraverso percorsi multipli con ritardi diversi, causando interferenza), e il **rumore** (segnale indesiderato sovrapposto a quello utile).

#### Definizione – SNR — Signal-to-Noise Ratio

Il rapporto segnale-rumore misura la qualità del segnale ricevuto rispetto al rumore di fondo:

$$SNR_{dB} = 10 \log_{10} \frac{P_{signal}}{P_{noise}}$$

### 11.3.2 Teorema di Shannon

Il risultato fondamentale della teoria dell'informazione stabilisce la capacità massima di un canale:

$$C = B \log_2(1 + SNR)$$

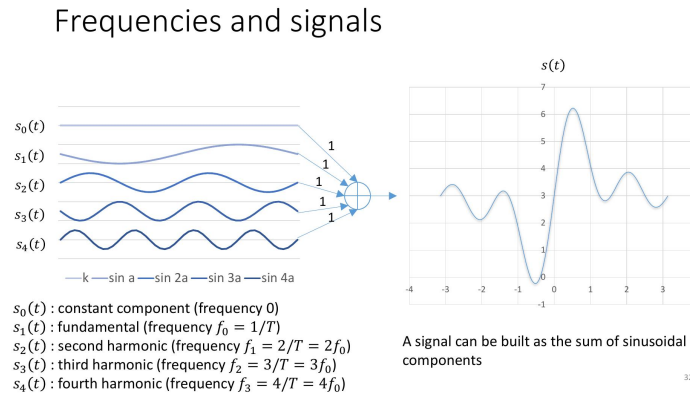
dove  $B$  è la banda disponibile in Hz,  $SNR$  il rapporto segnale-rumore, e  $C$  la capacità del canale in bit/secondo. Questo crea un trade-off inevitabile: ridurre la distorsione di quantizzazione alla sorgente aumenta la quantità di informazione da trasmettere, che deve rientrare entro la capacità  $C$ .

## 11.4 Serie di Fourier: dal Tempo alla Frequenza

### 11.4.1 L'Intuizione Fondamentale

Un segnale periodico può essere visto come una sovrapposizione di componenti sinusoidali, ciascuna con frequenza, ampiezza e fase specifiche. Questo cambio di prospettiva — dal dominio del tempo al **dominio della frequenza** — è cruciale perché il canale trasmissivo agisce in modo diverso su componenti a frequenza diversa. Per analizzare e predire tale comportamento, occorre identificare esplicitamente le componenti frequenziali di un segnale.

La serie di Fourier decompone una funzione come somma di infinite funzioni oscillanti a frequenze diverse. È formalmente un cambio di coordinate: dal dominio del tempo a quello della frequenza. La base di questa decomposizione è un insieme di funzioni  $\varphi_n(t)$  ortogonali, analogamente alla decomposizione di un vettore in uno spazio vettoriale.



**Figura 11.4.** Un segnale si costruisce sommando componenti sinusoidali: la componente costante ( $s_0$ ), la fondamentale ( $s_1$ ) e le armoniche superiori ( $s_2, s_3, s_4$ ). Cambiare i coefficienti cambia il segnale risultante.

### 11.4.2 Segnali Periodici Continui

Un segnale continuo  $s(t) : \mathbb{R} \rightarrow \mathbb{R}$  è **periodico con periodo  $T$**  se

$$s(t) = s(t + T) \quad \forall t \in \mathbb{R}$$

I segnali periodici si possono studiare interamente nell'intervallo  $[0, T]$ . La frequenza fondamentale è  $f = 1/T$ . Un segnale non periodico è detto **aperiodico**.

### 11.4.3 Definizione della Serie di Fourier

**Definizione – Serie di Fourier (intervallo  $[-\pi, \pi]$ )**

Data una funzione continua  $s(t) : \mathbb{R} \rightarrow \mathbb{R}$  periodica in  $[-\pi, \pi]$ :

$$s(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} (a_n \cos(nt) + b_n \sin(nt))$$

con coefficienti:

$$a_0 = \frac{1}{\pi} \int_{-\pi}^{\pi} s(t) dt \quad a_n = \frac{1}{\pi} \int_{-\pi}^{\pi} s(t) \cos(nt) dt \quad b_n = \frac{1}{\pi} \int_{-\pi}^{\pi} s(t) \sin(nt) dt$$

Il termine  $\frac{1}{2}a_0$  è la **componente continua** (media del segnale); i termini  $a_n \cos(nt) + b_n \sin(nt)$  sono le **armoniche** di frequenza  $n/T$ .

#### 11.4.4 Condizioni di Dirichlet

La serie di Fourier non è definita per qualsiasi funzione. Non sono note le condizioni necessarie, ma esistono condizioni **sufficienti** (teorema di Dirichlet):

##### Teorema – Teorema di Dirichlet

Se  $s(t)$  è periodica e **piecewise continuous** (composta da un numero finito di pezzi continui su ogni sottointervallo finito, con limite finito nei punti di discontinuità), allora la serie di Fourier di  $s(t)$  esiste e converge in  $\mathbb{R}$ .

#### 11.4.5 Esempio: Onda Quadra

Consideriamo il segnale periodico con periodo  $2\pi$ :

$$s(t) = \begin{cases} 2 & \text{se } -\pi < t < 0 \\ 1 & \text{se } 0 \leq t \leq \pi \end{cases}$$

Il calcolo dei coefficienti porta a:

$$a_0 = 3, \quad a_n = 0, \quad b_n = \begin{cases} 0 & \text{se } n \text{ pari} \\ -\frac{2}{n\pi} & \text{se } n \text{ dispari} \end{cases}$$

La serie risultante, ponendo  $n = 2k - 1$ , è:

$$s(t) = \frac{3}{2} - \sum_{k=1}^{\infty} \frac{2}{(2k-1)\pi} \sin((2k-1)t)$$

Solo le armoniche **dispari** contribuiscono, e la loro ampiezza cala come  $1/n$ .

#### 11.4.6 Fenomeno di Gibbs

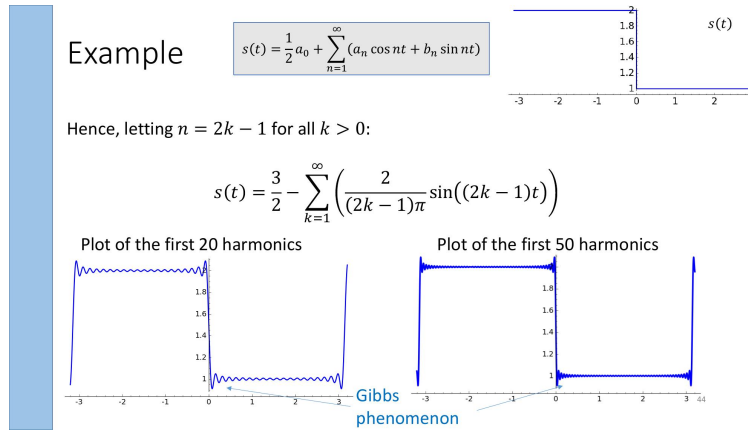
Quando la serie di Fourier approssima una funzione con discontinuità (come un'onda quadra), si osserva un **overshoot** intorno ai punti di salto che non scompare mai, per quanto si aumenti il numero di armoniche. Questo è il **fenomeno di Gibbs**: la serie converge in senso  $L^2$ , ma nelle vicinanze di ogni discontinuità l'errore rimane del  $\sim 9\%$  del salto, indipendentemente dall'ordine di troncamento.

### 11.5 Serie di Fourier con Periodo Arbitrario

La definizione si estende naturalmente a segnali con periodo generico  $T$ . Tramite la sostituzione  $y = 2\pi t/T$ , un segnale periodico in  $[-T/2, T/2]$  si trasforma in uno periodico in  $[-\pi, \pi]$ , e i coefficienti diventano:

$$s(t) = \frac{1}{2}a_0 + \sum_{n=1}^{\infty} \left( a_n \cos \frac{2\pi n t}{T} + b_n \sin \frac{2\pi n t}{T} \right)$$

con



**Figura 11.5.** Fenomeno di Gibbs: aggiungere più armoniche riduce le oscillazioni nel tratto piatto, ma l'overshoot al salto rimane costante (~9% del salto).

$$a_0 = \frac{2}{T} \int_{-T/2}^{T/2} s(t) dt \quad a_n = \frac{2}{T} \int_{-T/2}^{T/2} s(t) \cos \frac{2\pi nt}{T} dt \quad b_n = \frac{2}{T} \int_{-T/2}^{T/2} s(t) \sin \frac{2\pi nt}{T} dt$$

## 11.6 Numeri Complessi e Formula di Eulero

Prima di passare alla forma esponenziale della serie, è necessario richiamare i **numeri complessi**. Un numero complesso  $\bar{x} = a + jb$  ha parte reale  $a$  e parte immaginaria  $b$  (con  $j = \sqrt{-1}$ ). Nel piano complesso è rappresentato come un vettore di modulo  $|\bar{x}| = \sqrt{a^2 + b^2}$  e fase  $\varphi = \tan^{-1}(b/a)$ .

### Definizione – Formula di Eulero

$$e^{\pm j\varphi} = \cos \varphi \pm j \sin \varphi$$

Da cui si derivano le formule inverse:

$$\cos \varphi = \frac{e^{j\varphi} + e^{-j\varphi}}{2} \quad \sin \varphi = \frac{e^{j\varphi} - e^{-j\varphi}}{2j}$$

Usando l'esponenziale di Eulero, ogni numero complesso si scrive compattamente come  $\bar{x} = |x|e^{j\varphi}$ , che combina modulo e fase in un unico termine. Questo è particolarmente potente per rappresentare segnali: ampiezza e fase di ogni armonica vengono codificate in un unico coefficiente complesso.

## 11.7 Serie di Fourier in Forma Esponenziale

### 11.7.1 La Base Esponenziale

La serie di Fourier generalizzata al dominio complesso usa come base l'insieme di funzioni:

$$e^{j2\pi n F t} \quad \forall n \in \mathbb{Z}$$

poiché  $e^{j2\pi n F t} = \cos(2\pi n F t) + j \sin(2\pi n F t)$ , ogni elemento della base è una sinusoide complessa alla frequenza  $nF$ .

**Definizione – Serie di Fourier esponenziale**

Dato un segnale periodico  $s(t) = s(t + T)$  con frequenza fondamentale  $F = 1/T$ :

$$s(t) = \sum_{n=-\infty}^{+\infty} S_n e^{j2\pi n F t}$$

dove i coefficienti  $S_n$  (in generale complessi) si calcolano come:

$$S_n = \frac{1}{T} \int_0^T s(t) e^{-j2\pi n F t} dt$$

I coefficienti  $S_n$  codificano sia ampiezza che fase di ogni armonica:  $S_n = |S_n|e^{j\phi_n}$ , dove  $|S_n|$  è l'ampiezza e  $\phi_n = \arg(S_n)$  è la fase dell'armonica  $n$ -esima.

**11.7.2 Interpretazione Geometrica**

Per  $n = 0$ : la componente continua (media del segnale),  $S_0 = \frac{1}{T} \int_0^T s(t) dt$ .

Per  $n = \pm 1$ : frequenza fondamentale  $F$ , periodo  $T$ .

Per  $|n| > 1$ : armoniche di frequenza  $nF$ , periodo  $T/n$ .

**Tip – Perché la forma complessa?**

Con i numeri reali occorrono due coefficienti ( $a_n$  e  $b_n$ ) per descrivere ogni armonica. Con la forma complessa basta un solo coefficiente  $S_n$  che ne racchiude entrambi. Questo semplifica enormemente la manipolazione matematica e consente di rappresentare segnali  $s(t) : \mathbb{R} \rightarrow \mathbb{C}$ , generalizzando la serie per funzioni reali.

**11.7.3 Esempio: Onda Rettangolare con Base Esponenziale**

Consideriamo il segnale di periodo  $T$  e frequenza  $F = 1/T$ :

$$s(t) = \begin{cases} 1 & \text{se } |t| \leq T/4 \\ 0 & \text{se } T/4 < |t| \leq T/2 \end{cases}$$

Il calcolo dei coefficienti dà:

$$S_n = \begin{cases} 1/2 & n = 0 \\ 0 & n \text{ pari} \\ \frac{(-1)^k}{-(2k-1)\pi} & n = 2k-1 \end{cases}$$

La serie risultante è quindi:

$$s(t) = \frac{1}{2} + \sum_{k=-\infty}^{+\infty} -\frac{(-1)^k}{(2k-1)\pi} \cos(2\pi(2k-1)Ft)$$

Il grafico seguente mostra la convergenza della serie per  $T = 4$  con  $k \in [-1, 1]$  (verde),  $k \in [-2, 2]$  (rosso) e  $k \in [-100, 100]$  (blu):

### Example

Recall now that a complex  $\bar{x} = a + jb$  can also be written as  $\bar{x} = |x|e^{j\theta}$

- where  $|x| = \sqrt{a^2 + b^2}$  and  $\theta = \tan^{-1} \frac{b}{a}$ ,
- or also  $a = |x| \cos \theta$ ;  $b = |x| \sin \theta$

and, by the Euler's exponential, we have:  $|x|e^{j\theta} = |x| \cos \theta + j|x| \sin \theta$

Hence, if  $n$  odd,  $S_n = j \frac{2}{n\pi}$  (where  $a = 0, b = \frac{2}{n\pi}, \theta_n = \tan^{-1} \frac{b}{a}$ ) can be written as:

$$S_n = |S_n|e^{j\theta_n}$$

**Figura 11.6.** Al crescere del numero di armoniche considerate (verde  $\rightarrow$  rosso  $\rightarrow$  blu) la serie converge all'onda rettangolare. La forma in blu con  $k=100$  è quasi perfetta, con il solo fenomeno di Gibbs ai salti.

## 11.8 Lo Spettro di un Segnale

La sequenza ordinata dei coefficienti  $\{S_n\}_{n=-\infty}^{+\infty}$  è chiamata **spettro** del segnale. Conoscere lo spettro equivale a conoscere il segnale (dove  $s(t)$  è continua; nei punti di discontinuità vale la media dei limiti laterali, per il teorema di Dirichlet).

Poiché i coefficienti  $S_n$  sono in generale numeri complessi anche quando  $s(t)$  è reale, lo spettro non può essere rappresentato con un singolo grafico 2D. Si usano invece due diagrammi separati, sfruttando la rappresentazione  $S_n = |S_n|e^{j\theta_n}$ :

- **Spettro di ampiezza:** il grafico di  $|S_n|$  in funzione di  $n$  (o della frequenza  $nF$ )
- **Spettro di fase:** il grafico di  $\theta_n = \arg(S_n)$  in funzione di  $n$

### Tip – Spettro dell'onda quadra antisimmetrica

Per il segnale  $s(t) = -1$  se  $0 < t \leq T/2$ ,  $s(t) = 1$  se  $T/2 < t \leq T$ , i coefficienti sono:

$$S_n = \begin{cases} 0 & n \text{ pari} \\ \frac{2}{n\pi} e^{j\pi/2} & n > 0, \text{ dispari} \\ -\frac{2}{n\pi} e^{-j\pi/2} & n < 0, \text{ dispari} \end{cases}$$

Lo spettro di ampiezza decade come  $2/(|n|\pi)$  per gli indici dispari; lo spettro di fase è costante a  $+\pi/2$  per  $n > 0$  e  $-\pi/2$  per  $n < 0$ .

## 11.9 Segnali Aperiodici e Supporto Limitato

Anche per i segnali aperiodici con supporto limitato a  $[a, b]$  — cioè  $s(t) = 0$  per  $t \notin [a, b]$  — è possibile calcolare la serie di Fourier. Tuttavia, la serie assume implicitamente che il segnale sia periodico con periodo  $T = b - a$ . La conseguenza è che, invertendo la serie (da  $S_n$  a  $s(t)$ ), si ottiene l'**estensione periodica** del segnale originale, non il segnale aperiodico stesso.

$$s^*(t) = \sum_{n=-\infty}^{+\infty} s(t - nT)$$



Questo passaggio è cruciale: la serie di Fourier non può rappresentare segnali davvero aperiodici. Per questi occorre la **Trasformata di Fourier** (argomento delle lezioni successive), che è in un certo senso il limite della serie di Fourier per  $T \rightarrow \infty$ .

---

## Capitolo 12

# Lezione 26 — (Lab) Network Slicing

### 12.1 (Lab) Network Slicing con SDN e ComNetsEmu

Questa lezione hands-on mette in pratica il concetto di **network slicing** partendo dalla teoria del 5G per arrivare a un'implementazione funzionante basata su [[Lezione 15 - (Lab) SDN Architecture|SDN]] con il controller Ryu e l'emulatore ComNetsEmu. L'obiettivo centrale è dimostrare come sia possibile partizionare una singola infrastruttura fisica in sotto-reti logicamente isolate, ciascuna con garanzie proprie di banda e connettività.

---

#### 12.1.1 Contesto: il 5G e la necessità del Network Slicing

Il 5G non è semplicemente un incremento di velocità rispetto al 4G: è una riprogettazione fondamentale dell'architettura di rete, motivata dalla coesistenza di tre categorie di servizio con requisiti radicalmente diversi tra loro.

##### Definizione – mMTC — massive Machine Type Communications

Comunicazione tra un numero elevatissimo di dispositivi (fino a 200.000/km<sup>2</sup>), tipicamente sensori a bassa potenza e costo. I requisiti dominanti sono la **massima efficienza energetica** e la **scalabilità della connessione**, non la latenza o la banda.

##### Definizione – URLLC — Ultra-Reliable Low-Latency Communications

Servizi che richiedono simultaneamente **bassa latenza** ( 5 ms per sistemi di trasporto intelligenti, 1 ms per motion control industriale) e **alta affidabilità**. Esempi: guida autonoma, robot industriali, smart grid.

##### Definizione – eMBB — Enhanced Mobile Broadband

Servizi ad alta densità di dati: streaming 4K/8K, realtà aumentata e virtuale, ologrammi. Il requisito primario è la **capacità di trasmissione** e la copertura ampia.

La coesistenza di questi tre profili rende impossibile ottimizzare un'unica rete per tutti contemporaneamente. La soluzione è il **network slicing**: invece di una rete monolitica con compromessi su tutto, si creano più reti virtuali end-to-end sopra la stessa infrastruttura fisica, ognuna ottimizzata per il proprio caso d'uso.

### 12.1.2 Network Slicing: il concetto

Il network slicing sfrutta le risorse dell'infrastruttura fisica per creare multiple **sotto-reti virtuali** (slice), ciascuna delle quali si comporta come una rete indipendente dal punto di vista delle applicazioni che vi girano sopra.

Ogni slice è definita in termini di: - **funzionalità** — quali funzioni di rete sono presenti - **QoS** — banda garantita, latenza massima, tasso di perdita tollerato - **isolamento** — le slice non interferiscono tra loro né in termini di prestazioni né di sicurezza

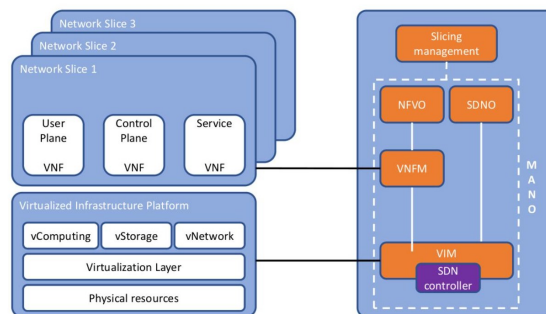
#### Ruolo di NFV e SDN

Le due tecnologie abilitanti del network slicing sono [[Lezione 20 - (Lab) Network Function Virtualization|NFV]] e [[Lezione 15 - (Lab) SDN Architecture|SDN]], con ruoli complementari.

**NFV** (Network Function Virtualization) si occupa di *implementare* le funzioni di rete di ogni slice come Virtual Network Functions (VNF) eseguite su hardware commodity. L'isolamento tra slice è garantito dalla virtualizzazione: ogni VNF vede solo le risorse assegnate, indipendentemente dalle altre slice sulla stessa macchina fisica.

**SDN** (Software-Defined Networking) si occupa di *governare* il comportamento del traffico all'interno di ogni slice. Una volta definita la slice, il controller SDN monitora e impone i requisiti di QoS installando le flow rule appropriate sugli switch.

#### Architettura generica per il Network Slicing



**Figura 12.1.** Architettura generica per il network slicing

*Fig. — Architettura generica per il network slicing: le slice (sinistra) poggiano sull'infrastruttura virtualizzata, gestita dal framework MANO (destra).*

Il piano di gestione (MANO) si articola su più livelli:

- **Slicing Management:** punto di ingresso per la definizione delle slice
- **NFVO** (NFV Orchestrator): orchestra le risorse NFV a livello di sistema
- **SDNO** (SDN Orchestrator): orchestra le risorse SDN
- **VNFM** (VNF Manager): gestisce il ciclo di vita di ogni singola VNF (creazione, scaling, terminazione)
- **VIM** (Virtualized Infrastructure Manager): controlla direttamente le risorse virtualizzate (compute, storage, network)
- **SDN Controller:** implementa la logica di forwarding sugli switch

#### Il caso d'uso 5G con RAN

In un deployment 5G reale, l'architettura a slice percorre tutta la rete dall'accesso radio al core cloud. La **Radio Access Network (RAN)** è scomposta in tre unità funzionali:

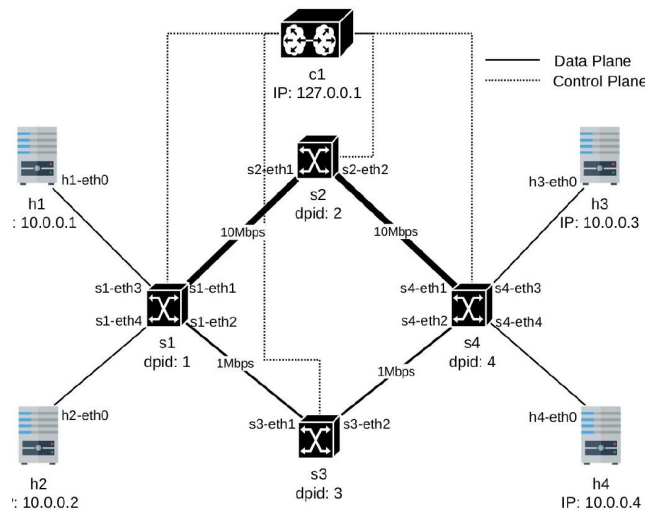
- **Radio Unit (RU)**: l'hardware radio fisico, tipicamente sull'antenna
- **Distributed Unit (DU)**: elabora i livelli MAC e Fisico, distribuita vicino alla RU
- **Centralized Unit (CU)**: gestisce i protocolli di livello superiore (RRC, PDCP) e può essere ospitata nel cloud

Slice diverse (UHD, Phone, Massive IoT, Mission-critical IoT) attraversano la stessa RAN ma mantengono path di dati separati sia nell'Edge Cloud (NFV) sia nel Core Cloud (NFV), con controller SDN dedicati a ogni livello.

### 12.1.3 Topologia dell'esperimento

L'esperimento implementa il network slicing su scala ridotta usando **ComNetsEmu** come emulatore e **Ryu** come controller SDN. La rete è organizzata come segue.

#### Descrizione della rete fisica



12

**Figura 12.2.** Topologia ad anello con quattro switch e quattro host

*Fig. — Topologia ad anello: il percorso superiore (S1→S2→S4) a 10 Mbps è destinato alla slice video; quello inferiore (S1→S3→S4) a 1 Mbps alla slice HTTP. Le linee tratteggiate rappresentano il control plane.*

La topologia è un **anello di quattro switch** (S1, S2, S3, S4) con host alle estremità:

- S1 ha quattro porte: eth1 verso S2, eth2 verso S3, eth3 verso h1, eth4 verso h2
- S4 ha la stessa struttura speculare: eth1 verso S2, eth2 verso S3, eth3 verso h3, eth4 verso h4
- S2 e S3 sono semplicemente di transito: due porte, una verso S1 e una verso S4
- I link superiori (via S2) hanno banda 10 Mbps; quelli inferiori (via S3) hanno banda 1 Mbps

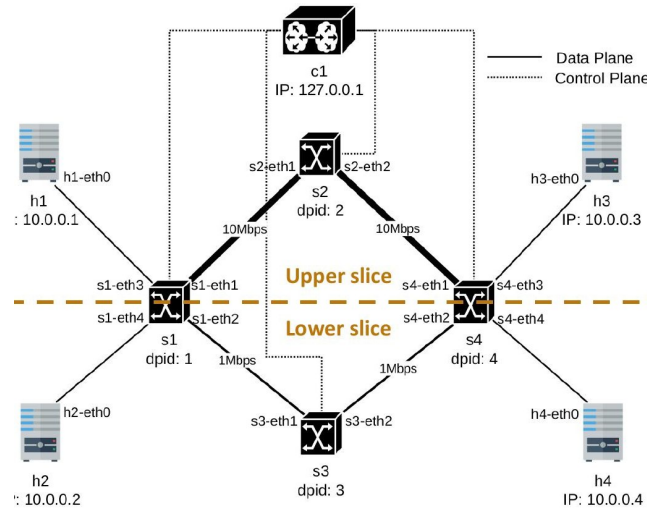
#### Obiettivo: il Topology Slicing

L'obiettivo non è realizzare un instradamento tradizionale (dove tutti gli host si raggiungono), ma imporre un **partizionamento rigido** della topologia:

- **Upper Slice**: h1 h3, interconnessi esclusivamente tramite i link a 10 Mbps (via S2)
- **Lower Slice**: h2 h4, interconnessi esclusivamente tramite i link a 1 Mbps (via S3)

**Attenzione – Isolamento completo**

h1 e h2 non possono comunicare tra loro, anche se sono entrambi collegati a S1. Il controller impone che il traffico entrante da eth3 (h1) esca solo da eth1 (verso S2), e il traffico da eth4 (h2) esca solo da eth2 (verso S3). La separazione è logica, non fisica.



13

**Figura 12.3.** Le due slice risultanti: la linea arancione separa l'Upper Slice (S1-eth1/eth3 S2 S4-eth1/eth3) dalla Lower Slice (S1-eth2/eth4 S3 S4-eth2/eth4).

Ogni host ha quindi una propria **vista della rete**: dal punto di vista di h1, la rete è semplicemente un collegamento diretto verso h3 a 10 Mbps; h2 e h4 sono invisibili.

### 12.1.4 Implementazione con Ryu e Mininet

Il laboratorio si basa su due script Python nella directory `comnetseму/app/realizing_network_slicing/`:

1. `network.py` — definisce la topologia e avvia Mininet
2. `topologyslicing.py` — applicazione Ryu che programma il comportamento degli switch

#### `network.py`: definizione della topologia

Lo script costruisce la topologia usando le API di Mininet. I punti chiave sono la configurazione dei link con limitazione di banda (TCLink) e l'assegnazione di DPID univoci agli switch.

```
host_config = dict(inNamespace=True)
http_link_config = dict(bw=1) # 1 Mbps
video_link_config = dict(bw=10) # 10 Mbps

## Switch con DPID esplicito
for i in range(4):
    sconfig = {"dpid": "%016x" % (i + 1)}
    self.addSwitch("s%d" % (i + 1), **sconfig)

## Link tra switch: ordine importante per i numeri di porta!
self.addLink("s1", "s2", **video_link_config) # s1-eth1 s2-eth1
self.addLink("s2", "s4", **video_link_config) # s2-eth2 s4-eth1
self.addLink("s1", "s3", **http_link_config) # s1-eth2 s3-eth1
```

```
self.addLink("s3", "s4", **http_link_config)    # s3-eth2  s4-eth2

## Host: h1,h2 su s1; h3,h4 su s4
self.addLink("h1", "s1")    # → s1-eth3
self.addLink("h2", "s1")    # → s1-eth4
self.addLink("h3", "s4")    # → s4-eth3
self.addLink("h4", "s4")    # → s4-eth4
```

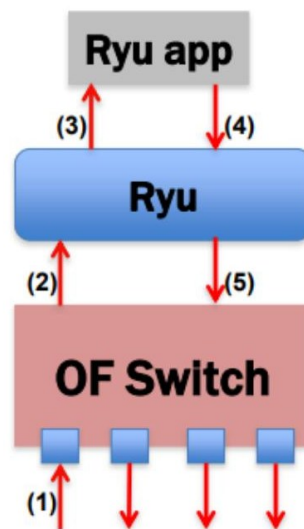
#### Tip – Ordine dei link = numeri di porta

In Mininet, le porte di uno switch vengono numerate nell'ordine in cui i link vengono aggiunti. Poiché S1 ottiene prima il link verso S2 (eth1), poi verso S3 (eth2), poi h1 (eth3) e h4 (eth4), le porte hanno esattamente questi numeri. Lo stesso vale per S4. Questo dettaglio è fondamentale per capire la tabella `slice_to_port` in `topologyslicing.py`.

La rete viene avviata con un **controller remoto** (Ryu) in ascolto su `127.0.0.1:6633`, e `autoStaticArp=True` per evitare che i pacchetti ARP broadcast debbano essere gestiti dalla logica di slicing.

### Programmare una rete SDN con Ryu

**Ryu** è un framework per controller SDN scritto in Python. Si basa su un modello a **eventi**: l'applicazione si registra per ricevere notifiche di specifici eventi di rete (arrivo di un pacchetto, connessione/disconnessione di uno switch, cambio di link) e reagisce installando o modificando le flow rule.



**Figura 12.4.** Architettura Ryu: Ryu app sopra il layer Ryu sopra l'OF Switch

*Fig. — Stack Ryu: la Ryu app riceve eventi (freccia ) e invia comandi (freccia ) al layer Ryu, che comunica con l'OF Switch tramite OpenFlow (freccia ).*

Una **Ryu app** è una classe Python che: 1. Estende `ryu.base.app_manager.RyuApp` 2. Dichiarare la versione di OpenFlow supportata (`OFP_VERSIONS`) 3. Registra handler per specifici eventi con il decoratore `@set_ev_cls`

**topologyslicing.py: la logica del controller**

**Inizializzazione e slice\_to\_port** Il cuore della logica di slicing è il dizionario `slice_to_port`, che mappa per ogni switch (identificato dal DPID) quale porta di uscita usare a seconda della porta di ingresso:

```
self.slice_to_port = {
    1: {1: 3, 3: 1, 2: 4, 4: 2}, # S1: eth1 eth3 (upper), eth2 eth4 (lower)
    4: {1: 3, 3: 1, 2: 4, 4: 2}, # S4: stesso schema di S1
    2: {1: 2, 2: 1},             # S2: forwarding diretto
    3: {1: 2, 2: 1},             # S3: forwarding diretto
}
```

**Tip – Lettura della tabella per S1**

Per S1 (dpid=1): se un pacchetto arriva su eth1 (da S2, upper path), viene inoltrato su eth3 (verso h1). Se arriva su eth3 (da h1), torna su eth1 (verso S2). Specularmente, eth2 (da S3, lower path) eth4 (verso h2). Il traffico di h1 non può mai raggiungere eth2 o eth4, garantendo l'isolamento.

La separazione è quindi **esclusivamente statistica su S1 e S4**: S2 e S3 non devono fare nulla di speciale, perché non ricevono mai traffico cross-slice (la separazione avviene prima di loro).

**Gestione degli eventi OpenFlow** La classe gestisce due eventi principali:

**switch\_features\_handler** (evento EventOFPSwitchFeatures, fase CONFIG\_DISPATCHER): viene invocato quando uno switch si connette al controller per la prima volta. Installa la **table-miss flow entry** con priorità 0 e match vuoto (corrisponde a qualsiasi pacchetto): l'azione è inviare al controller con OFPP\_CONTROLLER. Questa regola garantisce che i primi pacchetti di ogni flusso vengano consegnati all'applicazione.

**\_packet\_in\_handler** (evento EventOFPPacketIn, fase MAIN\_DISPATCHER): viene invocato ogni volta che uno switch riceve un pacchetto senza una flow rule corrispondente e lo manda al controller. La logica è:

1. Legge il DPID dello switch e la porta di ingresso
2. Consulta `slice_to_port` per determinare la porta di uscita
3. Installa una **flow rule permanente** con `add_flow` (così i pacchetti successivi dello stesso flusso non passeranno più dal controller)
4. Invia immediatamente il pacchetto corrente con `_send_package`

```
def _packet_in_handler(self, ev):
    msg = ev.msg
    datapath = msg.datapath
    in_port = msg.match["in_port"]
    dpid = datapath.id

    out_port = self.slice_to_port[dpid][in_port]

    actions = [datapath.ofproto_parser.OFPActionOutput(out_port)]
    match = datapath.ofproto_parser.OFPMatch(in_port=in_port)
    self.add_flow(datapath, 1, match, actions)
    self._send_package(msg, datapath, in_port, actions)
```

**add\_flow** costruisce e invia un messaggio OFPFlowMod al datapath (lo switch), che modifica la flow table installando la nuova regola. Da quel momento in poi, i pacchetti che corrispondono al match vengono gestiti direttamente dallo switch senza coinvolgere il controller.

**Nota – Reactive vs Proactive flow installation**

Questa applicazione usa l'approccio **reattivo**: le regole vengono installate al primo pacchetto. In alternativa, un controller **proattivo** potrebbe installare tutte le regole all'avvio (`switch_features_handler`)

senza aspettare traffico reale. Per questo schema di slicing statico, l'approccio proattivo sarebbe più efficiente poiché elimina la latenza del primo pacchetto.

### 12.1.5 Esercizi

#### Esercizio A: esecuzione e verifica

L'avvio richiede due terminali distinti (il controller e Mininet devono girare in parallelo):

```
## Terminale 1 - avvia il controller Ryu
ryu-manager topology_slicing.py

## Terminale 2 - avvia Mininet (richiede sudo)
sudo python3 network.py
```

Una volta avviata la rete, si possono eseguire le verifiche:

#### 1. Verifica dell'isolamento (ping)

Il ping deve avere successo solo tra host della stessa slice. Se la slicing funziona correttamente: - **h1 ping h3** → successo (upper slice) - **h2 ping h4** → successo (lower slice) - **h1 ping h2** → fallimento (cross-slice, non esiste path)

#### 2. Verifica della banda (iperf)

```
## Nel terminale Mininet
mininet> iperf h1 h3    # deve mostrare ~10 Mbps
mininet> iperf h2 h4    # deve mostrare ~1 Mbps
```

#### 3. Ispezione delle flow rule installate

```
mininet> sh ovs-ofctl dump-flows s1
```

Dopo il primo ping tra h1 e h3, devono essere visibili due flow entry su S1: - match **in\_port=1** → action **output:3** - match **in\_port=3** → action **output:1**

La regola di default (table-miss, priorità 0) ha action **CONTROLLER**, visibile anche prima del ping.

#### 4. Cattura e analisi dei messaggi OpenFlow

Per osservare il ciclo completo di comunicazione controller-switch, si può catturare il traffico sulla porta 6633 con tcpdump e analizzarlo in Wireshark:

```
## Apri un nuovo terminale
sudo tcpdump -i any port 6633 -w ofcapture.del.pcap

## Cancella le flow rule su s1 per forzare nuovi PACKET_IN
mininet> sh ovs-ofctl del-flows s1 table=0,in_port="s1-eth1"
mininet> sh ovs-ofctl del-flows s1 table=0,in_port="s1-eth3"

## Genera traffico e osserva i messaggi
mininet> h1 ping h3
```

In Wireshark si possono distinguere tre tipi di messaggi OpenFlow rilevanti: - **PACKET\_IN**: lo switch invia il pacchetto al controller (manca la flow rule) - **PACKET\_OUT**: il controller risponde con l'azione immediata per il pacchetto corrente - **FLOW\_MOD**: il controller installa la regola permanente sullo switch



## Capitolo 13

# Teoria dei Segnali: Trasformata di Fourier, Campionamento e DFT

Questa lezione è la continuazione diretta della [[Lezione 25 - (Lab) Teoria dei Segnali - Introduzione e Serie di Fourier|Lezione 25]], che ha introdotto la classificazione dei segnali, la trasmissione, la serie di Fourier e lo spettro. Qui si affronta il salto concettuale fondamentale: estendere l'analisi in frequenza ai segnali **non periodici** mediante la Trasformata di Fourier, poi si passa al problema pratico della conversione analogico-digitale e all'analisi spettrale di segnali campionati tramite la DFT.

---

### 13.1 Trasformata di Fourier

#### 13.1.1 Dal Discreto al Continuo: Passaggio dalla Serie alla Trasformata

La serie di Fourier, vista nella lezione precedente, funziona bene per segnali periodici: rappresenta il segnale come somma di armoniche a frequenze discrete  $nF = n/T$ . Ma la stragrande maggioranza dei segnali reali di interesse — impulsi, transitori, frammenti audio, burst radio — non sono periodici. Come estendere l'analisi in frequenza a questi segnali?

L'idea è elegante: si considera il segnale non periodico come il limite di un segnale periodico il cui periodo  $T$  tende all'infinito. Quando  $T$  cresce, la frequenza fondamentale  $F = 1/T$  diminuisce, e le armoniche discrete  $nF$  si avvicinano sempre di più tra loro. Nel limite  $T \rightarrow \infty$ , la spaziatura tra le armoniche  $\Delta_f = 1/T \rightarrow 0$  e lo spettro discreto diventa uno **spettro continuo**.

#### Definizione – Trasformata Continua di Fourier (CFT)

Un segnale non periodico  $s(t)$  può essere espresso come:

$$s(t) = \int_{-\infty}^{+\infty} S(f) e^{j2\pi ft} df$$

dove  $S(f)$  è la **densità spettrale complessa**, data dalla trasformata diretta:

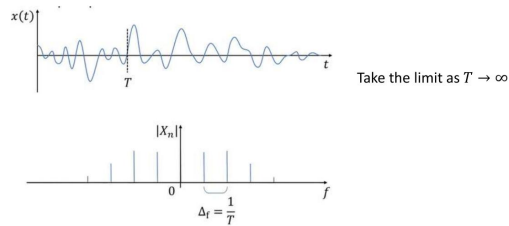
$$S(f) = \mathcal{F}(s(t)) = \int_{-\infty}^{+\infty} s(t) e^{-j2\pi ft} dt$$

Si scrive in forma compatta  $s(t) \iff S(f)$ .

A differenza della serie di Fourier — che produce un insieme **discreto** di coefficienti  $S_n$  — la CFT produce uno spettro **continuo**  $S(f)$  definito su tutte le frequenze reali  $f \in (-\infty, +\infty)$ .

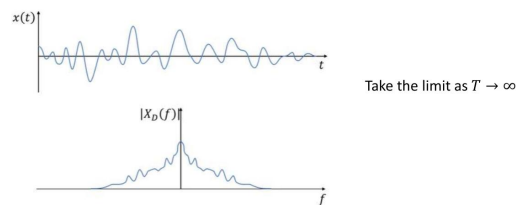
## Towards the continuous FT...

- **Fourier series** assume a signal of period  $T$ 
  - Complex amplitudes  $S_n$  at discrete frequencies  $nF = n/T$
  - Forward transform operator is an integral, reverse is a summation



**Figura 13.1.** Segnale periodico e relativo spettro discreto. All'aumentare di  $T$ , le righe spettrali si avvicinano. Prendendo il limite  $T \rightarrow \infty$  si ottiene lo spettro continuo.

## Towards the continuous FT...



**Figura 13.2.** Quando  $T \rightarrow \infty$ , il segnale "dimentica" la propria periodicit  e lo spettro  $|X_D(f)|$  diventa una funzione continua della frequenza.

### 13.1.2 Esempio: Segnale Pulsato

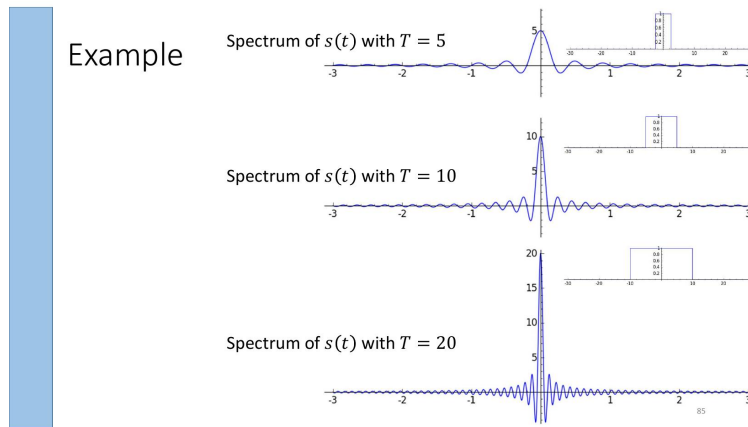
Consideriamo il segnale rettangolare:

$$s(t) = \begin{cases} 1 & \text{se } |t| \leq T/2 \\ 0 & \text{altrimenti} \end{cases}$$

La sua trasformata continua di Fourier si calcola analiticamente:

$$S(f) = \int_{-T/2}^{T/2} e^{-j2\pi ft} dt = T \cdot \text{sinc}(\pi f T) = \frac{\sin(\pi f T)}{\pi f}$$

Il risultato è una funzione **sinc** nel dominio della frequenza. Un fatto cruciale è che all'aumentare di  $T$  (impulso più largo nel tempo), lo spettro si restringe in frequenza, e viceversa. Questo è il principio di **incertezza tempo-frequenza**: un segnale non può essere contemporaneamente concentrato sia nel tempo che in frequenza.



**Figura 13.3.** Spettri del segnale pulsato al variare della durata  $T$ . All'aumentare di  $T$  il lobo principale della sinc si restringe, riflettendo il principio di incertezza tempo-frequenza.

### 13.1.3 Banda di un Segnale e Filtraggio

Lo spettro di un segnale generico si estende su tutte le frequenze in  $(-\infty, +\infty)$ . Tuttavia, nella pratica non tutte le componenti frequenziali sono ugualmente utili o necessarie. Filtrare un segnale significa **selezionare un sottoinsieme delle sue componenti spettrali**. Le motivazioni principali sono:

- alcune frequenze contengono l'informazione rilevante, altre corrispondono a rumore o interferenze
- i canali di comunicazione hanno banda disponibile limitata
- si vuole separare segnali che si sovrappongono in frequenza

I filtri principali sono: - **Filtro passa-basso** (*low-pass*): lascia passare le frequenze sotto una soglia  $f_c$ , blocca le alte frequenze - **Filtro passa-alto** (*high-pass*): lascia passare le frequenze sopra  $f_c$ , blocca le basse - **Filtro passa-banda** (*band-pass*): lascia passare le frequenze in un intervallo  $[f_1, f_2]$

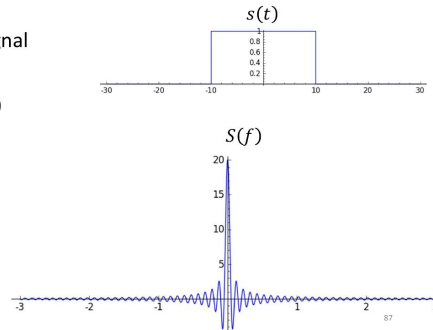
### Example

Consider again the pulse signal  
with  $T = 20$ :

$$s(t) = \begin{cases} 1 & \text{if } |t| \leq 10 \\ 0 & \text{otherwise} \end{cases}$$

and its spectrum:

$$S(f) = \frac{\sin(20 \cdot \pi \cdot f)}{\pi \cdot f}$$



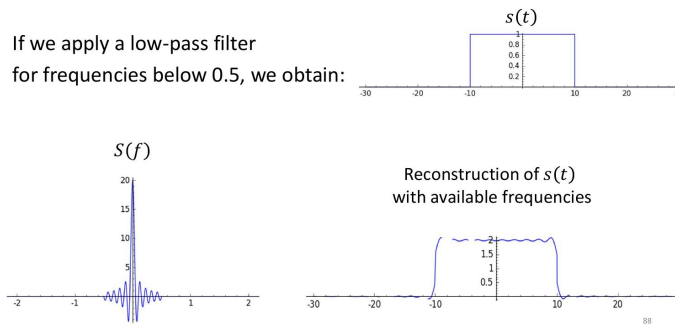
**Figura 13.4.** Segnale pulsato con  $T=20$ : il segnale nel dominio del tempo (rettangolo di ampiezza 1 e durata 20) e il corrispondente spettro  $S(f)=\sin(20\pi f)/(\pi f)$ .

### Tip – Filtraggio del segnale pulsato

Consideriamo il segnale rettangolare con  $T = 20$  e il suo spettro  $S(f) = \sin(20\pi f)/(\pi f)$ .

### Example

If we apply a low-pass filter  
for frequencies below 0.5, we obtain:



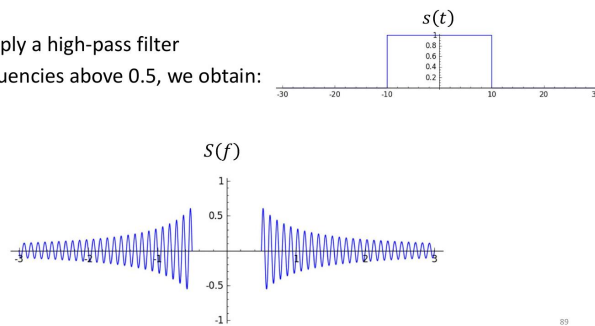
**Figura 13.5.** Filtro passa-basso (soglia 0.5 Hz): lo spettro viene troncato alle basse frequenze. La ricostruzione nel dominio del tempo riproduce approssimativamente la forma del rettangolo, con arrotondamento agli spigoli.

### Tip – Intuizione sul filtraggio

Applicare un filtro nel dominio della frequenza equivale a moltiplicare lo spettro  $S(f)$  per la risposta del filtro  $H(f)$ . Nel dominio del tempo questo corrisponde a una convoluzione del segnale con la risposta impulsiva del filtro. Le basse frequenze determinano il “contorno grossolano” del segnale; le alte frequenze determinano i dettagli fini e gli spigoli.

### Example

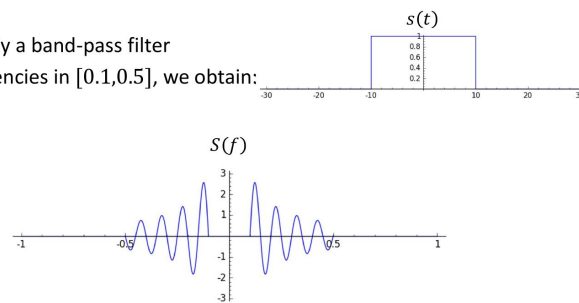
If we apply a high-pass filter  
for frequencies above 0.5, we obtain:



**Figura 13.6.** Filtro passa-alto (soglia 0.5 Hz): rimangono solo le componenti ad alta frequenza dello spettro. Il segnale ricostruito ha una forma oscillatoria concentrata ai bordi del rettangolo originale.

### Example

If we apply a band-pass filter  
for frequencies in  $[0.1, 0.5]$ , we obtain:



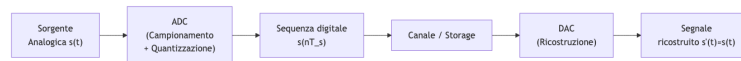
**Figura 13.7.** Filtro passa-banda (intervallo  $[0.1, 0.5]$  Hz): vengono selezionate le componenti in una fascia di frequenze. Il risultato è un segnale oscillante.

### 13.1.4 Confronto: Serie di Fourier vs Trasformata

|                 | Serie di Fourier               | Trasformata di Fourier               |
|-----------------|--------------------------------|--------------------------------------|
| Tipo di segnale | Periodico                      | Non periodico                        |
| Spettro         | Discreto (coefficienti $S_n$ ) | Continuo ( $S(f)$ )                  |
| Output          | Insieme di coefficienti        | Funzione continua di frequenza       |
| Esempi tipici   | Onda quadra, sinusoidi         | Impulsi, transienti, frammenti audio |

## 13.2 Da Segnale Analogico a Digitale: Campionamento e Quantizzazione

I sistemi digitali moderni (computer, smartphone, dispositivi IoT) lavorano esclusivamente con dati discreti. Un segnale analogico continuo, come una voce o un sensore di temperatura, deve essere convertito in una sequenza di valori discreti. Questo processo si articola in due fasi: **campionamento** e **quantizzazione**.



**Figura 13.8.** Pipeline di conversione analogico-digitale e digitale-analogica. L'ADC introduce inevitabilmente un errore di quantizzazione; il DAC ricostruisce un'approssimazione del segnale originale.

### 13.2.1 Campionamento

**Campionare** un segnale  $s(t)$  significa estrarre i valori che il segnale assume a istanti discreti regolarmente spazati. Il parametro chiave è il **periodo di campionamento**  $T_s$  (o equivalentemente la **frequenza di campionamento**  $f_s = 1/T_s$ ):

$$g(nT_s) = s(nT_s), \quad n \in \mathbb{Z}$$

Il risultato è una sequenza di numeri reali. A differenza della quantizzazione, il campionamento — sotto opportune ipotesi — non introduce perdita di informazione.

### 13.2.2 Quantizzazione

Mentre il campionamento opera sull'asse del tempo, la quantizzazione opera sull'asse dell'ampiezza: trasforma ogni campione reale  $x \in \mathbb{R}$  in un valore intero  $y \in [0, 2^R - 1]$ , dove  $R$  è il numero di bit del quantizzatore. Si usa un **quantizzatore scalare a  $R$  bit**:

- la **risoluzione** per un segnale con valori in  $[0, M]$  è  $M/2^R$
- il **bitrate** di una sorgente campionata a  $f_s$  e quantizzata a  $R$  bit è:  $B = R \cdot f_s$  bit/s

#### Attenzione – Errore di quantizzazione

La quantizzazione introduce un errore irreversibile: due valori analogici diversi possono essere mappati allo stesso livello digitale. Aumentare  $R$  riduce l'errore (migliore fedeltà), ma aumenta il bitrate da trasmettere o memorizzare.

#### Tip – Sistema telefonico analogico

La voce umana ha un contenuto informativo utile fino a circa 4 kHz (limite telefonico accettabile). Quindi: - frequenza di campionamento:  $f_s = 8$  kHz (doppio di 4 kHz, come vedremo con Nyquist) -

quantizzazione su  $R = 8$  bit per campione - bitrate risultante:  $B = 8 \times 8000 = 64$  kbps

### 13.2.3 Interpolazione e Ricostruzione

Dato un insieme di campioni, esistono vari metodi di ricostruzione del segnale analogico: - **Zero-order hold**: ogni campione viene “tenuto” fino al successivo (segnale a gradini) - **First-order hold**: interpolazione lineare tra campioni adiacenti - **Interpolazione con seno cardinale** (*sinc interpolation*): metodo ottimale, usato nella dimostrazione del teorema di Nyquist

## 13.3 Il Teorema di Campionamento di Nyquist-Shannon

La domanda centrale della conversione A/D è: **a quale frequenza minima devo campionare per poter ricostruire perfettamente il segnale originale?**

### Teorema – Teorema di Nyquist-Shannon

Sia  $s(t)$  un segnale con spettro nullo per frequenze superiori a  $f_M$  (segnale **a banda limitata**). Allora  $s(t)$  è completamente determinato dai suoi campioni presi all'intervallo:

$$T_s \leq \frac{1}{2f_M}$$

ovvero con frequenza di campionamento  $f_s \geq 2f_M$ . La frequenza minima  $f_N = 2f_M$  è chiamata **frequenza di Nyquist** o **tasso di Nyquist**.

La ricostruzione esatta si ottiene tramite interpolazione con seno cardinale:

$$s(t) = \sum_{n=-\infty}^{+\infty} s(nT_s) \cdot \frac{\sin(\pi f_s(t - nT_s))}{\pi f_s(t - nT_s)}$$

Il teorema richiede che il segnale sia **strettamente a banda limitata** — condizione che, come vedremo, non è mai soddisfatta dai segnali reali.

## 13.4 Aliasing: Il Problema del Sottocampionamento

Per capire cosa succede quando si campiona troppo lentamente, bisogna guardare allo spettro del segnale campionato nel dominio della frequenza.

### 13.4.1 FT del Segnale Campionato

Quando si campiona un segnale  $s(t)$  con frequenza  $f_s$ , la trasformata di Fourier del segnale campionato **consiste in infinite repliche** dello spettro originale  $S(f)$ , centrate alle frequenze multiple di  $f_s$ :

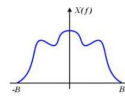
$$S_c(f) = f_s \sum_{k=-\infty}^{+\infty} S(f - kf_s)$$

La spaziatura tra le repliche è pari a  $f_s$ : **più veloce è il campionamento, più distanti sono le repliche.**

### Definizione – Aliasing

L'**aliasing** è la distorsione che si verifica quando le repliche spettrali generate dal campionamento si **sovrappongono** tra loro. Quando c'è sovrapposizione, le componenti frequenziali si mescolano e il

### FT of a band limited signal



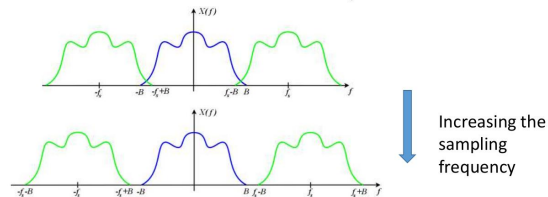
Fourier Transform of a band limited signal

- when a signal  $x(t)$  is sampled, the Fourier transform of the sampled signal consists of shifted and scaled copies of  $X(f)$

**Figura 13.9.** Trasformata di Fourier di un segnale a banda limitata  $B$ . Quando il segnale viene campionato, lo spettro si “replica” attorno a multipli della frequenza di campionamento  $f_s$ .

### FT of a band limited signal - sampled

- The spacing between the replicas of  $X(f)$  depends on the sampling rate
  - **the faster we sample, the further apart are the replicas of  $X(f)$ .**
- In general, the spectrum of the sampled signal  $s_c(t)$  consists of infinite replicas of the spectrum of  $s(t)$  centered at frequencies multiple of  $f_s$
- If we sample every  $T$  seconds then the spacing between the replicas in frequency domain is equal to the sampling frequency  $f_s = 1/T$



116

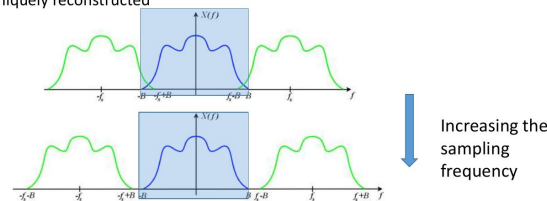
**Figura 13.10.** Aumentando la frequenza di campionamento, le repliche dello spettro si allontanano. Con campionamento sufficientemente veloce (in basso), le repliche non si sovrappongono e il segnale originale è recuperabile applicando un filtro passa-basso.



segnale originale non può più essere ricostruito univocamente.

### FT of a band limited signal – sampled and aliasing

- **Aliasing** is the distortion caused by **overlap between the replicated spectra** generated by sampling.
- After sampling, the spectrum of the signal is repeated around multiples of  $f_s$ .  
If the original signal is not sufficiently band-limited, or if  $f_s$  is too low, these replicas overlap.
- When overlap occurs, the frequency content is mixed, and the original signal cannot be uniquely reconstructed



**Figura 13.11.** Aliasing per frequenza di campionamento insufficiente: la spaziatura tra repliche è minore di  $2B$  e le code degli spettri si sovrappongono, rendendo impossibile la ricostruzione.

La condizione di Nyquist garantisce che **non ci sia sovrapposizione**: se  $f_s \geq 2f_M$ , le repliche distano almeno  $2B$  e possono essere separate con un filtro passa-basso ideale.

#### 13.4.2 Cosa Nyquist Non Ha Detto

Il teorema di Nyquist è spesso mal interpretato nella pratica. Ecco le trappole più comuni:

##### Attenzione – Limiti pratici del teorema di Nyquist

1. **Nessun segnale reale è strettamente a banda limitata.** Un segnale a banda strettamente limitata deve estendersi infinitamente nel tempo, il che è fisicamente impossibile. In pratica, i segnali hanno uno spettro che decade ma non si azzera mai.
2. **Campionare a  $2f_M$  non basta se non si usa un filtro anti-aliasing perfetto.** I filtri reali non hanno una risposta rettangolare ideale: frequenze superiori a  $f_M$  passano, seppur attenuate. Il risultato è un aliasing residuo non trascurabile.
3. **La soluzione pratica è l'oversampling:** campionare a una frequenza significativamente superiore a  $2f_M$ , e applicare un filtro anti-aliasing con taglio a  $f_M$  (anche se non ideale, l'attenuazione supplementare è sufficiente).

##### Tip – Oversampling nel telefono (old analog)

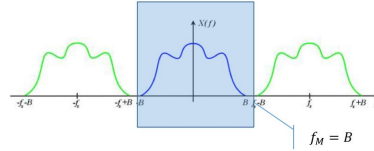
Il telefono analogico tradizionale tagliava la voce a  $\sim 3$  kHz (non 4 kHz) e campionava a 8 kHz: non il doppio del taglio (che sarebbe 6 kHz), ma un oversampling moderato. La differenza copre l'imperfezione del filtro e le componenti residue tra 3 e 4 kHz.

##### Nota – Downsampling e segnali periodici

Una curiosità: per segnali **strettamente periodici** e stabili, si può addirittura *sottocampionare* ( $f_s < 2f_M$ ). Ogni campione cade in una fase diversa del ciclo (perché il periodo di campionamento non è un multiplo del periodo del segnale). Raccogliendo abbastanza campioni nel tempo, si coprono tutte le fasi del segnale e la ricostruzione diventa possibile. Questo funziona **solo** se il segnale non varia nel tempo.

## Aliasing

That's why Nyquist requires that  $s(t)$  is limited in band!  
 If  $s(t)$  is strictly limited in band (i.e. it has a spectrum null at frequencies above  $f_M$ ), we can choose  $f_s$  to ensure that the replicas of the spectrum of  $s(t)$  do not overlap in the spectrum of the sampled signal.



118

**Figura 13.12.** Quando la condizione di Nyquist è soddisfatta ( $f_s = 2B = 2f_M$ ), le repliche dello spettro si toccano esattamente senza sovrapporsi: il segnale originale è recuperabile con un filtro ideale.

## 13.5 Trasformata Discreta di Fourier (DFT)

Nel contesto del **digital signal processing**, né la serie di Fourier (richiede un segnale periodico continuo) né la CFT (richiede un'integrazione continua) sono direttamente applicabili: si ha sempre un numero finito di campioni. La **DFT** è lo strumento adatto.

### 13.5.1 Motivazione

Data una sequenza finita di  $N$  campioni  $s_k$  (con  $k = 0, 1, \dots, N-1$ ), ottenuta campionando un segnale a frequenza  $f_s$  per un tempo  $T = N/f_s$ , si tratta il segnale discreto come se fosse **periodico con periodo  $T$**  e si calcola la serie di Fourier corrispondente. I coefficienti risultanti formano la DFT.

### 13.5.2 Definizione della DFT

#### Definizione – Trasformata Discreta di Fourier (DFT)

Data una sequenza  $s_k$  di  $N$  valori ( $k = 0, 1, \dots, N-1$ ):

**Trasformata diretta:**

$$S_n = \sum_{k=0}^{N-1} s_k e^{-j2\pi nk/N}, \quad n = 0, 1, \dots, N-1$$

**Trasformata inversa:**

$$s_k = \frac{1}{N} \sum_{n=0}^{N-1} S_n e^{j2\pi nk/N}$$

La frequenza del bin  $n$ -esimo è  $f_n = n \cdot \Delta f$ , dove la **risoluzione in frequenza** è:

$$\Delta f = \frac{f_s}{N} = \frac{1}{T}$$

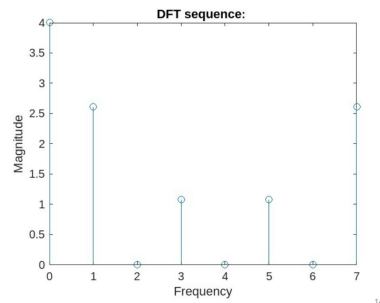
La DFT prende in input un array di  $N$  valori e restituisce un array di  $N$  valori complessi: entrambe le operazioni sono **somme** (non integrali), quindi direttamente implementabili su un computer.

**Tip – Esempio DFT**

Sequenza di 8 campioni:  $s[n] = 1$  per  $n = 0, 1, 2, 3$  e  $s[n] = 0$  per  $n = 4, 5, 6, 7$ .  
 I coefficienti DFT risultano: -  $S_0 = 4$  (componente DC) -  $S_1 = 1 - j2.414$  -  $S_2 = 1 - j0.414$   
 -  $S_3 = 1 + j0.414$  -  $S_4 = 1 + j2.414$  -  $S_5 = S_6 = S_7 = 0$  (per parità)

**Example**

Amplitude of the DFT



**Figura 13.13.** Spettro di ampiezza  $|S_n|$  della DFT dell'esempio. Il valore DC ( $n=0$ ) vale 4, poi i bin hanno ampiezze che decrescono. La simmetria attorno a  $N/2$  è caratteristica della DFT di segnali reali.

**13.5.3 Fast Fourier Transform (FFT)**

Il calcolo diretto della DFT richiede  $O(N^2)$  operazioni. Per sequenze lunghe (audio HD:  $N = 44100$  o più), questo è proibitivo. La **FFT** (Fast Fourier Transform) è una famiglia di algoritmi che sfruttano la struttura periodica dell'esponenziale complesso per ridurre la complessità a  $O(N \log N)$ .

**Nota – Condizione sulla lunghezza**

Per la FFT più efficiente (algoritmo di Cooley-Tukey),  $N$  deve essere una potenza di 2:  $N = 2^k$ . Per questo le frequenze di campionamento standard come 44100 Hz vengono spesso arrotondate a  $N = 32768$  o 65536 campioni per la trasformazione.

**13.5.4 Commenti sulla DFT**

**Risoluzione in frequenza.** La DFT non ha una risoluzione arbitraria: i “bin” sono spaziati di  $\Delta f = f_s/N$ . Se una componente del segnale cade tra due bin, la sua energia si spalma su più bin adiacenti (**spectral leakage**). Per migliorare la risoluzione si deve aumentare  $N$  (osservare il segnale più a lungo).

**Frequenza di Nyquist nella DFT.** L'analisi spettrale affidabile tramite DFT è possibile solo per frequenze  $f < f_s/2$ : le componenti oltre la frequenza di Nyquist vengono mappate a frequenze errate (aliasing). Filtri anti-aliasing vengono applicati prima del campionamento per rimuovere le componenti fuori banda.